

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP
HỌC PHẦN: KIẾN TRÚC PHẦN MỀM
Đề tài: Tiểu luận môn học V2

Giảng viên hướng dẫn : PGS.TS Trần Đình Quế
Họ và tên sinh viên : Bùi Thế Vĩnh Nguyên
Mã sinh viên : B22DCCN588
Nhóm : 08.02

Hà Nội – 2026

MỤC LỤC

CHƯƠNG 1. Từ Monolithic đến Microservices và DDD.....	1
1.1 Giới thiệu Monolithic Architecture	1
1.1.1 Khái niệm.....	1
1.1.2 Cấu trúc điển hình	1
1.1.3 Ví dụ thực tế.....	2
1.1.4 Nhược điểm chi tiết.....	4
1.1.5 Khi nào nên dùng Monolithic.....	4
1.2 Microservices Architecture	5
1.2.1 Khái niệm.....	5
1.2.2 Đặc điểm.....	5
1.2.3 So sánh Monolithic vs Microservices	6
1.2.4 Ưu điểm	9
1.2.5 Nhược điểm.....	10
1.2.6 Nguyên tắc thiết kế.....	10
1.3 Các trường hợp sử dụng thực tế (Case Studies)	11
1.3.1 Netflix: Tiên phong trong kiến trúc Microservices	11
1.3.2 Amazon: Cuộc cách mạng "Two-Pizza Team" và sự ra đời của AWS	13
1.4 Domain Driven Design (DDD)	14
1.4.1 Mục tiêu.....	14
1.4.2 Các khái niệm cốt lõi	14
1.4.3 Context Map.....	16
1.4.4 DDD trong Microservices.....	16
1.5 Case Study: Healthcare (Luyện Decomposition)	17
1.5.1 Mô tả bài toán.....	17

1.5.2 Bước 1: Xác định Domain	18
1.5.3 Bước 2: Xác định Bounded Context.....	18
1.5.4 Bước 3: Phân rã thành Microservices.....	20
1.5.5 Bước 4: Xác định quan hệ	21
1.5.6 Ví dụ API.....	22
1.5.7 Triển khai Healthcare	23
1.6 Kết luận	24
CHƯƠNG 2. Phát triển Hệ E-Commerce Microservices.....	26
2.1 Phân tích yêu cầu.....	26
2.1.1 Phân tích Use Case Khách hàng	26
2.1.2 Phân tích Use Case Nhân viên	27
2.1.3 Tương tác hệ thống hỗ trợ	28
2.1.4 Non-functional Requirements.....	28
2.2 Phân rã hệ thống thành các service.....	29
2.2.1 Xác định các Bounded Context	29
2.2.2 Shared Kernel	30
2.2.3 Xác định các lớp thực thể chi tiết cho từng Bounded Context	32
2.2.4 Thiết kế hệ thống Microservices theo Bounded Context.....	34
2.2.5 Xác định các lớp Controller cho từng Microservice	34
2.3 User Service	39
2.3.1 Sơ đồ lớp thiết kế	39
2.3.2 Data Model.....	40
2.3.3 Django Model	40
2.4 Staff Service	41
2.4.1 Sơ đồ lớp thiết kế	41
2.4.2 Data Model.....	42

2.4.3 Django Model	42
2.5 Product Service.....	43
2.5.1 Sơ đồ lớp thiết kế	43
2.5.2 Data Model.....	44
2.5.3 Django Model	45
2.5.4 Thiết kế API	46
2.5.5 Luồng xử lý	47
2.6 Cart Service	47
2.6.1 Sơ đồ lớp thiết kế	47
2.6.2 Data Model.....	48
2.6.3 Django Model	48
2.6.4 Thiết kế API	48
2.6.5 Luồng xử lý	49
2.7 Order Service	49
2.7.1 Sơ đồ lớp thiết kế	49
2.7.2 Data Model.....	50
2.7.3 Django Model	50
2.7.4 Thiết kế API	51
2.7.5 Luồng xử lý	51
2.8 Payment Service	52
2.8.1 Sơ đồ lớp thiết kế	52
2.8.2 Data Model.....	52
2.8.3 Django Model	53
2.8.4 Thiết kế API	54
2.8.5 Luồng xử lý	54

2.9 Shipping Service	55
2.9.1 Sơ đồ lớp thiết kế	55
2.9.2 Data Model.....	55
2.9.3 Django Model	56
2.9.4 Thiết kế API	57
2.9.5 Luồng xử lý	57
2.10 Comment Service	58
2.10.1 Sơ đồ lớp thiết kế	58
2.10.2 Data Model	58
2.10.3 Django Model	59
2.10.4 Thiết kế API	60
2.10.5 Luồng xử lý.....	60
2.11 Luồng hệ thống tổng thể.....	60
2.11.1 Luồng xem sản phẩm và Thêm vào giỏ hàng (Product View & Cart Flow)	60
2.11.2 Luồng mua sắm và Thanh toán (Purchase & Checkout Flow)	61
2.11.3 Hệ thống tư vấn thông minh (AI Chatbot Flow)	62
2.11.4 Theo dõi hành vi và Gợi ý (Behavior Tracking Flow).....	62
2.11.5 Quản lý Đánh giá và Bình luận (Product Review Flow)	63
2.12 Các hệ quản trị cơ sở dữ liệu được sử dụng.....	63
2.12.1 MySQL	63
2.12.2 PostgreSQL	64
2.12.3 So sánh MySQL và PostgreSQL	64
2.13 Kết luận	64
CHƯƠNG 3. AI Service cho tư vấn sản phẩm	65
3.1 Mô tả yêu cầu hệ thống AI	65

3.2 Deep Learning trong tư vấn sản phẩm	65
3.2.1 Các mô hình sử dụng	65
3.2.2 Cấu trúc mã nguồn và Quy trình huấn luyện	66
3.2.3 Dữ liệu thử nghiệm hiện tại	66
3.3 Triển khai (Deploy)	67
3.4 Kiến trúc RAG (Retrieval-Augmented Generation)	68
3.4.1 Pipeline RAG chi tiết	68
3.4.2 Cơ sở dữ liệu tri thức (Knowledge Base)	69
3.4.3 Chiến lược Hybrid Retrieval Fusion	73
3.4.4 Cơ chế Dual-Pipeline (Fast vs Full)	73
3.4.5 Tích hợp Deep Learning vào RAG	74
3.5 Tích hợp E-commerce	74
CHƯƠNG 4. Xây dựng hệ thống hoàn chỉnh	76
4.1 Mô tả kiến trúc hệ thống	76
4.1.1 Mô hình hệ thống (Microservices Architecture)	76
4.1.2 Nguyên tắc thiết kế cốt lõi	76
4.2 Các công nghệ sử dụng	77
4.3 Triển khai hệ thống	77
4.3.1 Cấu trúc dự án (System Structure)	77
4.3.2 Triển khai với Docker và Docker Compose	78
4.3.3 Điều phối tại API Gateway	79
4.4 Kết quả triển khai	80
4.4.1 Giao diện Quản trị và Dashboard	80
4.4.2 Luồng mua hàng hoàn chỉnh (End-to-End)	81

DANH MỤC HÌNH VẼ

Hình 1.1	Biểu đồ lớp pha phân tích hệ thống BookStore	2
Hình 1.2	Biểu đồ lớp pha thiết kế hệ thống BookStore	3
Hình 1.3	Kiến trúc Microservices của hệ thống BookStore	7
Hình 1.4	Cấu trúc chi tiết của Service quản lý sách (BookService)	9
Hình 1.5	Cấu trúc chi tiết của Service xử lý đơn hàng (OrderService)	9
Hình 1.6	Minh họa mô hình Bounded Context	17
Hình 1.7	Mô hình Bounded Context cho hệ thống Healthcare	20
Hình 1.8	Sơ đồ mối quan hệ giữa các Microservices trong hệ thống Healthcare	22
Hình 1.9	Cấu trúc tổ chức mã nguồn của hệ thống Healthcare	24
Hình 1.10	Khai báo Model thực tế cho các Entity trong Healthcare Service	24
Hình 2.1	Sơ đồ Use Case tổng quan cho Khách hàng	26
Hình 2.2	Sơ đồ Use Case tổng quan cho Nhân viên	28
Hình 2.3	Mô hình phân rã Bounded Context của hệ thống theo DDD	31
Hình 2.4	Sơ đồ các lớp thực thể (Entity Classes) trong pha phân tích	33
Hình 2.5	Sơ đồ mối quan hệ và tương tác giữa các Microservices	39
Hình 2.6	Sơ đồ lớp thiết kế User Service	40
Hình 2.7	Sơ đồ Data Model User Service	40
Hình 2.8	Cấu trúc mã nguồn Model của User Service	41
Hình 2.9	Sơ đồ lớp thiết kế Staff Service	41
Hình 2.10	Sơ đồ Data Model Staff Service	42
Hình 2.11	Cấu trúc mã nguồn Model của Staff Service	43
Hình 2.12	Sơ đồ lớp thiết kế Product Service	44
Hình 2.13	Sơ đồ Data Model Product Service	45
Hình 2.14	Cấu trúc mã nguồn Model của Product Service	46
Hình 2.15	Sơ đồ lớp thiết kế Cart Service	47
Hình 2.16	Sơ đồ Data Model Cart Service	48
Hình 2.17	Cấu trúc mã nguồn Model của Cart Service	48
Hình 2.18	Sơ đồ lớp thiết kế Order Service	49
Hình 2.19	Sơ đồ Data Model Order Service	50
Hình 2.20	Cấu trúc mã nguồn Model của Order Service	51
Hình 2.21	Sơ đồ lớp thiết kế Payment Service	52
Hình 2.22	Sơ đồ Data Model Payment Service	53
Hình 2.23	Cấu trúc mã nguồn Model của Payment Service	54
Hình 2.24	Sơ đồ lớp thiết kế Shipping Service	55

Hình 2.25	Sơ đồ Data Model Shipping Service	56
Hình 2.26	Cấu trúc mã nguồn Model của Shipping Service	57
Hình 2.27	Sơ đồ lớp thiết kế Comment Service	58
Hình 2.28	Sơ đồ Data Model Comment Service	59
Hình 2.29	Cấu trúc mã nguồn Model của Comment Service	59
Hình 2.30	Sơ đồ tuần tự luồng xem sản phẩm và thêm vào giỏ hàng	61
Hình 2.31	Sơ đồ tuần tự luồng mua sắm và thanh toán	61
Hình 2.32	Sơ đồ tuần tự luồng tư vấn AI	62
Hình 2.33	Sơ đồ tuần tự luồng theo dõi hành vi	63
Hình 2.34	Sơ đồ tuần tự luồng đánh giá sản phẩm	63
Hình 3.1	Kiến trúc tổng thể của AI Service và các thành phần chính	65
Hình 3.2	Mô phỏng quy trình và mã nguồn huấn luyện mô hình	66
Hình 3.3	Dữ liệu hành vi mẫu được sử dụng để huấn luyện mô hình	67
Hình 3.4	Cấu trúc thư mục triển khai AI Service	68
Hình 3.5	Quy trình Hybrid Retrieval và tích hợp LLM trong hệ thống RAG	69
Hình 3.6	Mô hình đồ thị tri thức người dùng - sản phẩm	70
Hình 3.7	Câu lệnh Cypher tạo thực thể và quan hệ	71
Hình 3.8	Truy vấn gợi ý sản phẩm dựa trên quan hệ đồ thị	72
Hình 3.9	Truy vấn lọc sản phẩm qua quan hệ IN_CATEGORY	72
Hình 3.10	Minh họa các cụm tương tác đa người dùng trên đồ thị	73
Hình 3.11	Giao diện hiển thị sản phẩm gợi ý tại trang chủ	74
Hình 3.12	Giao diện Chatbot tương tác và tư vấn sản phẩm	75
Hình 4.1	Mô hình kiến trúc Microservices của hệ thống ecom-final	76
Hình 4.2	Cấu trúc tổ chức mã nguồn hệ thống	78
Hình 4.3	Cấu hình điều phối các dịch vụ bằng Docker Compose	79
Hình 4.4	Cấu hình Routing và Proxy tại API Gateway	80
Hình 4.5	Giao diện quản lý sản phẩm tại trung tâm vận hành	81
Hình 4.6	Tính năng chỉnh sửa thông tin sản phẩm và thuộc tính động	81
Hình 4.7	Minh họa luồng thanh toán trên giao diện khách hàng	82

DANH MỤC BẢNG BIỂU

Bảng 1.1	So sánh tổng quan kiến trúc Monolithic và Microservices	6
Bảng 1.2	So sánh cụ thể hệ thống BookStore giữa Monolithic và Microservices	8
Bảng 1.3	Tóm tắt timeline phát triển kiến trúc tại Netflix	13
Bảng 1.4	So sánh đặc trưng Microservices giữa Amazon và Netflix	14
Bảng 1.5	Phân rã Bounded Context thành Microservices	21
Bảng 1.6	Các API chính trong hệ thống Healthcare	23
Bảng 2.1	Bảng thiết kế API của Product Service	46
Bảng 2.2	Bảng thiết kế API của Cart Service	49
Bảng 2.3	Bảng thiết kế API của Order Service	51
Bảng 2.4	Bảng thiết kế API của Payment Service	54
Bảng 2.5	Bảng thiết kế API của Shipping Service	57
Bảng 2.6	Bảng thiết kế API của Comment Service	60
Bảng 2.7	So sánh MySQL và PostgreSQL trong hệ thống	64
Bảng 3.1	Vai trò của các nhánh trong Hybrid Retrieval	73

CHƯƠNG 1. Từ Monolithic đến Microservices và DDD

1.1 Giới thiệu Monolithic Architecture

1.1.1 Khái niệm

Monolithic Architecture (kiến trúc nguyên khối) là một phong cách thiết kế phần mềm trong đó toàn bộ hệ thống được xây dựng, vận hành và triển khai dưới dạng một ứng dụng duy nhất. Trong mô hình này, tất cả các thành phần chức năng của hệ thống, bao gồm giao diện người dùng, xử lý nghiệp vụ và truy xuất dữ liệu, đều được tích hợp trong cùng một codebase và hoạt động trong cùng một tiến trình.

Đặc trưng quan trọng của kiến trúc monolithic là tính liên kết chặt chẽ (tight coupling) giữa các module. Các thành phần trong hệ thống thường không có ranh giới rõ ràng, dẫn đến việc phụ thuộc lẫn nhau ở mức cao. Khi một phần của hệ thống thay đổi, các phần khác có thể bị ảnh hưởng trực tiếp hoặc gián tiếp.

Ngoài ra, toàn bộ hệ thống được triển khai như một đơn vị duy nhất (single deployment unit). Điều này có nghĩa là bất kỳ thay đổi nào, dù nhỏ, cũng yêu cầu phải build và deploy lại toàn bộ ứng dụng.

1.1.2 Cấu trúc điển hình

Kiến trúc monolithic thường được tổ chức theo mô hình phân tầng (Layered Architecture) nhằm tách biệt các trách nhiệm trong hệ thống. Một cấu trúc điển hình bao gồm các tầng sau:

- **Presentation Layer (tầng trình bày):** chịu trách nhiệm xử lý giao diện người dùng hoặc các API endpoint, tiếp nhận request và trả về response.
- **Business Logic Layer (tầng nghiệp vụ):** thực hiện các xử lý logic chính của hệ thống, bao gồm các quy tắc nghiệp vụ và luồng xử lý.
- **Data Access Layer (tầng truy xuất dữ liệu):** đảm nhiệm việc tương tác với cơ sở dữ liệu thông qua các thao tác như truy vấn, thêm, sửa, xóa dữ liệu.
- **Database:** là cơ sở dữ liệu dùng chung cho toàn bộ hệ thống.

Trong mô hình này, các tầng thường tương tác với nhau theo chiều từ trên xuống dưới. Các lời gọi giữa các thành phần diễn ra trong cùng một tiến trình (in-process communication), do đó có hiệu năng cao nhưng đồng thời cũng làm gia tăng mức độ phụ thuộc giữa các thành phần.

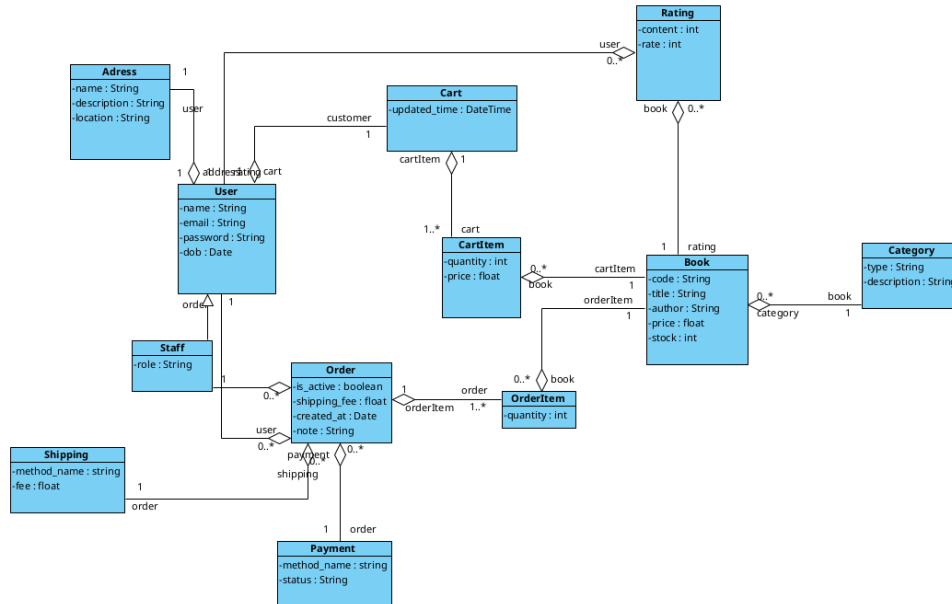
Một đặc điểm đáng chú ý khác là việc sử dụng cơ sở dữ liệu dùng chung (shared database). Tất cả các module đều truy cập và thao tác trên cùng một schema dữ liệu, điều này giúp đơn giản hóa việc quản lý dữ liệu ban đầu nhưng lại gây khó khăn trong việc mở

rộng và tái cấu trúc hệ thống về sau.

1.1.3 Ví dụ thực tế

a, Phân tích hệ thống BookStore

Hệ BookStore sử dụng kiến trúc monolithic có biểu đồ lớp pha phân tích như sau:



Hình 1.1: Biểu đồ lớp pha phân tích hệ thống BookStore

Nhóm quản lý chủ thể (Actors & Entities)

- **User:** Thực thể đại diện cho tất cả người dùng trong hệ thống. Chứa các thông tin cơ bản như name, email, password, và ngày sinh (dob).
- **Staff:** Là một dạng người dùng đặc biệt (kế thừa từ User) có vai trò (role) quản lý trong hệ thống.
- **Address:** Lưu trữ thông tin vị trí chi tiết của người dùng.

Nhóm quản lý kho hàng (Product Management)

- **Book:** Trung tâm của hệ thống, lưu trữ mã sách (code), tiêu đề, tác giả, giá và số lượng tồn kho (stock).
- **Category:** Dùng để phân loại sách theo các tiêu chí khác nhau.
- **Rating:** Cho phép người dùng đánh giá (rate) và để lại nội dung (content) phản hồi về sản phẩm.

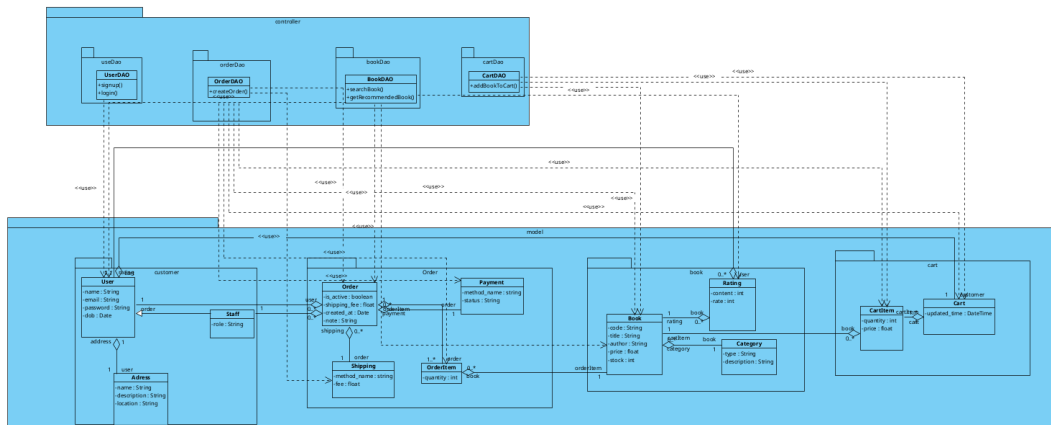
Nhóm quy trình mua hàng (Sales Process)

- **Cart & CartItem:** Giỏ hàng tạm thời của khách hàng. Một Cart có thể chứa nhiều

CartItem, mỗi mục liên kết trực tiếp với một Book và số lượng cụ thể.

- **Order & OrderItem:** Chuyển đổi từ giỏ hàng sang giao dịch chính thức. Order lưu trữ ngày tạo, phí vận chuyển (shipping_fee) và trạng thái đơn hàng.
- **Payment & Shipping:** Các thành phần xác định phương thức thanh toán và giao hàng để hoàn tất chu trình bán hàng.

b, Cấu trúc triển khai dạng Monolithic



Hình 1.2: Biểu đồ lớp pha thiết kế hệ thống BookStore

Cấu trúc đóng gói (Packaging Strategy)

Hệ thống được thiết kế theo kiến trúc nhiều tầng logic và triển khai trong một tiến trình thực thi thống nhất của framework Django:

- **Layer Model (Entities):** Đóng vai trò biểu diễn và quản lý dữ liệu của hệ thống. Trong Django, tầng này được triển khai thông qua Django Models để định nghĩa các thực thể như Book, Order, User. Các lớp này liên kết với nhau thông qua các quan hệ như ForeignKey, OneToOneField, ManyToManyField và được ánh xạ trực tiếp xuống các bảng trong MySQL thông qua Django ORM.
- **Layer Controller (Views/Logic):** Đóng vai trò tiếp nhận yêu cầu từ người dùng, xử lý logic nghiệp vụ và điều phối dữ liệu giữa giao diện và cơ sở dữ liệu. Trong Django, tầng này được triển khai chủ yếu thông qua Views kết hợp với URL Configuration (URL conf) để tiếp nhận HTTP Request, xử lý nghiệp vụ và tương tác với ORM trước khi trả về HTTP Response tương ứng.
- **Layer Presentation (Templates/UI):** Đóng vai trò hiển thị dữ liệu và xây dựng giao diện tương tác với người dùng. Trong Django, tầng này được triển khai thông qua Django Template Engine để tạo các trang HTML động, nhận dữ liệu từ Views và hiển thị nội dung theo mô hình tách biệt giữa giao diện và xử lý nghiệp vụ.

Đặc điểm triển khai thiết kế

Thiết kế Monolithic mang lại các đặc trưng vận hành cốt lõi:

- **Giao tiếp nội bộ tốc độ cao:** Lời gọi hàm trực tiếp trong cùng tiến trình giúp loại bỏ hoàn toàn độ trễ mạng.
- **Cơ sở dữ liệu tập trung & Nhất quán (ACID):** Dùng chung một Database, dễ dàng thực hiện các transaction an toàn tuyệt đối.
- **Triển khai & Mở rộng đơn nhất:** Build và deploy thành một khối duy nhất, mở rộng bằng cách nhân bản toàn bộ hệ thống qua Load Balancer.
- **Chia sẻ chung tài nguyên & Công nghệ:** Các module dùng chung RAM, CPU và thống nhất trên một tech-stack duy nhất (như Django/Python).

1.1.4 Nhược điểm chi tiết

Mặc dù kiến trúc Monolithic có ưu điểm về tính đơn giản ban đầu, mô hình này vẫn tồn tại những hạn chế cốt lõi khi hệ thống mở rộng:

- **Khó mở rộng và lãng phí tài nguyên:** Scale toàn bộ ứng dụng thay vì từng chức năng cụ thể, gây tiêu tốn tài nguyên vô ích cho các module ít sử dụng.
- **Mức độ phụ thuộc cao (Tight Coupling):** Lỗi ở một module nhỏ (ví dụ: Cart) có thể gây treo toàn bộ tiến trình hệ thống.
- **Triển khai rủi ro và chậm chạp:** Một thay đổi nhỏ cũng yêu cầu build và deploy lại toàn bộ dự án. Thời gian chạy CI/CD và test tăng mạnh.
- **Kém linh hoạt về công nghệ:** Toàn bộ hệ thống bị ràng buộc chặt chẽ vào một framework/ngôn ngữ (ví dụ: chỉ dùng Django/Python).
- **Khó làm việc nhóm quy mô lớn:** Nhiều nhóm phát triển chia sẻ chung một codebase dễ gây xung đột (merge conflicts) và cản trở tiến độ.

1.1.5 Khi nào nên dùng Monolithic

Mặc dù tồn tại nhiều hạn chế về khả năng mở rộng, Monolithic vẫn phù hợp trong nhiều trường hợp nhờ tính đơn giản và dễ triển khai.

- **Dự án quy mô nhỏ hoặc trung bình:** Phù hợp với hệ thống chưa có quá nhiều chức năng hoặc người dùng.
- **Phát triển MVP:** Giúp xây dựng sản phẩm mẫu nhanh để kiểm chứng ý tưởng.
- **Nhóm phát triển nhỏ:** Dễ quản lý codebase và phối hợp hơn so với nhiều service độc lập.
- **Nghị vụ chưa phức tạp:** Không cần thiết phải phân tách thành nhiều service.

- **Yêu cầu triển khai nhanh:** Quy trình build và deploy đơn giản hơn.
- **Thiếu kinh nghiệm hệ phân tán:** Giảm độ phức tạp vận hành so với Microservices.
- **Ưu tiên tính nhất quán dữ liệu:** Dễ đảm bảo transaction và ACID với database tập trung.
- **Ứng dụng nội bộ doanh nghiệp:** Thường không yêu cầu scale quá lớn.
- **Chi phí hạ tầng hạn chế:** Tiết kiệm tài nguyên và chi phí triển khai.
- **Dự án học tập và nghiên cứu:** Giúp tập trung vào logic nghiệp vụ thay vì hạ tầng phân tán.
- **Cần hiệu năng giao tiếp nội bộ cao:** Các module giao tiếp trực tiếp nên độ trễ thấp.
- **Domain chưa ổn định:** Dễ thay đổi và tái cấu trúc hơn trong giai đoạn đầu phát triển.

1.2 Microservices Architecture

1.2.1 Khái niệm

Microservices Architecture (kiến trúc vi dịch vụ) là một phong cách thiết kế phần mềm trong đó hệ thống được chia thành nhiều dịch vụ nhỏ (services) hoạt động độc lập. Mỗi service chịu trách nhiệm cho một domain nghiệp vụ riêng biệt và có thể được phát triển, triển khai cũng như mở rộng độc lập với các service khác.

Khác với kiến trúc Monolithic, các thành phần trong Microservices không chạy chung trong một tiến trình mà giao tiếp với nhau thông qua mạng bằng các giao thức như HTTP/REST, gRPC hoặc Message Broker (RabbitMQ, Kafka).

Mỗi microservice thường sở hữu cơ sở dữ liệu riêng, giúp giảm sự phụ thuộc giữa các module và tăng tính tự chủ trong quản lý dữ liệu. Điều này cho phép từng service sử dụng công nghệ, ngôn ngữ lập trình hoặc database phù hợp nhất với yêu cầu nghiệp vụ của nó.

Một đặc điểm quan trọng của Microservices là khả năng triển khai độc lập (independent deployment). Khi thay đổi một service, hệ thống không cần build hoặc deploy lại toàn bộ ứng dụng như trong Monolithic.

Kiến trúc Microservices thường được áp dụng trong các hệ thống lớn, có nhiều nhóm phát triển và yêu cầu mở rộng linh hoạt theo từng chức năng nghiệp vụ.

1.2.2 Đặc điểm

Kiến trúc Microservices tập trung vào tính phân tán và tự chủ cao của từng thành phần:

- **Phân tách theo Domain:** Hệ thống chia thành các service độc lập, mỗi service đảm nhiệm một nghiệp vụ (ví dụ: OrderService, User Service).

- **Giao tiếp qua mạng & Database độc lập:** Trao đổi dữ liệu qua API/Message Broker; mỗi service sở hữu cơ sở dữ liệu riêng biệt.
- **Triển khai & Scale độc lập:** Cho phép cập nhật hoặc mở rộng tải riêng cho từng service mà không ảnh hưởng đến toàn hệ thống.
- **Đa dạng công nghệ (Polyglot):** Có thể áp dụng các ngôn ngữ lập trình và database khác nhau cho từng chức năng đặc thù.
- **Vận hành phức tạp:** Đòi hỏi hạ tầng phân tán mạnh mẽ (Docker, Kubernetes, API Gateway, Service Discovery, Monitoring tập trung).

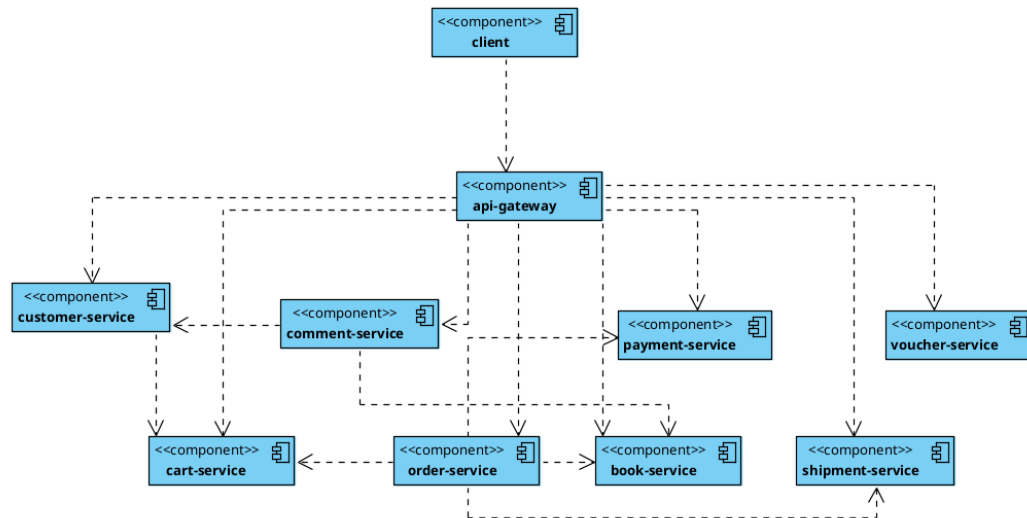
1.2.3 So sánh Monolithic vs Microservices

Tiêu chí	Monolithic	Microservices
Kiến trúc & Codebase	Một ứng dụng, chung một codebase.	Nhiều service độc lập, mỗi service một codebase riêng.
Giao tiếp & Dữ liệu	Gọi hàm trực tiếp, chung một database. Dễ đảm bảo ACID.	Giao tiếp qua mạng (API/gRPC), mỗi service có database riêng.
Triển khai & CI/CD	Deploy toàn bộ hệ thống cùng lúc. Khó linh hoạt cập nhật.	Deploy độc lập từng service. Hỗ trợ CI/CD hiệu quả.
Mở rộng & Tài nguyên	Phải scale toàn bộ ứng dụng, dễ lãng phí tài nguyên.	Scale linh hoạt chỉ các service cần thiết, tối ưu tài nguyên.
Bảo trì & Xử lý lỗi	Khó bảo trì. Một module lỗi có thể sập cả hệ thống.	Dễ bảo trì theo từng service. Lỗi được cô lập.
Công nghệ & Vận hành	Dùng một stack công nghệ. Dễ thiết lập nhưng kém linh hoạt.	Đa dạng công nghệ. Yêu cầu vận hành phức tạp (Docker, K8s).

Bảng 1.1: So sánh tổng quan kiến trúc Monolithic và Microservices

a, So sánh cụ thể với hệ thống BookStore

Dưới đây là kiến trúc của hệ thống BookStore khi được chuyển đổi sang mô hình Microservices:



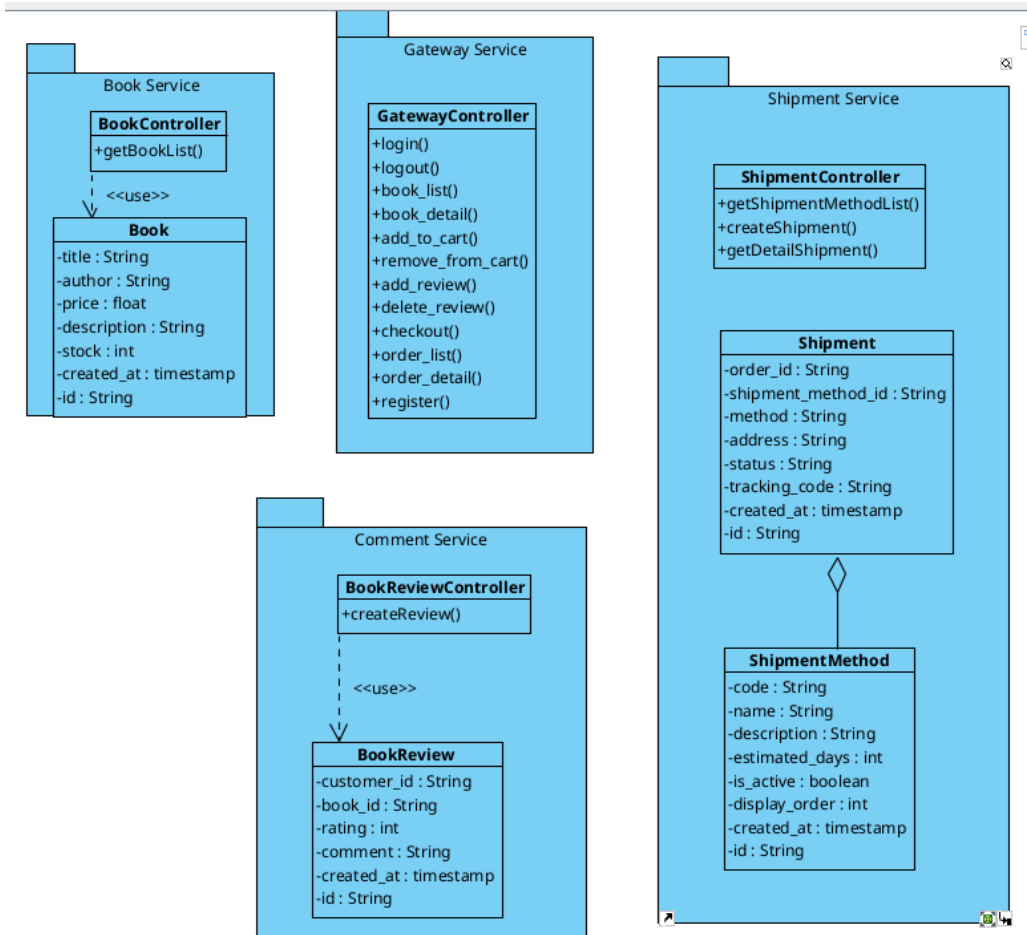
Hình 1.3: Kiến trúc Microservices của hệ thống BookStore

Dựa trên kiến trúc này, chúng ta có thể so sánh chi tiết giữa phiên bản Monolithic và phiên bản Microservices của hệ thống BookStore như sau:

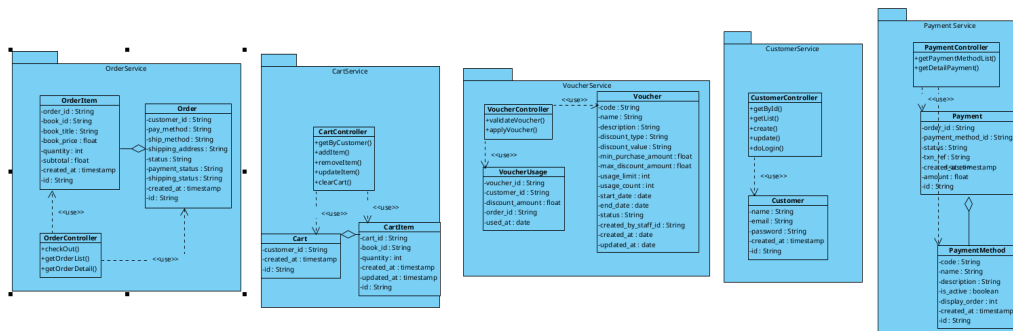
Tiêu chí	Bookstore Monolith (Django & MySQL)	Bookstore Microservices (Gateway, Book, Order...)
Tính phụ thuộc code	Chặt chẽ. Các lớp Book, User, Order nằm chung project, gọi hàm trực tiếp.	Độc lập. Mỗi dịch vụ như BookService, CommentService là một dự án riêng.
Cơ sở dữ liệu	Tập trung. Một database MySQL duy nhất cho toàn bộ hệ thống.	Phân tán. Mỗi dịch vụ có thể sở hữu database riêng để tối ưu dữ liệu.
Giao tiếp hệ thống	Nội bộ. Thông qua việc gọi hàm giữa các lớp (In-process calls).	Qua mạng. Sử dụng API Gateway để điều phối và các dịch vụ gọi nhau qua REST/gRPC.
Quản lý giao dịch	Đơn giản. Dùng transaction.atomic để đảm bảo tính nhất quán (ACID).	Phức tạp. Cần các mô hình như Saga để xử lý giao dịch trên nhiều dịch vụ.
Triển khai (Deploy)	Một tệp duy nhất. Phải build và chạy lại cả dự án dù chỉ sửa một lỗi nhỏ.	Từng phần. Có thể update VoucherService mà không ảnh hưởng PaymentService.
Khả năng mở rộng	Theo cụm. Phải nhân bản toàn bộ project lên nhiều server.	Chọn lọc. Chỉ tăng tài nguyên cho dịch vụ nào đang bị quá tải (ví dụ: BookService).
Độ phức tạp hạ tầng	Thấp. Dễ cài đặt, dễ chạy trên môi trường local để phát triển.	Cao. Cần các công cụ quản lý như Docker, Kubernetes, API Gateway.
Chịu lỗi (Reliability)	Kém. Một module (như Cart) bị treo có thể kéo sập toàn bộ hệ thống.	Tốt. Nếu CommentService lỗi, người dùng vẫn có thể đặt hàng bình thường.

Bảng 1.2: So sánh cụ thể hệ thống BookStore giữa Monolithic và Microservices

Các cấu trúc bên trong của từng service cũng được thiết kế theo hướng chuyên biệt và độc lập:



Hình 1.4: Cấu trúc chi tiết của Service quản lý sách (BookService)



Hình 1.5: Cấu trúc chi tiết của Service xử lý đơn hàng (OrderService)

1.2.4 Ưu điểm

Kiến trúc Microservices mang lại nhiều lợi ích trong việc phát triển và vận hành các hệ thống lớn, đặc biệt với các ứng dụng cần mở rộng linh hoạt.

- **Khả năng mở rộng linh hoạt & tối ưu tài nguyên:** Scale riêng lẻ các service có tải cao thay vì toàn bộ hệ thống.
- **Triển khai độc lập & CI/CD:** Mỗi service có thể được build, test và deploy độc lập,

giúp việc phát hành tính năng mới cực kỳ nhanh chóng.

- **Khả năng cô lập lỗi (Fault Isolation):** Lỗi hoặc sự cố ở một service (ví dụ: CommentService) không làm ảnh hưởng đến các nghiệp vụ quan trọng khác (như OrderService).
- **Đa dạng công nghệ (Polyglot):** Tự do sử dụng ngôn ngữ/database phù hợp nhất cho từng domain nghiệp vụ.
- **Dễ dàng làm việc nhóm:** Các team phát triển độc lập theo từng chức năng nghiệp vụ, giảm thiểu xung đột.

1.2.5 Nhược điểm

Bên cạnh các ưu điểm về mở rộng và linh hoạt, Microservices mang lại độ phức tạp khổng lồ về mặt phân tán:

- **Độ phức tạp vận hành & hạ tầng:** Cần thiết lập DevOps mạnh, quản lý container (Docker, Kubernetes) và hệ thống giám sát tập trung (Monitoring, Logging).
- **Khó khăn về nhất quán dữ liệu:** Vì dữ liệu phân tán ở nhiều database, việc đảm bảo ACID là bất khả thi, phải xử lý các Transaction phân tán (ví dụ: mô hình Saga).
- **Chi phí giao tiếp qua mạng (Network Overhead):** Giao tiếp REST/gRPC gây trễ, tốn chi phí parse dữ liệu so với gọi hàm nội bộ, và có rủi ro network failure.
- **Khó debug & tracing:** Việc truy vết lỗi khi request đi qua chuỗi hàng loạt các service khác nhau là một thách thức lớn.
- **Overkill cho dự án nhỏ:** Chuyển đổi sang Microservices khi dự án quy mô nhỏ sẽ gây tốn kém thời gian và chi phí không cần thiết.

1.2.6 Nguyên tắc thiết kế

Trong kiến trúc Microservices, việc phân tách hệ thống thành nhiều service độc lập cần tuân theo một số nguyên tắc thiết kế nhằm giảm sự phụ thuộc giữa các thành phần, đồng thời tăng khả năng mở rộng và bảo trì hệ thống.

Single Responsibility Principle

Mỗi service chỉ nên đảm nhiệm một nhóm chức năng nghiệp vụ cụ thể và có một lý do duy nhất để thay đổi. Việc giới hạn phạm vi trách nhiệm giúp service trở nên nhỏ gọn, dễ phát triển và dễ kiểm thử hơn.

Ví dụ, trong hệ thống BookStore:

- **User Service** quản lý thông tin và xác thực người dùng.
- **Order Service** xử lý nghiệp vụ đơn hàng.

- **Payment Service** xử lý thanh toán.

Khi mỗi service chỉ tập trung vào một domain riêng biệt, việc thay đổi hoặc mở rộng chức năng sẽ ít ảnh hưởng tới các thành phần khác của hệ thống.

Loose Coupling

Các service cần được thiết kế với mức độ phụ thuộc thấp nhằm tránh ảnh hưởng dây chuyền giữa các thành phần. Việc giao tiếp giữa các service thường được thực hiện thông qua REST API, gRPC hoặc Message Broker thay vì truy cập trực tiếp vào dữ liệu của nhau.

Nguyên tắc này giúp:

- Tăng khả năng triển khai độc lập.
- Giảm ảnh hưởng khi thay đổi hệ thống.
- Hỗ trợ mở rộng và thay thế service dễ dàng hơn.

Ví dụ, **Order Service** không truy cập trực tiếp vào database của **User Service** mà chỉ lấy dữ liệu người dùng thông qua API được cung cấp.

High Cohesion

Các chức năng liên quan chặt chẽ về nghiệp vụ nên được nhóm trong cùng một service nhằm tăng tính nhất quán và dễ bảo trì.

Một service có tính kết dính cao thường:

- Có phạm vi trách nhiệm rõ ràng.
- Chứa các logic nghiệp vụ liên quan trực tiếp với nhau.
- Dễ kiểm thử và quản lý hơn.

Ví dụ, toàn bộ chức năng thêm sản phẩm vào giỏ hàng, cập nhật số lượng và tính tổng tiền nên được đặt trong **Cart Service** thay vì phân tán sang nhiều service khác nhau.

1.3 Các trường hợp sử dụng thực tế (Case Studies)

Việc chuyển đổi từ kiến trúc Monolithic sang Microservices không chỉ là một xu hướng công nghệ mà là sự thay đổi tất yếu của các tập đoàn công nghệ lớn để đáp ứng quy mô người dùng khổng lồ. Dưới đây là hai ví dụ điển hình:

1.3.1 Netflix: Tiên phong trong kiến trúc Microservices

Sự phát triển của kiến trúc Microservices tại Netflix được xem là "sách giáo khoa" cho ngành phần mềm hiện đại, đánh dấu bước chuyển mình từ một dịch vụ thuê đĩa sang đế chế streaming toàn cầu. Quá trình này được chia thành 5 giai đoạn chính:

a, Giai đoạn 1: Monolith và Sự cố bước ngoặt (2000–2008)

Ban đầu, Netflix vận hành một ứng dụng Java Monolithic lớn chạy trên cơ sở dữ liệu quan hệ tại các trung tâm dữ liệu riêng. Tuy nhiên, khi chuyển sang dịch vụ streaming, hệ thống bộc lộ nhiều điểm yếu: một lỗi nhỏ có thể làm sập toàn bộ dịch vụ, tốc độ triển khai chậm và cơ sở dữ liệu trở thành nút thắt cổ chai. Đỉnh điểm là vào năm 2008, một sự cố nghiêm trọng gây lỗi dữ liệu khiến dịch vụ downtime trong nhiều ngày. Đây chính là bước ngoặt khiến Netflix quyết định từ bỏ kiến trúc truyền thống.

b, Giai đoạn 2: Cloud Migration và SOA (2009–2011)

Netflix bắt đầu chuyển dịch từ trung tâm dữ liệu riêng lên Amazon Web Services (AWS). Trong giai đoạn này, họ thực hiện phân rã monolith thành các dịch vụ nhỏ theo định hướng SOA (Service-Oriented Architecture), tiền thân của Microservices. Các service đầu tiên như User, Recommendation và Billing bắt đầu có database riêng và khả năng scale độc lập.

c, Giai đoạn 3: Hệ sinh thái Microservices và Chaos Engineering (2011–2014)

Đây là thời kỳ bùng nổ các công cụ nội bộ của Netflix:

- **Eureka:** Giải quyết bài toán Service Discovery.
- **Hystrix:** Triển khai mô hình Circuit Breaker, giúp ngăn chặn lỗi dây chuyền.
- **Zuul:** Đóng vai trò API Gateway để quản lý routing và xác thực.
- **Chaos Engineering:** Netflix tạo ra *Chaos Monkey* để tự động kiểm tra khả năng chịu lỗi của hệ thống.

d, Giai đoạn 4: Cloud-Native và Containerization (2015–2018)

Netflix tối ưu hóa hạ tầng bằng cách chuyển sang sử dụng container với nền tảng quản lý nội bộ tên là *Titus*. Giai đoạn này tập trung vào khả năng Auto-scaling và quy trình CI/CD hoàn thiện.

e, Giai đoạn 5: Real-time, Data và Global Scale (2018–nay)

Hiện nay, Netflix vận hành hàng nghìn microservices trên phạm vi toàn cầu, kết hợp Event-driven với Kafka, GraphQL và AI.

Thời gian	Giai đoạn phát triển
2000–2008	Monolithic Architecture (Java & Relational DB)
2008	Sự cố Database failure lớn thúc đẩy thay đổi
2009	Bắt đầu di cư lên đám mây AWS
2010–2012	Hình thành các microservices đầu tiên
2012–2015	Phát triển hệ sinh thái OSS (Eureka, Hystrix, Zuul)
2015–2018	Cloud-native và Containerization (Titus)
2018–nay	Event-driven, AI và Global Scale

Bảng 1.3: Tóm tắt timeline phát triển kiến trúc tại Netflix

1.3.2 Amazon: Cuộc cách mạng "Two-Pizza Team" và sự ra đời của AWS

Amazon là một trong những công ty đầu tiên thực hiện chuyển đổi sang kiến trúc SOA ở quy mô cực lớn, thậm chí sớm hơn cả Netflix.

a, Giai đoạn 1: Monolith thời kỳ đầu (1994–2000)

Ban đầu, Amazon là một codebase Monolithic khổng lồ. Khi mở rộng sang nhiều ngành hàng, hệ thống bắt đầu quá tải và các nhóm phát triển phụ thuộc lẫn nhau một cách chặt chẽ.

b, Bước ngoặt: Chỉ thị "API Mandate" của Jeff Bezos (2002)

Jeff Bezos đã đưa ra bản chỉ thị nổi tiếng: tất cả các nhóm phải giao tiếp thông qua API; không được phép truy cập trực tiếp vào DB của nhóm khác; và mọi API phải được thiết kế để có thể công khai ra bên ngoài.

c, Giai đoạn 2: SOA và mô hình "Two-Pizza Team" (2002–2006)

Amazon tách hệ thống thành các dịch vụ nhỏ và áp dụng mô hình "Two-Pizza Team": mỗi nhóm phát triển phải đủ nhỏ để ăn no với 2 cái pizza, hoàn toàn làm chủ service của mình.

d, Giai đoạn 3: Sự ra đời của AWS (2003–2008)

Việc xây dựng hạ tầng cho microservices đã dẫn đến sự ra đời của AWS (SQS, S3, EC2), vốn ban đầu phục vụ chính nhu cầu nội bộ của Amazon.

e, Giai đoạn 4: Event-driven và Hệ phân tán (2008–2015)

Amazon công bố *Dynamo paper* (2007), đặt nền móng cho phong trào NoSQL và tính nhất quán cuối cùng.

f, Giai đoạn 5: Cloud-native và Cell-based Architecture (2015–nay)

Hiện nay Amazon sử dụng kiến trúc *Cell-based* để tăng cường khả năng cô lập lỗi ở quy mô hàng chục nghìn services.

Tiêu chí	Amazon	Netflix
Thế mạnh	Đi đầu về SOA và hạ tầng đám mây.	Đi đầu về Cloud-native và tính chịu lỗi.
Đóng góp	Khai sinh ra AWS và phong trào NoSQL.	Khai sinh ra Chaos Engineering và bộ OSS.
Văn hóa	Ownership cực mạnh (Two-Pizza Team).	Tự do và trách nhiệm (Freedom & Responsibility).

Bảng 1.4: So sánh đặc trưng Microservices giữa Amazon và Netflix

1.4 Domain Driven Design (DDD)

1.4.1 Mục tiêu

Domain Driven Design (DDD) là phương pháp thiết kế phần mềm tập trung vào việc mô hình hóa hệ thống dựa trên nghiệp vụ thực tế (business domain). Thay vì ưu tiên cấu trúc kỹ thuật hoặc công nghệ, DDD hướng tới việc xây dựng hệ thống phản ánh đúng quy trình hoạt động của doanh nghiệp.

DDD được đề xuất nhằm giải quyết vấn đề các hệ thống lớn thường trở nên phức tạp, khó mở rộng và khó bảo trì khi business logic ngày càng tăng.

Các mục tiêu chính của DDD bao gồm:

- Mô hình hóa chính xác nghiệp vụ của hệ thống.
- Tạo ngôn ngữ chung giữa lập trình viên và chuyên gia domain.
- Giảm độ phức tạp thông qua việc phân chia domain rõ ràng.
- Tăng khả năng bảo trì và mở rộng hệ thống.
- Hỗ trợ việc phân tách Microservices theo nghiệp vụ.

DDD đặc biệt phù hợp với kiến trúc Microservices vì giúp xác định ranh giới service dựa trên business domain thay vì technical layer.

1.4.2 Các khái niệm cốt lõi

Entity

Entity là đối tượng có định danh riêng (ID) và tồn tại xuyên suốt vòng đời hệ thống. Giá trị thuộc tính của entity có thể thay đổi theo thời gian nhưng định danh vẫn được giữ nguyên.

Ví dụ:

- User
- Product

- **Order**

Trong hệ thống BookStore, mỗi đơn hàng (**Order**) có mã định danh riêng và được theo dõi xuyên suốt quá trình xử lý.

Value Object

Value Object là đối tượng không có định danh riêng và được so sánh dựa trên giá trị của nó thay vì ID.

Đặc điểm của Value Object:

- Không có ID.
- Thường mang tính immutable.
- Hai object được xem là giống nhau nếu toàn bộ giá trị giống nhau.

Ví dụ:

- Address
- Money
- PhoneNumber

Trong hệ thống BookStore, địa chỉ giao hàng (**Address**) có thể được xem là một Value Object.

Aggregate

Aggregate là một nhóm các Entity và Value Object liên quan chặt chẽ với nhau và được quản lý như một đơn vị thống nhất.

Mỗi Aggregate sẽ có một Aggregate Root chịu trách nhiệm quản lý toàn bộ dữ liệu bên trong aggregate đó.

Ví dụ:

- **Order** là Aggregate Root.
- **OrderItem** thuộc Aggregate của **Order**.

Trong mô hình này, các service bên ngoài chỉ tương tác thông qua Aggregate Root nhằm đảm bảo tính nhất quán dữ liệu.

Bounded Context

Bounded Context là ranh giới xác định phạm vi áp dụng của một mô hình nghiệp vụ cụ thể. Mỗi bounded context có business logic, dữ liệu và thuật ngữ riêng.

Ví dụ trong hệ thống BookStore:

- User Context
- Order Context
- Inventory Context
- Payment Context

Việc xác định bounded context giúp tránh tình trạng business logic bị trộn lẫn giữa các domain khác nhau.

1.4.3 Context Map

Context Map là sơ đồ mô tả mối quan hệ giữa các bounded context trong hệ thống và cách chúng tương tác với nhau.

Context Map giúp:

- Xác định dependency giữa các domain.
- Giảm coupling không cần thiết.
- Hỗ trợ phân tách Microservices rõ ràng hơn.

Một số mô hình quan hệ phổ biến bao gồm:

Shared Kernel: Trong mô hình này, hai bounded context chia sẻ một phần model hoặc thư viện chung.

Ví dụ:

- User Context và Order Context cùng sử dụng model Address.

Mô hình Shared Kernel giúp giảm trùng lặp dữ liệu nhưng đồng thời cũng làm tăng mức độ phụ thuộc giữa các context.

Customer-Supplier: Một bounded context đóng vai trò cung cấp dữ liệu hoặc dịch vụ cho bounded context khác thông qua API hoặc event.

Ví dụ:

- Payment Context sử dụng dữ liệu từ Order Context.

Trong mô hình này:

- Supplier chịu trách nhiệm duy trì API ổn định.
- Customer phụ thuộc vào dữ liệu hoặc service được cung cấp.

1.4.4 DDD trong Microservices

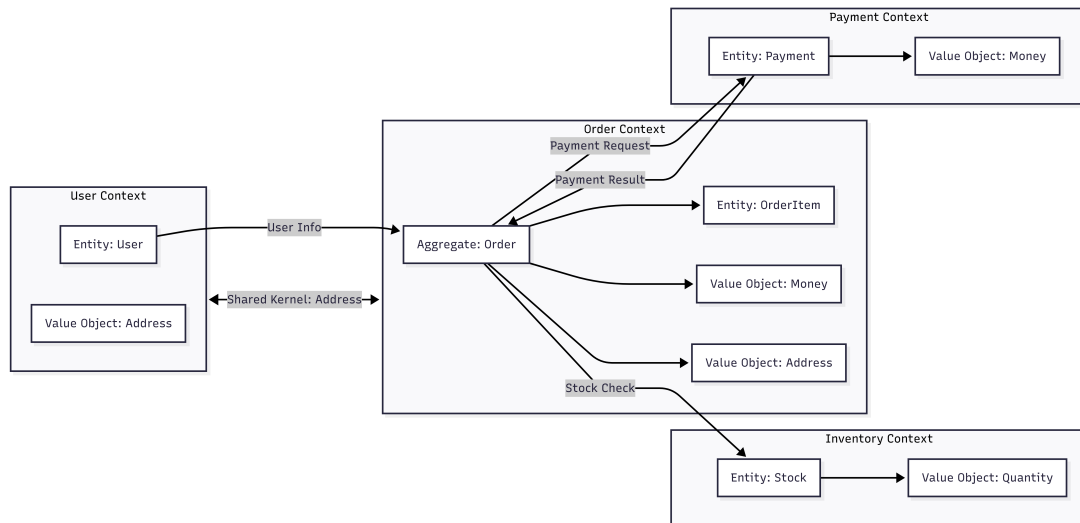
DDD thường được sử dụng kết hợp với Microservices để xác định ranh giới service dựa trên domain nghiệp vụ thay vì tầng kỹ thuật.

Một nguyên tắc phổ biến là:

Mỗi bounded context tương ứng với một microservice

Ví dụ:

- User Context → User Service
- Order Context → Order Service
- Payment Context → Payment Service



Hình 1.6: Minh họa mô hình Bounded Context

Cách tiếp cận này mang lại nhiều lợi ích:

- Giảm coupling giữa các service.
- Dễ triển khai và mở rộng độc lập.
- Tăng khả năng bảo trì hệ thống.
- Hỗ trợ tổ chức nhóm phát triển theo domain.

Ngoài ra, DDD khuyến khích tránh phân chia service theo technical layer như Controller Service hoặc Database Service. Thay vào đó, service nên được thiết kế xoay quanh business domain để đảm bảo tính độc lập và phản ánh đúng nghiệp vụ của hệ thống.

1.5 Case Study: Healthcare (Luyện Decomposition)

1.5.1 Mô tả bài toán

Hệ thống Healthcare được xây dựng nhằm hỗ trợ quản lý và vận hành các hoạt động trong bệnh viện hoặc phòng khám. Hệ thống cho phép quản lý thông tin bệnh nhân, bác sĩ, lịch khám, hồ sơ bệnh án, thanh toán và thuốc điều trị.

Trong mô hình ban đầu, toàn bộ chức năng được xây dựng theo kiến trúc Monolithic.

Khi số lượng người dùng và dữ liệu tăng lên, hệ thống bắt đầu gặp các vấn đề như:

- Khó mở rộng từng chức năng riêng lẻ.
- Khó bảo trì khi business logic ngày càng phức tạp.
- Việc triển khai cập nhật ảnh hưởng toàn bộ hệ thống.
- Các module phụ thuộc chặt chẽ vào nhau.

Do đó, hệ thống cần được phân rã theo hướng Microservices kết hợp Domain Driven Design (DDD) để tăng khả năng mở rộng và bảo trì.

1.5.2 Bước 1: Xác định Domain

Domain là các nhóm nghiệp vụ chính trong hệ thống Healthcare. Việc xác định domain giúp phân chia hệ thống thành các khu vực chức năng rõ ràng.

Các domain chính bao gồm:

- **Patient Management Domain:** Quản lý thông tin bệnh nhân.
- **Doctor Management Domain:** Quản lý thông tin bác sĩ và chuyên khoa.
- **Appointment Domain:** Quản lý lịch hẹn khám bệnh.
- **Medical Record Domain:** Quản lý hồ sơ bệnh án và lịch sử khám chữa bệnh.
- **Prescription Domain:** Quản lý đơn thuốc và thuốc điều trị.
- **Billing Domain:** Quản lý thanh toán và hóa đơn.
- **Notification Domain:** Gửi email hoặc thông báo lịch khám cho bệnh nhân.

Việc phân chia domain giúp xác định rõ phạm vi trách nhiệm của từng phần trong hệ thống.

1.5.3 Bước 2: Xác định Bounded Context

Sau khi xác định domain, hệ thống tiếp tục được chia thành các bounded context để cô lập business logic và dữ liệu của từng nghiệp vụ.

Các bounded context trong hệ thống bao gồm:

- **Patient Context**
 - Quản lý thông tin bệnh nhân.
 - Lưu trữ hồ sơ cá nhân và lịch sử cơ bản.
- **Doctor Context**
 - Quản lý thông tin bác sĩ.
 - Quản lý chuyên khoa và lịch làm việc.

- **Appointment Context**

- Đặt lịch khám bệnh.
- Kiểm tra lịch trống của bác sĩ.

- **Medical Record Context**

- Quản lý bệnh án.
- Lưu trữ kết quả khám và chẩn đoán.

- **Prescription Context**

- Quản lý đơn thuốc.
- Theo dõi thuốc được kê cho bệnh nhân.

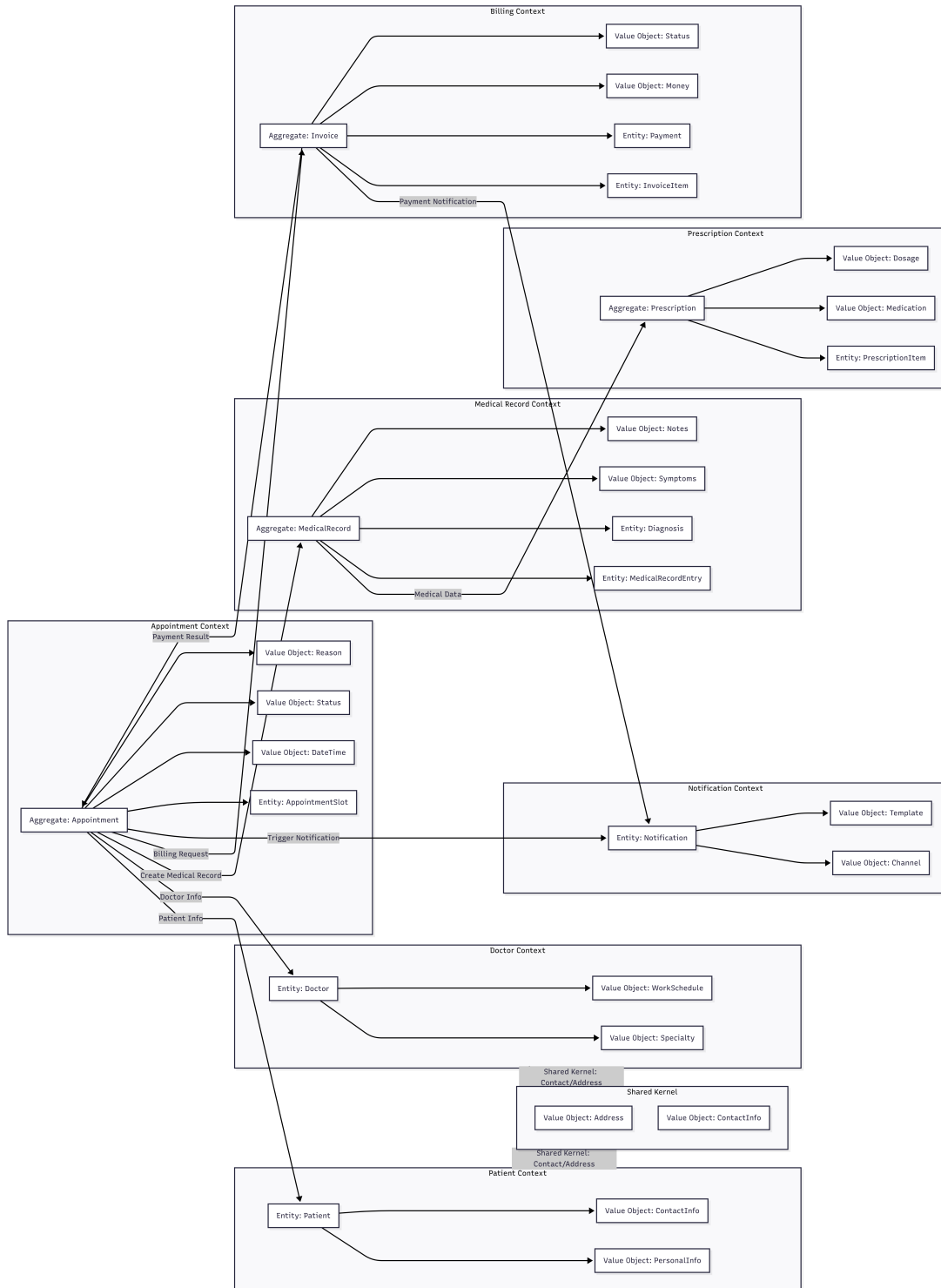
- **Billing Context**

- Quản lý hóa đơn và thanh toán.
- Theo dõi trạng thái thanh toán.

- **Notification Context**

- Gửi email hoặc SMS nhắc lịch khám.
- Gửi thông báo thanh toán.

Mỗi bounded context có business logic và database riêng nhằm giảm coupling giữa các module.



Hình 1.7: Mô hình Bounded Context cho hệ thống Healthcare

1.5.4 Bước 3: Phân rã thành Microservices

Sau khi xác định bounded context, mỗi context sẽ được triển khai thành một microservice tương ứng.

Bounded Context	Microservice
Patient Context	Patient Service
Doctor Context	Doctor Service
Appointment Context	Appointment Service
Medical Record Context	Medical Record Service
Prescription Context	Prescription Service
Billing Context	Billing Service
Notification Context	Notification Service

Bảng 1.5: Phân rã Bounded Context thành Microservices

Mỗi service:

- Có business logic riêng.
- Có database riêng.
- Có thể triển khai độc lập.
- Giao tiếp với nhau thông qua API hoặc Message Broker.

1.5.5 Bước 4: Xác định quan hệ

Sau khi phân rã hệ thống thành các microservice, cần xác định mối quan hệ và cách giao tiếp giữa các service nhằm đảm bảo toàn bộ hệ thống hoạt động đồng bộ.

Trong hệ thống Healthcare, các service không hoạt động độc lập hoàn toàn mà thường xuyên trao đổi dữ liệu với nhau thông qua API hoặc event.

Ví dụ:

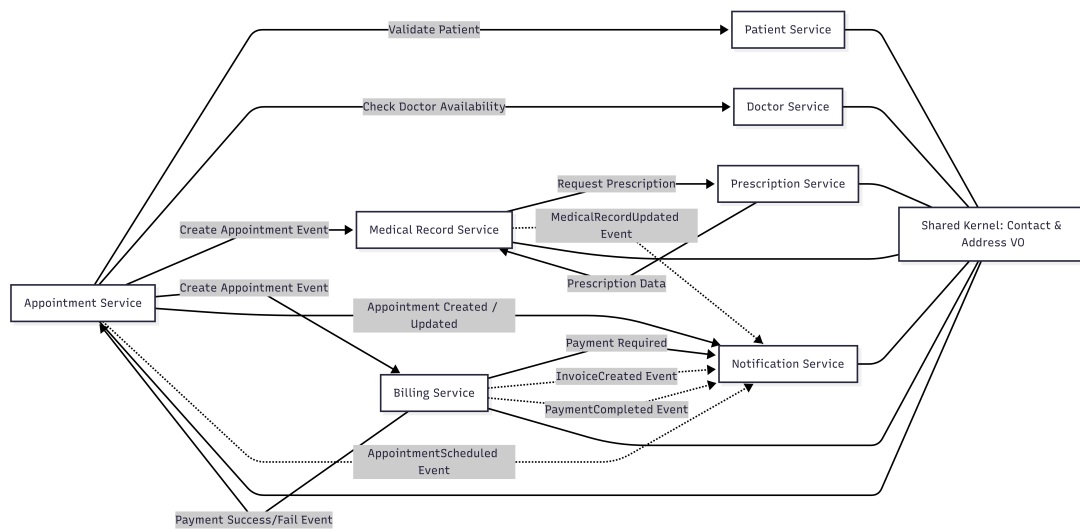
- **Appointment Service** cần lấy thông tin bệnh nhân từ **Patient Service** và lịch làm việc của bác sĩ từ **Doctor Service** trước khi tạo lịch khám.
- **Medical Record Service** sử dụng dữ liệu từ **Appointment Service** để lưu kết quả khám bệnh và chẩn đoán.
- **Prescription Service** dựa trên thông tin bệnh án từ **Medical Record Service** để tạo đơn thuốc.
- **Billing Service** nhận dữ liệu từ **Appointment Service** nhằm tạo hóa đơn thanh toán cho bệnh nhân.
- **Notification Service** gửi email hoặc SMS nhắc lịch khám sau khi đặt lịch thành công hoặc gửi thông báo sau khi thanh toán hoàn tất.

Các quan hệ này thể hiện mô hình Customer-Supplier trong DDD:

- Một service đóng vai trò Supplier cung cấp dữ liệu hoặc chức năng.
- Service còn lại đóng vai trò Customer sử dụng dữ liệu đó để thực hiện nghiệp vụ của mình.

Việc xác định rõ dependency giữa các service giúp:

- Giảm coupling giữa các domain.
- Hỗ trợ triển khai độc lập.
- Dễ mở rộng hệ thống về sau.



Hình 1.8: Sơ đồ mối quan hệ giữa các Microservices trong hệ thống Healthcare

1.5.6 Ví dụ API

Các microservice trong hệ thống Healthcare giao tiếp với nhau thông qua REST API để trao đổi dữ liệu và thực hiện nghiệp vụ.

Microservice	API	Chức năng
Patient Service	GET /patients/{id}	Lấy thông tin bệnh nhân
	POST /patients	Tạo bệnh nhân mới
Doctor Service	GET /doctors/{id}	Lấy thông tin bác sĩ
	GET /doctors/{id}/schedule	Lấy lịch làm việc bác sĩ
Appointment Service	POST /appointments	Tạo lịch khám
	GET /appointments/{id}	Lấy thông tin lịch khám
	PUT /appointments/{id}/cancel	Hủy lịch khám
Medical Record Service	POST /medical-records	Tạo bệnh án
	GET /medical-records/{id}	Lấy thông tin bệnh án
Prescription Service	POST /prescriptions	Tạo đơn thuốc
	GET /prescriptions/{id}	Lấy thông tin đơn thuốc
Billing Service	POST /billings	Tạo hóa đơn thanh toán
	GET /billings/{id}	Lấy thông tin hóa đơn
Notification Service	POST /notifications/email	Gửi email thông báo
	POST /notifications/sms	Gửi SMS nhắc lịch khám

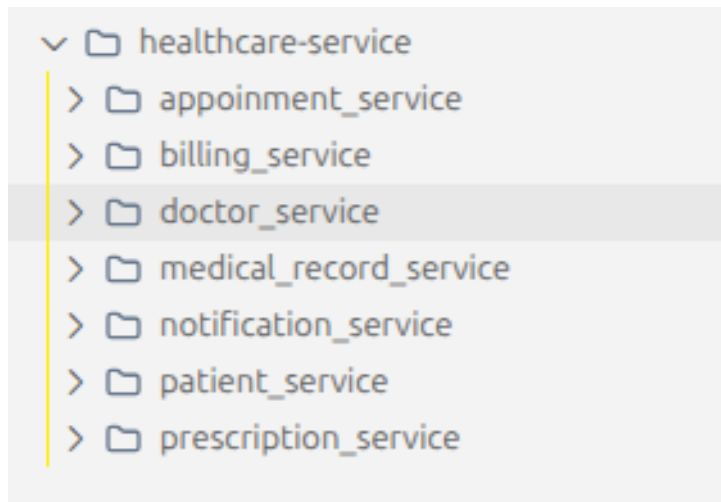
Bảng 1.6: Các API chính trong hệ thống Healthcare

1.5.7 Triển khai Healthcare

Dựa trên việc phân rã các Bounded Context, hệ thống Healthcare được triển khai dưới dạng các Microservices độc lập. Mỗi service quản lý mã nguồn và cơ sở dữ liệu riêng biệt.

a, Cấu trúc mã nguồn triển khai

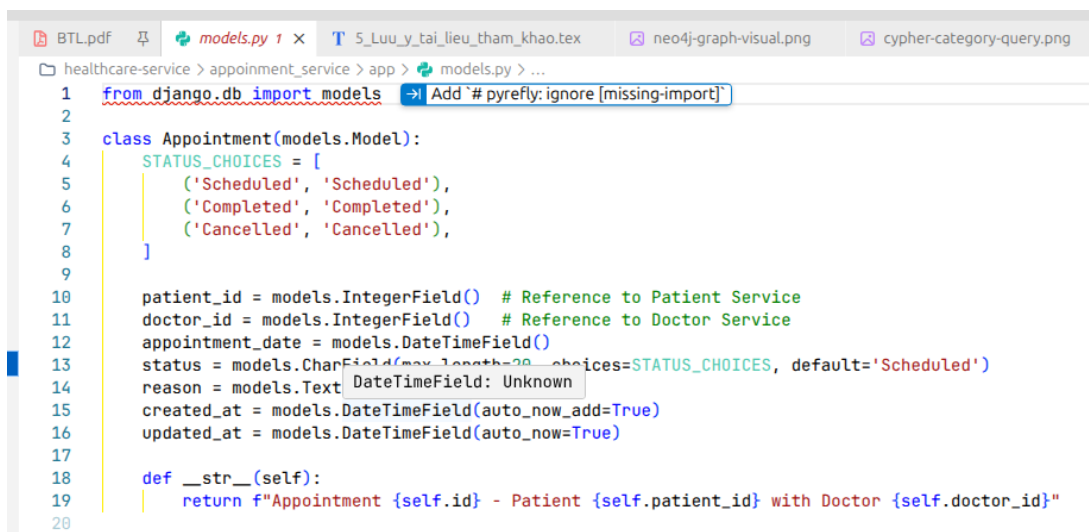
Việc tổ chức mã nguồn theo từng service giúp các nhóm phát triển có thể làm việc độc lập mà không gây xung đột. Dưới đây là cấu trúc thư mục điển hình cho các microservices trong hệ thống:



Hình 1.9: Cấu trúc tổ chức mã nguồn của hệ thống Healthcare

b, Triển khai Models

Mỗi microservice định nghĩa các Model (Entity/Value Object) dựa trên Bounded Context đã xác định. Ví dụ, trong Appointment Service, các Model được thiết kế để quản lý trạng thái lịch khám và các ràng buộc thời gian.



Hình 1.10: Khai báo Model thực thể cho các Entity trong Healthcare Service

1.6 Kết luận

Việc lựa chọn giữa kiến trúc Monolithic và Microservices phụ thuộc hoàn toàn vào quy mô, yêu cầu nghiệp vụ và nguồn lực của dự án. Monolithic vẫn là lựa chọn tối ưu cho các hệ thống nhỏ, cần triển khai nhanh nhờ tính đơn giản trong thiết kế. Ngược lại, Microservices là giải pháp tất yếu cho các hệ thống lớn, phức tạp, đòi hỏi khả năng mở rộng linh hoạt và triển khai độc lập.

Sự kết hợp giữa Microservices và Domain Driven Design (DDD) giúp hệ thống phản

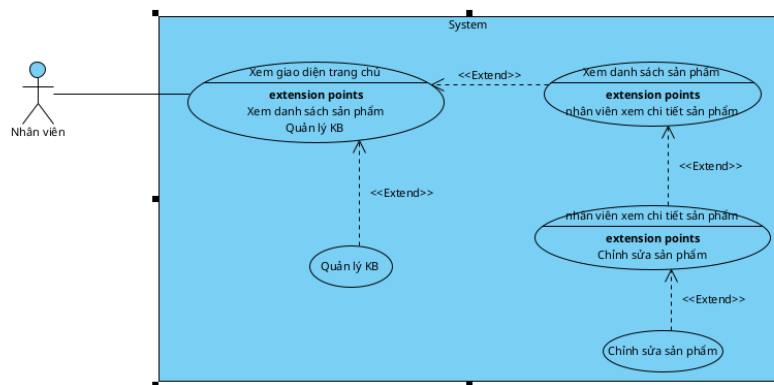
ánh chính xác nghiệp vụ thực tế, giảm thiểu sự phụ thuộc giữa các thành phần thông qua Bounded Context. Case study Healthcare đã minh chứng cho hiệu quả của việc phân rã hệ thống dựa trên domain-centric, giúp tối ưu hóa khả năng bảo trì và vận hành.

Tóm lại, không có kiến trúc hoàn hảo cho mọi trường hợp. Các tổ chức cần cân nhắc kỹ lưỡng giữa lợi ích về tính linh hoạt của Microservices và chi phí vận hành hạ tầng phức tạp để đưa ra quyết định phù hợp nhất với lộ trình phát triển của mình.

STT	Use Case	Phân tích yêu cầu chức năng
1	Đăng ký & Đăng nhập	Hệ thống cho phép người dùng tạo tài khoản và xác thực để truy cập các tính năng cá nhân hóa (Customer Context).
2	Xem danh sách sản phẩm	Hiển thị danh mục sản phẩm, tìm kiếm và nhận các gợi ý từ AI (Catalog & AI Context).
3	Xem chi tiết sản phẩm	Cung cấp thông tin chi tiết, hình ảnh và cho phép để lại đánh giá, bình luận (Catalog & Comment Context).
4	Hỏi đáp với Chatbot	Tương tác trực tiếp với trợ lý ảo RAG để được tư vấn sản phẩm và giải đáp thắc mắc (AI Context).
5	Quản lý giỏ hàng	Thêm, xóa và cập nhật số lượng sản phẩm trước khi mua hàng (Cart Context).
6	Thực hiện đặt hàng	Quy trình chuyển đổi từ giỏ hàng sang đơn hàng, áp dụng mã giảm giá và xác nhận thông tin (Ordering Context).
7	Thanh toán đơn hàng	Thực hiện giao dịch thông qua các cổng thanh toán trực tuyến hoặc chọn COD (Payment Context).
8	Theo dõi vận chuyển	Kiểm tra lộ trình giao hàng và thông tin mã vận đơn (Shipping Context).
9	Quản lý hồ sơ & Đơn hàng	Cập nhật thông tin cá nhân và xem lại lịch sử các giao dịch đã thực hiện.

2.1.2 Phân tích Use Case Nhân viên

Nhóm chức năng này tập trung vào quản trị dữ liệu hệ thống và vận hành AI.



Hình 2.2: Sơ đồ Use Case tổng quan cho Nhân viên

STT	Use Case	Phân tích yêu cầu chức năng
1	Xem giao diện quản trị	Giao diện tổng quan, tích hợp các lối tắt đến quản lý sản phẩm và vận hành hệ thống (Staff Context).
2	Quản lý danh mục & sản phẩm	Cập nhật thông tin, giá cả và số lượng tồn kho của các mặt hàng (Catalog Context).
3	Quản lý KB (Knowledge Base)	Cập nhật dữ liệu tri thức cho AI để nâng cao chất lượng tư vấn của Chatbot (AI Context).
4	Điều phối Đơn hàng & Vận chuyển	Tiếp nhận đơn hàng mới, cập nhật trạng thái và quản lý thông tin giao hàng (Ordering & Shipping Context).
5	Giám sát Thanh toán	Theo dõi các luồng tiền, xác nhận giao dịch và xử lý các vấn đề liên quan đến thanh toán (Payment Context).
6	Quản lý Đánh giá	Kiểm duyệt và phản hồi các bình luận, đánh giá từ phía khách hàng (Comment Context).

2.1.3 Tương tác hệ thống hỗ trợ

- **Hệ thống AI:** Đóng vai trò là Actor phụ, cung cấp dữ liệu cho các Use Case liên quan đến **Gợi ý sản phẩm** và **Chatbot**.

2.1.4 Non-functional Requirements

Bên cạnh các chức năng, hệ thống cần đảm bảo các yêu cầu phi chức năng sau:

- **Khả năng mở rộng (Scalability):** Hỗ trợ tăng tải theo từng service và có khả năng mở rộng ngang (Horizontal Scaling).
- **Hiệu năng (Performance):** Thời gian phản hồi nhanh, hỗ trợ xử lý nhiều người

dùng đồng thời, tối ưu truy vấn dữ liệu và caching.

- **Tính sẵn sàng (Availability):** Hệ thống hoạt động ổn định, giảm thiểu thời gian downtime.
- **Bảo mật (Security):** Xác thực và phân quyền, bảo vệ thông tin thanh toán, mã hóa dữ liệu nhạy cảm, chống tấn công SQL Injection, XSS, CSRF.
- **Khả năng bảo trì (Maintainability):** Dễ nâng cấp và mở rộng, code tổ chức rõ ràng, hỗ trợ logging và monitoring.
- **Khả năng chịu lỗi (Fault Tolerance):** Hệ thống không bị sập toàn bộ khi một service gặp lỗi, có cơ chế retry và backup.
- **Tính nhất quán dữ liệu (Data Consistency):** Đảm bảo dữ liệu đơn hàng/thanh toán không sai lệch, đồng bộ dữ liệu giữa các service.
- **Khả năng triển khai (Deployability):** Hỗ trợ Docker/Kubernetes, dễ dàng tích hợp CI/CD.

2.2 Phân rã hệ thống thành các service

Để xây dựng một hệ thống E-Commerce hiện đại có khả năng mở rộng cao và tích hợp trí tuệ nhân tạo, chúng tôi áp dụng phương pháp thiết kế hướng miền (Domain-Driven Design - DDD). Hệ thống được phân rã thành các đơn vị nghiệp vụ độc lập thông qua hai bước chính: xác định các vùng ngữ cảnh (Bounded Context) và ánh xạ chúng sang kiến trúc Microservices.

2.2.1 Xác định các Bounded Context

Dựa trên phân tích các hoạt động nghiệp vụ thực tế, hệ thống được chia thành 09 Bounded Context cốt lõi:

- **Customer Context:** Chịu trách nhiệm quản lý toàn bộ vòng đời của khách hàng. Ngữ cảnh này không chỉ bao gồm các tính năng cơ bản như đăng ký, đăng nhập mà còn tập trung vào việc quản lý các sở thích cá nhân (personal preferences) và hành vi người dùng, cung cấp dữ liệu nền tảng quan trọng để phục vụ các dịch vụ AI cá nhân hóa.
- **Staff Context:** Tập trung hoàn toàn vào các nghiệp vụ quản trị và vận hành hệ thống nội bộ. Vùng này xử lý việc phân quyền nhân viên (RBAC), kiểm soát các hoạt động quản lý danh mục, đơn hàng và giám sát toàn bộ luồng vận hành của hệ thống Microservices để đảm bảo tính an toàn.
- **Catalog Context:** Đóng vai trò là trung tâm dữ liệu sản phẩm. Nó quản lý thông tin chi tiết về sản phẩm, cấu trúc danh mục (Category) và tình trạng tồn kho hàng hóa (Inventory), đảm bảo tính nhất quán và chính xác của dữ liệu khi hiển thị đến khách

hàng.

- **Cart Context:** Chuyên biệt cho việc quản lý trạng thái mua sắm tạm thời của khách hàng. Ngữ cảnh này xử lý các hoạt động thêm, xóa, cập nhật số lượng sản phẩm trong giỏ hàng và lưu trữ thông tin giỏ hàng theo từng định danh người dùng.
- **Ordering Context:** Tập trung vào quy trình chuyển đổi từ giỏ hàng sang đơn hàng chính thức. Ngữ cảnh này xử lý việc tạo đơn hàng, tính toán tổng tiền và cập nhật trạng thái vòng đời đơn hàng.
- **Payment Context:** Quản lý quy trình thanh toán của hệ thống. Ngữ cảnh này chịu trách nhiệm tích hợp các giải pháp thanh toán trực tuyến (như VNPAY, MoMo), xử lý xác thực giao dịch và lưu trữ lịch sử thanh toán.
- **Shipping Context:** Phụ trách các nghiệp vụ liên quan đến vận chuyển hàng hóa. Bao gồm việc tính phí giao hàng, lựa chọn đơn vị vận chuyển, tạo mã vận đơn (Tracking Number) và cập nhật lộ trình giao hàng đến khách hàng.
- **AI Service Context:** Một thành phần đặc thù phụ trách các tính năng thông minh của hệ thống. Vùng này bao gồm hệ thống gợi ý sản phẩm dựa trên các mô hình Deep Learning, Chatbot hỗ trợ tư vấn mua sắm sử dụng công nghệ RAG và quản trị cơ sở tri thức (Knowledge Base) để cung cấp thông tin chính xác nhất.
- **Comment Context:** Quản lý các tương tác và phản hồi của khách hàng sau khi trải nghiệm sản phẩm. Nó xử lý việc gửi bình luận, đánh giá chất lượng và hệ thống chấm điểm sản phẩm (Rating), giúp tăng cường niềm tin và cung cấp dữ liệu phản hồi cho hệ thống quản trị.

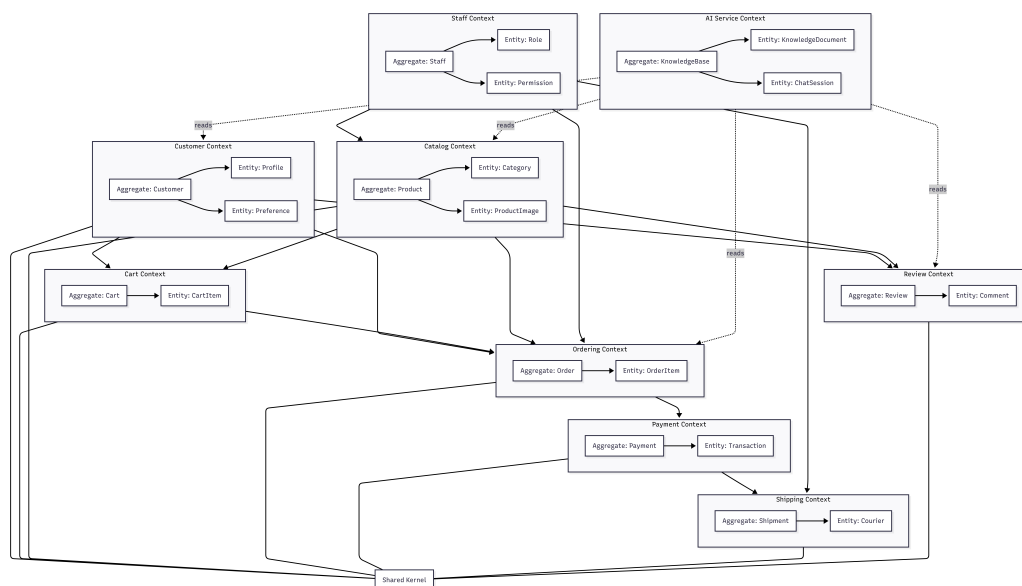
2.2.2 Shared Kernel

Để đảm bảo tính nhất quán dữ liệu giữa các Bounded Context khác nhau mà không làm tăng tính phụ thuộc (Coupling) quá mức, chúng tôi xác định một thành phần dùng chung gọi là **Shared Kernel**.

- **Address Shared Kernel:** Định nghĩa cấu trúc địa chỉ dùng chung cho toàn hệ thống. Thành phần này bao gồm các thuộc tính: `receiver_name`, `phone_number`, `province_city`, `district`, `ward`, `street_address`.
- **Ứng dụng:** Được chia sẻ giữa **Customer Context** (quản lý sở địa chỉ của khách hàng) và **Shipping Context** (địa chỉ giao hàng cụ thể cho từng đơn hàng).

Dưới đây là bảng tổng hợp các thành phần cốt lõi trong từng Bounded Context bao gồm Aggregate Root, các Entity và Value Object tương ứng:

Bounded Context	Aggregate Root	Entities	Value Objects
Customer Context	User, Profile	User, Profile	Address (Shared Kernel)
Staff Context	Staff	Staff, Role, Permission, PermissionRole	AccessScope, AuditLogEntry
Catalog Context	Product	Product, Category, Brand, Tag, ProductTag, ProductImage	Price, SKU, Stock, Attributes (JSON)
Cart Context	Cart	Cart, CartItem	Quantity, AddedAt
Ordering Context	Order	Order, OrderItem	OrderStatus, Money
Payment Context	Payment	Payment, PaymentMethod, TransactionRecord	PaymentMethod, Amount, Status
Shipping Context	Shipment	Shipment, Carrier	TrackingNumber, Status
AI Service Context	KnowledgeBase	KnowledgeDocument, ChatSession	VectorEmbedding, ConfidenceScore
Comment Context	Review	Review, Comment	Rating, Content



Hình 2.3: Mô hình phân rã Bounded Context của hệ thống theo DDD

2.2.3 Xác định các lớp thực thể chi tiết cho từng Bounded Context

Dựa trên phân tích yêu cầu chức năng và các Bounded Context đã xác định, chúng tôi cụ thể hóa các lớp thực thể (Entity Classes) cùng các thuộc tính cốt lõi của chúng:

1. Customer Context:

- **User:** id, username, password, email, date_joined.
- **Profile:** id, full_name, phone_number, gender, address_list (Address[]).

2. Staff Context:

- **Staff:** id, employee_id, full_name, department.
- **Role:** id, name, description.
- **Permission:** id, code, name, description.
- **PermissionRole:** id, permission_id, role_id.

3. Catalog Context:

- **Product:** id, name, description, price, sku, stock, category_id, attributes (JSON).
- **Category:** id, name, description.
- **Brand:** id, name.
- **Tag:** id, name, slug.
- **ProductTag:** id, product_id, tag_id.
- **ProductImage:** id, product_id, image_url.

4. Cart Context:

- **Cart:** id, user_id, created_at, attribute.
- **CartItem:** id, cart_id, product_id, quantity, added_at.

5. Ordering Context:

- **Order:** id, user_id, total_amount, status, created_at.
- **OrderItem:** id, order_id, product_id, quantity, unit_price.

6. Payment Context:

- **Payment:** id, order_id, amount, payment_method_id, status, transaction_id.
- **PaymentMethod:** id, name, description.
- **TransactionRecord:** id, payment_id, gateway_response, processed_at.

7. Shipping Context:

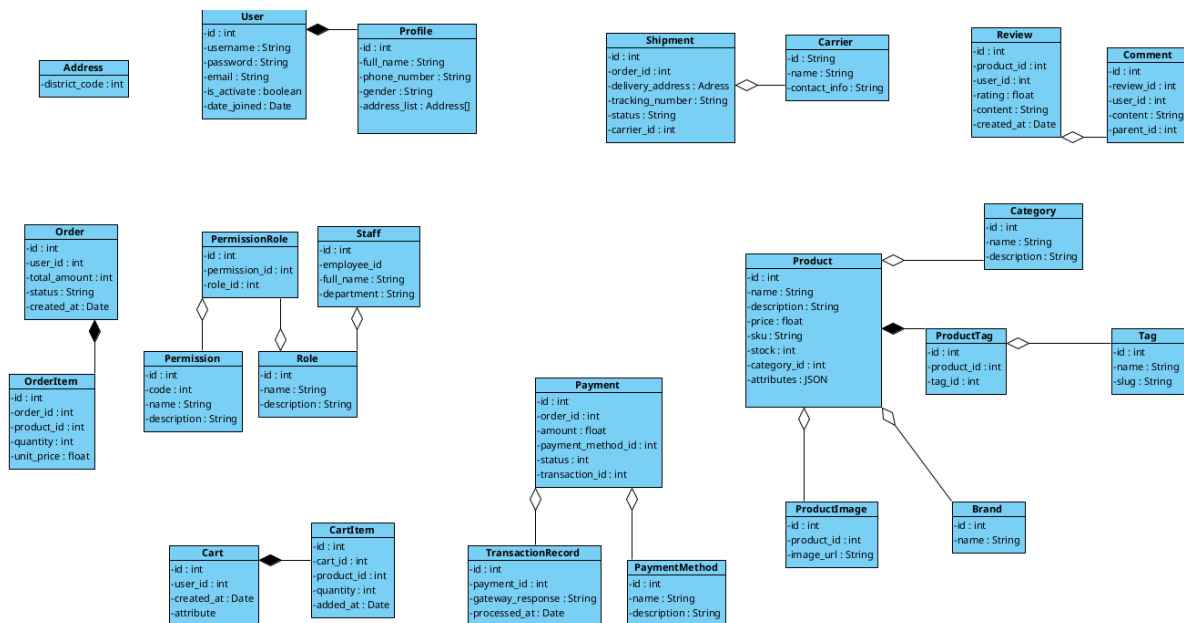
- **Shipment:** id, order_id, delivery_address (Address), tracking_number, status, carrier_id.
- **Carrier:** id, name, contact_info.

8. AI Service Context:

- **KnowledgeDocument:** id, title, content, source_url, vector_id.
- **ChatSession:** id, user_id, start_time, end_time, summary.

9. Comment Context:

- **Review:** id, product_id, user_id, rating, content, created_at.
- **Comment:** id, review_id, user_id, content, parent_id.



Hình 2.4: Sơ đồ các lớp thực thể (Entity Classes) trong pha phân tích

2.2.4 Thiết kế hệ thống Microservices theo Bounded Context

Sau khi xác định được các vùng ngữ cảnh, chúng tôi ánh xạ chúng thành các Microservices thực tế. Mỗi service sở hữu cơ sở dữ liệu riêng, đảm bảo tính Loose Coupling (ghép nối lỏng) và High Cohesion (độ gắn kết cao), giúp hệ thống có thể bảo trì và mở rộng độc lập.

Bounded Context	Microservice	Trách nhiệm chính
Customer Context	User Service	Quản lý tài khoản, hồ sơ khách hàng, sổ địa chỉ và kích hoạt tài khoản.
Staff Context	Staff Service	Quản trị nhân sự, phân quyền hệ thống (RBAC) và quản lý quyền truy cập.
Catalog Context	Product Service	Quản lý thông tin sản phẩm, danh mục, thương hiệu, nhãn (Tag) và kho hàng.
Cart Context	Cart Service	Quản lý giỏ hàng và các mặt hàng tạm thời của người dùng.
Ordering Context	Order Service	Xử lý quy trình đặt hàng, tính toán giá trị và quản lý vòng đời đơn hàng.
Payment Context	Payment Service	Xử lý giao dịch thanh toán, quản lý phương thức thanh toán và lịch sử giao dịch.
Shipping Context	Shipping Service	Quản lý đơn vị vận chuyển, tính phí và theo dõi hành trình đơn hàng.
AI Service Context	AI Service	Cung cấp Chatbot (RAG), hệ thống gợi ý và quản trị tri thức sản phẩm.
Comment Context	Comment Service	Quản lý đánh giá sản phẩm, bình luận và phản hồi từ khách hàng.

Các Microservices này tuân thủ các nguyên tắc thiết kế cốt lõi bao gồm: giao tiếp thông qua RESTful API, quyền sở hữu dữ liệu riêng biệt (Data Ownership) và sử dụng API Gateway làm điểm truy cập duy nhất.

2.2.5 Xác định các lớp Controller cho từng Microservice

Trong pha thiết kế chi tiết, chúng tôi xác định các lớp Controller đóng vai trò là "Lớp biên"(Boundary) để tiếp nhận và xử lý các yêu cầu từ phía Client. Dưới đây là các lớp Controller tiêu biểu kèm theo các phương thức chức năng chính:

1. User Service

Controller	Phương thức	Mô tả đơn giản
AuthController	register()	Đăng ký tài khoản người dùng mới.
	login()	Xác thực và đăng nhập vào hệ thống.
	logout()	Đăng xuất và hủy phiên làm việc.
ProfileController	get_profile()	Lấy thông tin chi tiết hồ sơ cá nhân.
	update_profile()	Cập nhật thông tin cơ bản (tên, số điện thoại).
	add_address()	Thêm địa chỉ giao hàng mới vào sổ địa chỉ.
	delete_address()	Xóa địa chỉ giao hàng không còn sử dụng.

2. Staff Service

Controller	Phương thức	Mô tả đơn giản
StaffManager	list_staff()	Danh sách toàn bộ nhân viên vận hành.
	create_staff()	Tạo tài khoản cho nhân sự mới.
	update_department()	Điều chuyển nhân viên giữa các phòng ban.
	update_status()	Khóa hoặc mở lại tài khoản nhân viên.
RBACController	list_roles()	Danh sách các vai trò (Admin, Manager, Staff).
	assign_role_to_staff()	Gán vai trò cụ thể cho một nhân viên.
	update_role_permissions()	Thay đổi danh sách quyền hạn của một vai trò.

3. Product Service

Controller	Phương thức	Mô tả đơn giản
CatalogController	list_products()	Lấy danh sách sản phẩm hiển thị trên sàn.
	get_product_detail()	Xem chi tiết thông tin và thuộc tính sản phẩm.
	filter_by_category()	Lọc danh sách sản phẩm theo danh mục.
	search_by_keyword()	Tìm kiếm sản phẩm theo tên hoặc từ khóa.
AdminProductController	create_product()	Đăng tải sản phẩm mới lên hệ thống.
	update_product()	Chỉnh sửa thông tin sản phẩm hiện có.
	delete_product()	Gỡ bỏ sản phẩm khỏi danh sách bán.
	update_stock_quantity()	Cập nhật số lượng tồn kho thực tế.
	manage_metadata()	Quản lý Brand, Category và các nhãn Tag.

4. Cart Service

Controller	Phương thức	Mô tả đơn giản
CartController	get_cart()	Lấy danh sách các sản phẩm đang có trong giỏ.
	add_item()	Thêm một sản phẩm mới vào giỏ hàng.
	update_item_quantity()	Thay đổi số lượng mua của một mặt hàng.
	remove_item()	Loại bỏ sản phẩm ra khỏi giỏ hàng.

5. Order Service

Controller	Phương thức	Mô tả đơn giản
CheckoutController	initiate_checkout()	Bắt đầu quy trình thanh toán đơn hàng.
	place_order()	Tạo đơn hàng chính thức vào hệ thống.
OrderHistoryController	list_user_orders()	Xem lịch sử tất cả các đơn hàng đã mua.
	get_order_detail()	Xem chi tiết trạng thái và các món trong đơn.
	cancel_order_request()	Gửi yêu cầu hủy đơn hàng (nếu cho phép).
AdminOrderController	list_all_orders()	Quản lý toàn bộ đơn hàng của sàn thương mại.
	update_order_status()	Thay đổi trạng thái đơn hàng (Đã xác nhận, Giao hàng).

6. Payment Service

Controller	Phương thức	Mô tả đơn giản
PaymentController	create_payment_transaction()	Tạo yêu cầu giao dịch thanh toán mới.
	handle_gateway_callback()	Xử lý kết quả trả về từ cổng (VNPAY, MoMo).
	get_payment_status()	Tra cứu trạng thái thực tế của giao dịch.
PaymentMethodController	list_available_methods()	Danh sách các cổng thanh toán đang hoạt động.

7. Shipping Service

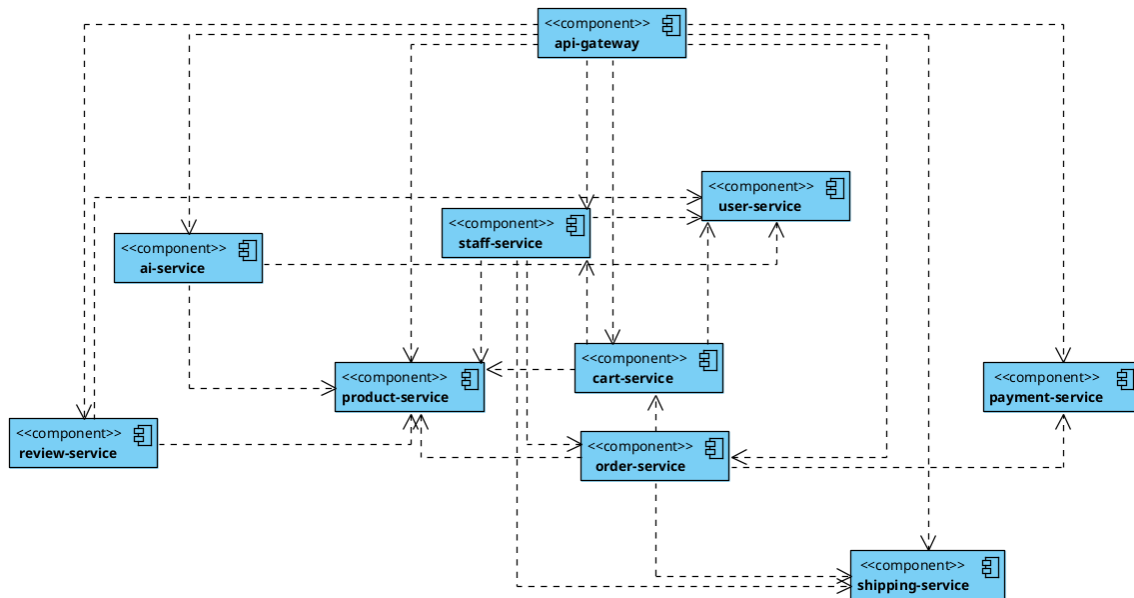
Controller	Phương thức	Mô tả đơn giản
ShipmentController	track_by_number()	Tra cứu lộ trình của kiện hàng theo mã vận đơn.
	get_shipping_fee()	Tính phí vận chuyển dựa trên địa chỉ và khối lượng.
AdminShippingController	create_shipment_order()	Đẩy thông tin đơn hàng sang đơn vị vận chuyển.
	update_tracking_status()	Cập nhật trạng thái giao hàng nội bộ.
	manage_carriers()	Quản lý danh sách các đối tác giao hàng.

8. AI Service

Controller	Phương thức	Mô tả đơn giản
AssistantController	send_query()	Chat với AI (RAG) để tư vấn sản phẩm.
	get_chat_history()	Xem lại các cuộc hội thoại tư vấn trước đó.
RecommendationController	get_personalized_suggestions()	Lấy danh sách gợi ý sản phẩm cho trang chủ.
	get_similar_products()	Gợi ý sản phẩm tương đương trên trang chi tiết.
KnowledgeBaseController	index_product_info()	Đồng bộ dữ liệu Catalog sang Vector Database.

9. Comment Service

Controller	Phương thức	Mô tả đơn giản
ReviewController	submit_review()	Đăng tải nhận xét và số sao cho sản phẩm.
	get_product_reviews()	Lấy toàn bộ đánh giá của một sản phẩm cụ thể.
CommentController	post_reply()	Phản hồi lại đánh giá của khách hàng.
	moderate_comment()	Ẩn/hiện các bình luận vi phạm chính sách.



Hình 2.5: Sơ đồ mối quan hệ và tương tác giữa các Microservices

Mô tả hệ thống:

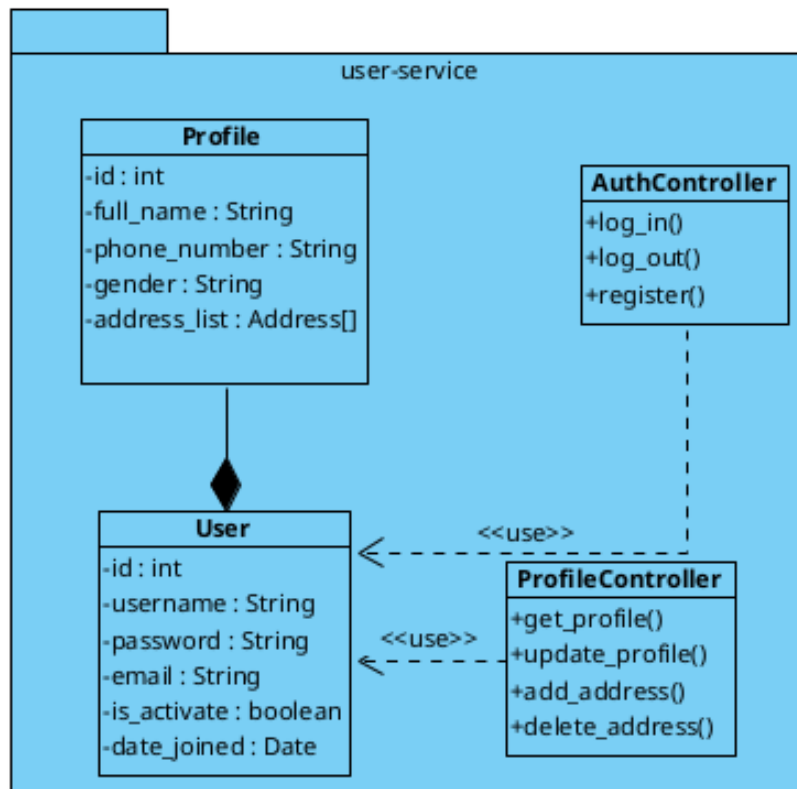
Hệ thống gồm 09 microservices tương tác qua API Gateway bằng giao thức REST API. Order Service thực hiện điều phối (orchestration) thông qua việc truy xuất Cart Service, Product Service, Payment Service và Shipping Service để hoàn tất quy trình đặt hàng. AI Service đồng bộ hóa dữ liệu từ Product Service sang Vector Database phục vụ xử lý RAG. Comment Service quản lý liên kết giữa User và Product qua các trường định danh (ID). Staff Service xử lý tác vụ quản trị và phân quyền RBAC. Kiến trúc đảm bảo tính độc lập (decoupling) và khả năng mở rộng (scalability) giữa các thành phần.

2.3 User Service

User Service là dịch vụ nền tảng quản lý danh tính người dùng, hồ sơ cá nhân và thông tin địa chỉ giao hàng. Dịch vụ này đảm bảo tính bảo mật và nhất quán của dữ liệu người dùng trên toàn hệ thống.

2.3.1 Sơ đồ lớp thiết kế

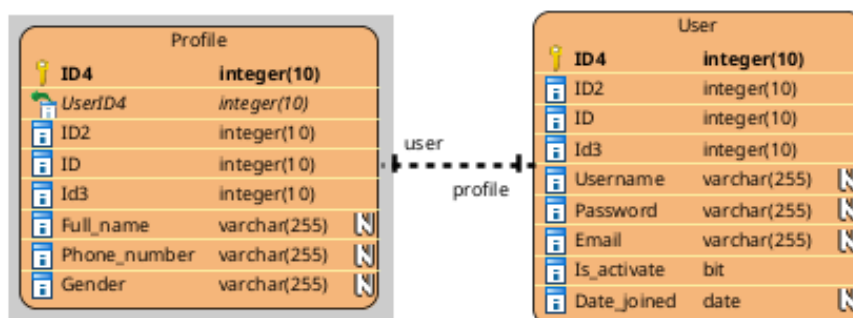
Dưới đây là sơ đồ lớp chi tiết của User Service:



Hình 2.6: Sơ đồ lớp thiết kế User Service

2.3.2 Data Model

Dưới đây là sơ đồ Data Model (Physical Schema) của User Service:



Hình 2.7: Sơ đồ Data Model User Service

2.3.3 Django Model

Hệ thống sử dụng các trường dữ liệu tiêu chuẩn kết hợp với các quan hệ One-to-Many cho sổ địa chỉ (Address Book).

```

ecommerce-service > user_service > app > models.py > ...
1 from django.db import models
2 from django.contrib.auth.models import User
3
4
5 class Profile(models.Model):
6     GENDER_CHOICES = [
7         ('M', 'Nam'),
8         ('F', 'Nu'),
9         ('O', 'Khac'),
10    ]
11    user = models.OneToOneField(User, on_delete=models.CASCADE)
12    full_name = models.CharField(max_length=255, blank=True)
13    phone_number = models.CharField(max_length=20, blank=True)
14    gender = models.CharField(max_length=1, choices=GENDER_CHOICES, blank=True)
15    # address TextField giữ lại cho backward compat (auth_profile GET/PUT)
16    address = models.TextField(blank=True)
17
18    def __str__(self):
19        return self.full_name or self.user.username
20
21
22 class Address(models.Model):
23     """Số địa chỉ giao hàng của khách hàng (Address Shared Kernel)."""
24     profile = models.ForeignKey(Profile, on_delete=models.CASCADE, related_name='address_list')
25     receiver_name = models.CharField(max_length=255)
26     phone_number = models.CharField(max_length=20)
27     province_city = models.CharField(max_length=100)
28     district = models.CharField(max_length=100)
29     ward = models.CharField(max_length=100)
30     street_address = models.CharField(max_length=255)
31     is_default = models.BooleanField(default=False)
32
33     def __str__(self):
34         return f"{self.receiver_name} - {self.street_address}, {self.ward}, {self.district}, {self.province_city}"
35

```

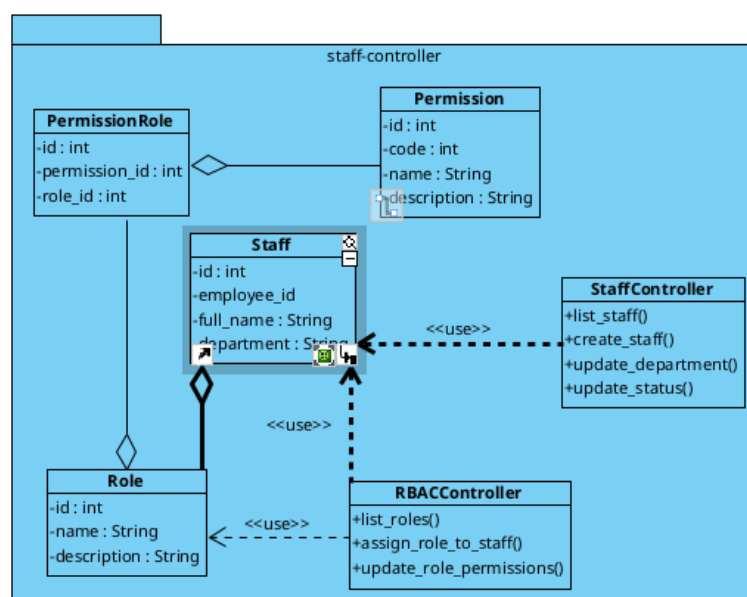
Hình 2.8: Cấu trúc mã nguồn Model của User Service

2.4 Staff Service

Staff Service xử lý các tác vụ quản trị, quản lý nhân sự vận hành và phân quyền truy cập dựa trên vai trò (RBAC - Role Based Access Control).

2.4.1 Sơ đồ lớp thiết kế

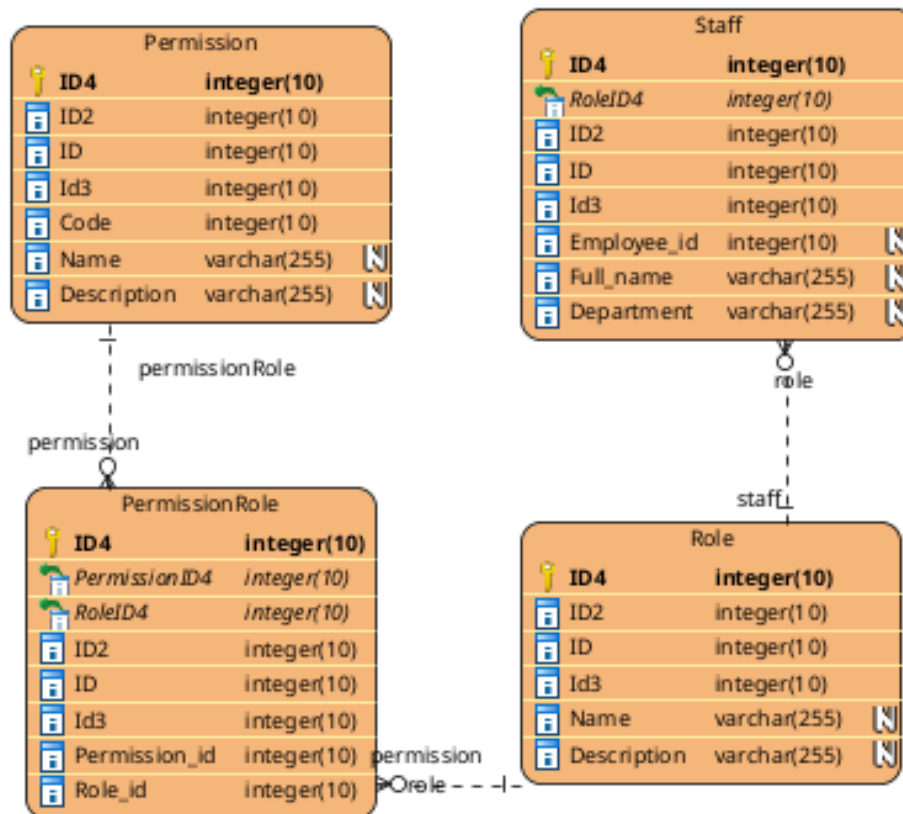
Dưới đây là sơ đồ lớp chi tiết của Staff Service:



Hình 2.9: Sơ đồ lớp thiết kế Staff Service

2.4.2 Data Model

Dưới đây là sơ đồ Data Model (Physical Schema) của Staff Service:



Hình 2.10: Sơ đồ Data Model Staff Service

2.4.3 Django Model

Mã nguồn Model của Staff Service tập trung vào việc quản lý các bảng Role, Permission và Department.


```

ecommerce-service > staff_service > app > models.py > ...
1 from django.db import models
2
3
4 class Role(models.Model):
5     name = models.CharField(max_length=50, unique=True)
6     description = models.TextField(blank=True)
7
8     def __str__(self):
9         return self.name
10
11
12 class Permission(models.Model):
13     code = models.CharField(max_length=100, unique=True)
14     name = models.CharField(max_length=100)
15     description = models.TextField(blank=True)
16
17     def __str__(self):
18         return self.code
19
20
21 class PermissionRole(models.Model):
22     role = models.ForeignKey(Role, on_delete=models.CASCADE, related_name='permission_roles')
23     permission = models.ForeignKey(Permission, on_delete=models.CASCADE, related_name='permission_roles')
24
25     class Meta:
26         unique_together = ('role', 'permission')
27
28     def __str__(self):
29         return f"{self.role.name} → {self.permission.code}"
30
31
32 class Staff(models.Model):
33     STATUS_CHOICES = [
34         ('active', 'Đang hoạt động'),
35         ('locked', 'Đã khóa'),
36     ]
37     user_id = models.IntegerField()
38     employee_id = models.CharField(max_length=50, blank=True)
39     full_name = models.CharField(max_length=255, blank=True)
40     department = models.CharField(max_length=100, blank=True)
41     role = models.ForeignKey(Role, on_delete=models.SET_NULL, null=True, blank=True, related_name='staff_members')
42     status = models.CharField(max_length=10, choices=STATUS_CHOICES, default='active')
43
44     def __str__(self):
45         return self.full_name or str(self.user_id)
46

```

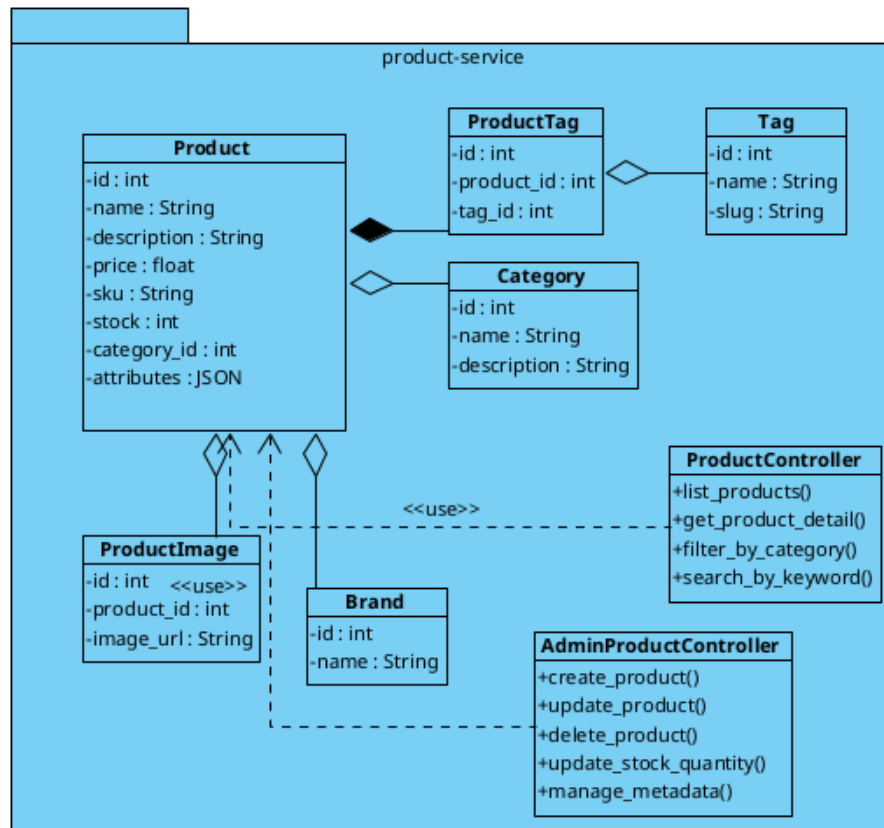
Hình 2.11: Cấu trúc mã nguồn Model của Staff Service

2.5 Product Service

Product Service là dịch vụ trung tâm chịu trách nhiệm quản lý toàn bộ dữ liệu về sản phẩm, danh mục, thương hiệu và các thuộc tính liên quan trong hệ thống. Dịch vụ này cung cấp khả năng tìm kiếm linh hoạt và quản lý kho hàng cơ bản.

2.5.1 Sơ đồ lớp thiết kế

Dưới đây là sơ đồ lớp chi tiết của Product Service, thể hiện mối quan hệ giữa các thực thể chính như Sản phẩm (Product), Danh mục (Category), Thẻ (Tag) và Hình ảnh (Image).



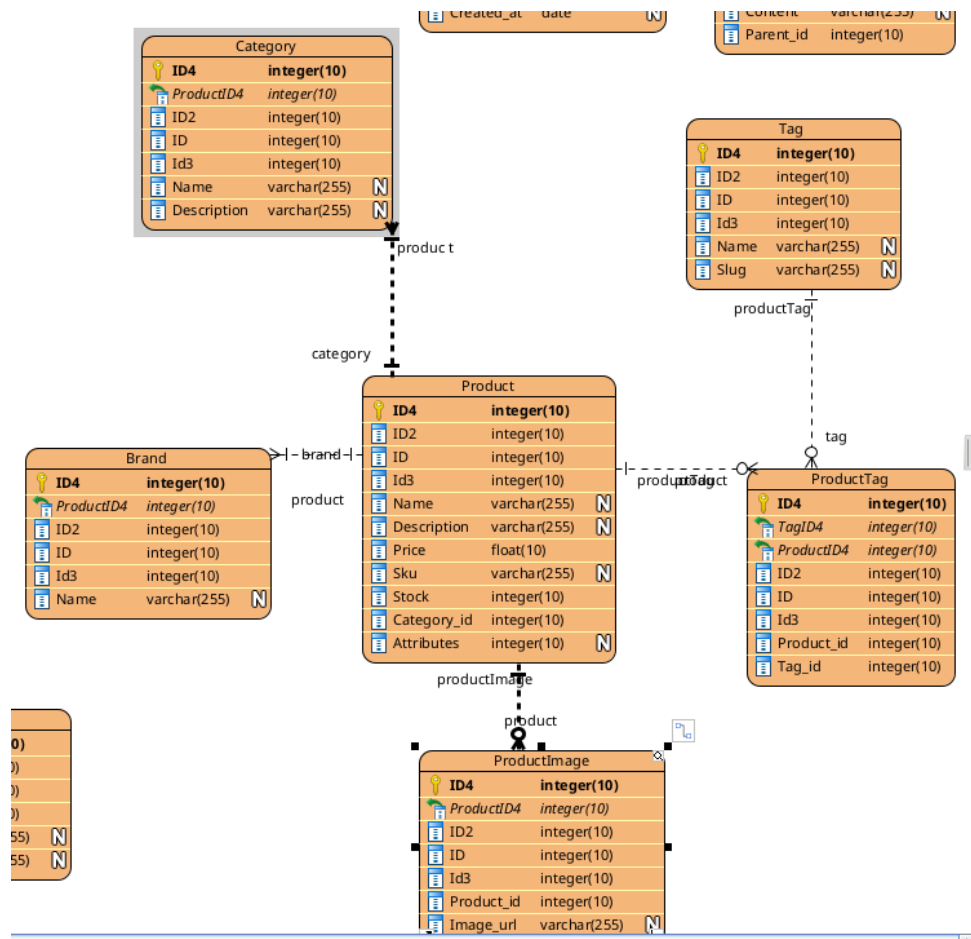
Hình 2.12: Sơ đồ lớp thiết kế Product Service

Sơ đồ lớp bao gồm:

- **ProductModel**: Thực thể chính lưu trữ thông tin sản phẩm (tên, giá, tồn kho) và các thuộc tính động qua trường JSON.
- **CategoryModel**: Quản lý cấu trúc danh mục phân cấp (Cha-Con).
- **TagModel**: Các thẻ phân loại sản phẩm.
- **ProductImageModel**: Quản lý các liên kết hình ảnh của sản phẩm.

2.5.2 Data Model

Dưới đây là sơ đồ Data Model (Physical Schema) của Product Service:



Hình 2.13: Sơ đồ Data Model Product Service

2.5.3 Django Model

Hệ thống sử dụng mô hình dữ liệu lai giữa quan hệ (SQL) và phi quan hệ (JSON) để tối ưu hóa tính linh hoạt. Các thông tin cốt lõi được lưu trữ trong các cột cố định, trong khi các thuộc tính đặc thù của từng loại sản phẩm được lưu trữ trong trường `attributes` kiểu JSON.

```

ecommerce-service > product_service > modules > catalog > infrastructure > models > product_model.py > ProductModel > Meta
1  from django.db import models
2
3  class ProductModel(models.Model):
4      name = models.TextField()
5      category_id = models.IntegerField()
6      price = models.DecimalField(max_digits=10, decimal_places=2)
7      stock = models.IntegerField(default=0)
8      attributes = models.JSONField(default=dict)
9
10     class Meta:
11         db_table = 'products'
12         verbose_name = 'Product'
13         verbose_name_plural = 'Products'
14
15     def __str__(self):
16         return self.name
17
18
19     class TagModel(models.Model):
20         name = models.CharField(max_length=100, unique=True)
21         slug = models.SlugField(max_length=100, unique=True)
22
23         class Meta:
24             db_table = 'tags'
25
26         def __str__(self):
27             return self.name
28
29
30     class ProductTagModel(models.Model):
31         product = models.ForeignKey(ProductModel, on_delete=models.CASCADE, related_name='product_tags')
32         tag = models.ForeignKey(TagModel, on_delete=models.CASCADE, related_name='product_tags')
33
34         class Meta:
35             db_table = 'product_tags'
36             unique_together = ('product', 'tag')
37
38
39     class ProductImageModel(models.Model):
40         product = models.ForeignKey(ProductModel, on_delete=models.CASCADE, related_name='images')
41         image_url = models.URLField(max_length=500)
42         order = models.PositiveSmallIntegerField(default=0)
43
44         class Meta:
45             db_table = 'product_images'
46             ordering = ['order']
47

```

Hình 2.14: Cấu trúc mã nguồn Model của Product Service

2.5.4 Thiết kế API

Dưới đây là bảng liệt kê các API chính được cung cấp bởi Product Service:

Phương thức	Endpoint	Mô tả chức năng
GET	/products/	Liệt kê sản phẩm (hỗ trợ lọc, tìm kiếm, sắp xếp)
POST	/products/	Tạo mới một sản phẩm
GET	/products/{id}/	Xem thông tin chi tiết sản phẩm
PUT	/products/{id}/	Cập nhật thông tin sản phẩm và lưu lịch sử
DELETE	/products/{id}/	Xóa sản phẩm khỏi hệ thống
PATCH	/products/{id}/stock/	Cập nhật nhanh số lượng tồn kho
GET	/products/{id}/history/	Truy xuất lịch sử thay đổi của sản phẩm
GET/POST	/categories/	Quản lý danh sách các danh mục sản phẩm
GET/POST	/brands/	Quản lý danh sách các thương hiệu
GET/POST	/tags/	Quản lý các thẻ (tags) phân loại

Bảng 2.1: Bảng thiết kế API của Product Service

2.5.5 Luồng xử lý

Các chức năng chính và luồng xử lý trong Product Service bao gồm:

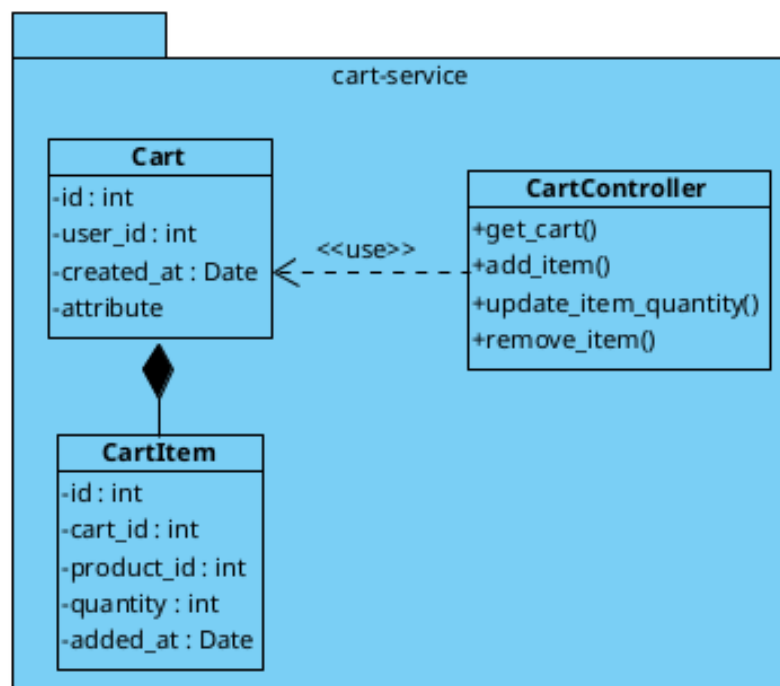
- **Quản lý danh mục và thuộc tính:** Định nghĩa cấu trúc ngành hàng và các thuộc tính đi kèm.
- **Quản lý thông tin sản phẩm:** Tiếp nhận và xử lý các yêu cầu CRUD (Thêm, Sửa, Xóa, Truy vấn) sản phẩm.
- **Tìm kiếm và Lọc:** Xử lý các truy vấn phức tạp dựa trên từ khóa, khoảng giá, danh mục và sắp xếp.
- **Cập nhật tồn kho:** Xử lý các yêu cầu điều chỉnh số lượng hàng hóa trong kho.
- **Quản lý hình ảnh:** Lưu trữ và quản lý thứ tự hiển thị hình ảnh sản phẩm.
- **Theo dõi lịch sử:** Tự động ghi lại các thay đổi quan trọng đối với thông tin sản phẩm để phục vụ kiểm tra (audit).

2.6 Cart Service

Cart Service chịu trách nhiệm quản lý giỏ hàng tạm thời của người dùng, cho phép thêm, xóa và cập nhật số lượng sản phẩm trước khi tiến hành đặt hàng.

2.6.1 Sơ đồ lớp thiết kế

Dưới đây là sơ đồ lớp chi tiết của Cart Service:



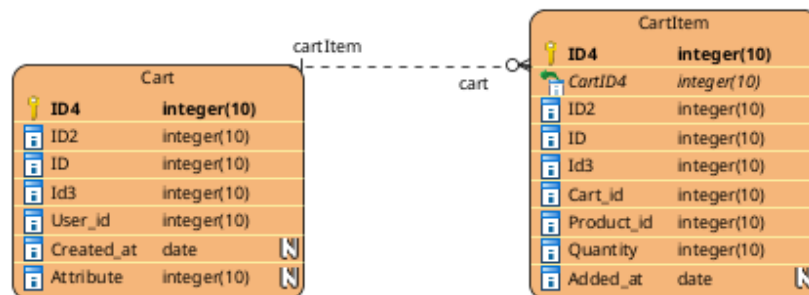
Hình 2.15: Sơ đồ lớp thiết kế Cart Service

Sơ đồ lớp bao gồm:

- **CartModel:** Thực thể chính đại diện cho giỏ hàng của một người dùng.
- **CartItemModel:** Chi tiết các sản phẩm trong giỏ hàng, bao gồm tham chiếu đến sản phẩm và số lượng.

2.6.2 Data Model

Dưới đây là sơ đồ Data Model (Physical Schema) của Cart Service:



Hình 2.16: Sơ đồ Data Model Cart Service

2.6.3 Django Model

Mô hình dữ liệu của Cart Service tập trung vào việc lưu trữ trạng thái mua sắm của khách hàng.

```
ecommerce-service > cart_service > app > models.py > CartItem > quantity
1  from django.db import models
2
3
4  class Cart(models.Model):
5      user_id = models.IntegerField()
6      created_at = models.DateTimeField(auto_now_add=True)
7      attribute = models.JSONField(default=dict, blank=True)
8
9
10 class CartItem(models.Model):
11     cart = models.ForeignKey(Cart, on_delete=models.CASCADE, related_name='items')
12     product_id = models.IntegerField()
13     quantity = models.IntegerField(default=1)
14     added_at = models.DateTimeField(auto_now_add=True)
15
```

Hình 2.17: Cấu trúc mã nguồn Model của Cart Service

2.6.4 Thiết kế API

Dưới đây là bảng liệt kê các API chính được cung cấp bởi Cart Service:

Phương thức	Endpoint	Mô tả chức năng
GET	/carts/	Lấy thông tin giỏ hàng của người dùng hiện tại
POST	/carts/	Khởi tạo hoặc cập nhật giỏ hàng
GET	/cartitems/	Liệt kê toàn bộ sản phẩm trong giỏ hàng
POST	/cartitems/	Thêm sản phẩm mới vào giỏ hàng
PUT/PATCH	/cartitems/{id}/	Cập nhật số lượng của một sản phẩm trong giỏ
DELETE	/cartitems/{id}/	Loại bỏ sản phẩm khỏi giỏ hàng

Bảng 2.2: Bảng thiết kế API của Cart Service

2.6.5 Luồng xử lý

Các chức năng chính và luồng xử lý trong Cart Service bao gồm:

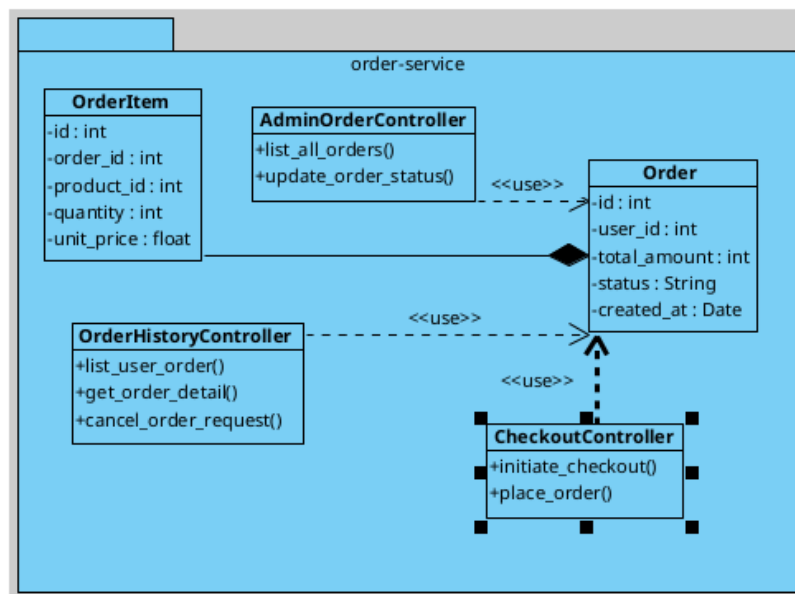
- **Quản lý giỏ hàng:** Tự động khởi tạo giỏ hàng khi người dùng thêm sản phẩm đầu tiên.
- **Đồng bộ số lượng:** Kiểm tra và cập nhật số lượng mua dựa trên yêu cầu của khách hàng.
- **Dọn dẹp giỏ hàng:** Xử lý việc làm trống giỏ hàng sau khi đơn hàng đã được tạo thành công.

2.7 Order Service

Order Service là dịch vụ quan trọng xử lý quy trình đặt hàng, tính toán tổng giá trị đơn hàng và quản lý vòng đời của đơn hàng từ khi khởi tạo đến khi hoàn tất.

2.7.1 Sơ đồ lớp thiết kế

Dưới đây là sơ đồ lớp chi tiết của Order Service:



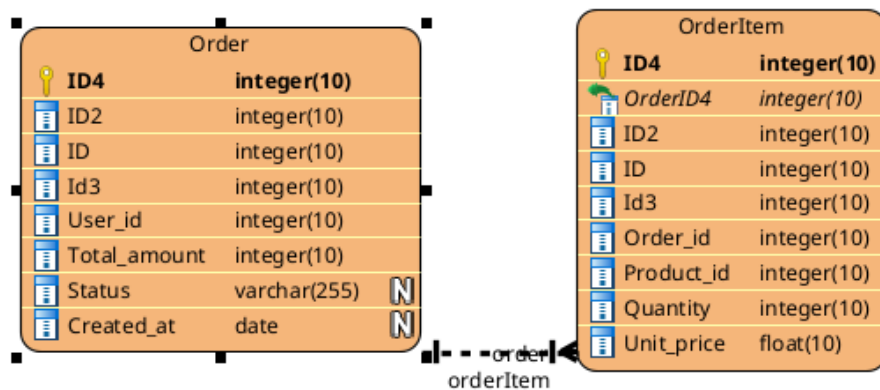
Hình 2.18: Sơ đồ lớp thiết kế Order Service

Sơ đồ lớp bao gồm:

- **OrderModel:** Lưu trữ thông tin tổng quát của đơn hàng (trạng thái, tổng tiền, ngày tạo).
- **OrderItemModel:** Lưu trữ thông tin chi tiết từng sản phẩm trong đơn hàng tại thời điểm mua (giá chốt, số lượng).

2.7.2 Data Model

Dưới đây là sơ đồ Data Model (Physical Schema) của Order Service:



Hình 2.19: Sơ đồ Data Model Order Service

2.7.3 Django Model

Mô hình dữ liệu của Order Service được thiết kế để đảm bảo tính toàn vẹn của dữ liệu giao dịch.


```
ecommerce-service > order_service > app > models.py > ...
1 from django.db import models
2
3
4 class Order(models.Model):
5     STATUS_CHOICES = [
6         ('PENDING', 'Chờ xử lý'),
7         ('CONFIRMED', 'Đã xác nhận'),
8         ('SHIPPING', 'Đang giao'),
9         ('DELIVERED', 'Đã giao'),
10        ('CANCELLED', 'Đã hủy'),
11    ]
12    user_id = models.IntegerField()
13    total_price = models.DecimalField(max_digits=10, decimal_places=2)
14    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='PENDING')
15    created_at = models.DateTimeField(auto_now_add=True)
16
17    def __str__(self):
18        return f"Order#{self.pk} user={self.user_id} status={self.status}"
19
20
21 class OrderItem(models.Model):
22     order = models.ForeignKey(Order, on_delete=models.CASCADE, related_name='items')
23     product_id = models.IntegerField()
24     quantity = models.IntegerField(default=1)
25     unit_price = models.DecimalField(max_digits=10, decimal_places=2)
26
27    def __str__(self):
28        return f"OrderItem#{self.pk} product={self.product_id} qty={self.quantity}"
29
```

Hình 2.20: Cấu trúc mã nguồn Model của Order Service

2.7.4 Thiết kế API

Dưới đây là bảng liệt kê các API chính được cung cấp bởi Order Service:

Phương thức	Endpoint	Mô tả chức năng
POST	/orders/	Thực hiện Checkout và tạo đơn hàng mới
GET	/orders/	Xem lịch sử danh sách đơn hàng của người dùng
GET	/orders/{id}/	Xem chi tiết trạng thái và các mục của một đơn hàng
POST	/orders/{id}/cancel/	Gửi yêu cầu hủy đơn hàng
GET	/orderitems/	Truy xuất chi tiết các mặt hàng trong đơn hàng

Bảng 2.3: Bảng thiết kế API của Order Service

2.7.5 Luồng xử lý

Các chức năng chính và luồng xử lý trong Order Service bao gồm:

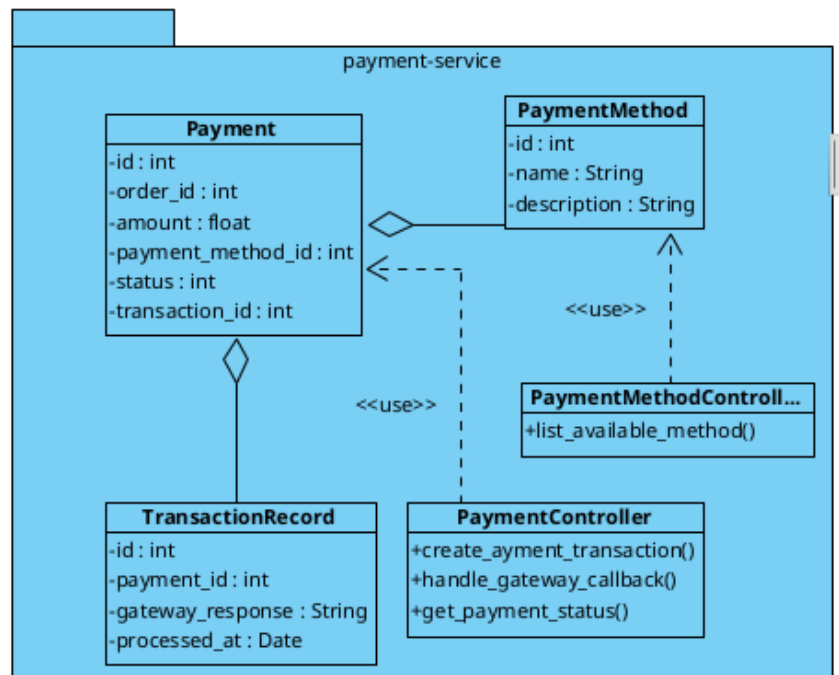
- **Quy trình Checkout:** Chuyển đổi dữ liệu từ Cart Service sang Order Service và xác thực tồn kho.
- **Quản lý trạng thái:** Cập nhật trạng thái đơn hàng (PENDING, CONFIRMED, SHIPPING, DELIVERED).
- **Xử lý hủy đơn:** Thực hiện các nghiệp vụ hoàn trả tồn kho khi đơn hàng bị hủy.

2.8 Payment Service

Payment Service chịu trách nhiệm tích hợp các cổng thanh toán và quản lý các giao dịch tài chính của hệ thống.

2.8.1 Sơ đồ lớp thiết kế

Dưới đây là sơ đồ lớp chi tiết của Payment Service:



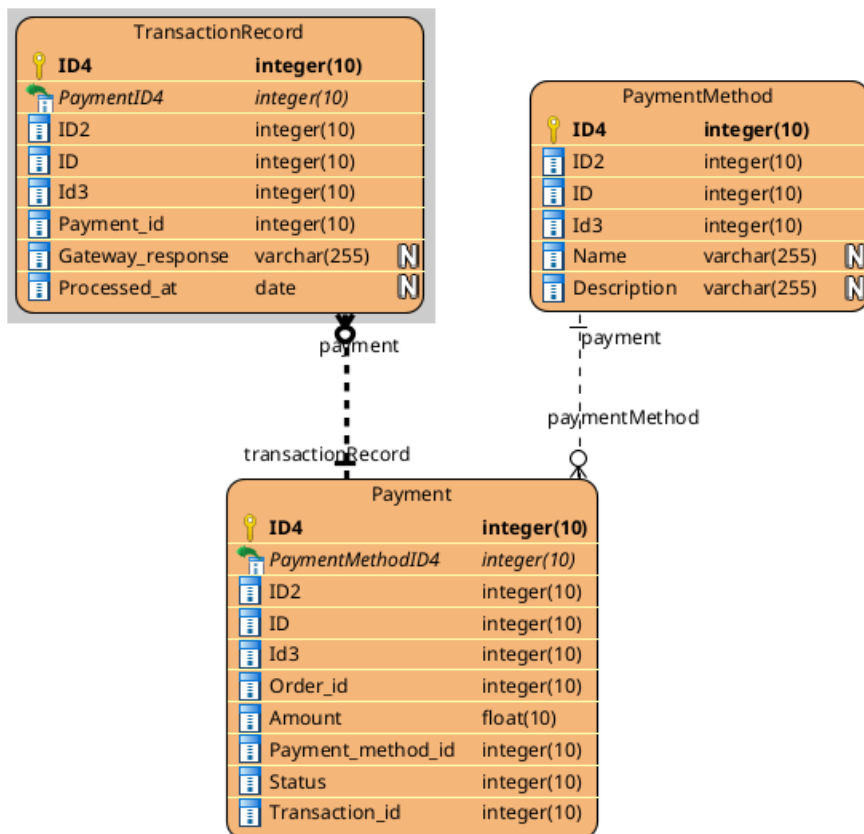
Hình 2.21: Sơ đồ lớp thiết kế Payment Service

Sơ đồ lớp bao gồm:

- **PaymentModel:** Thông tin thanh toán liên kết với đơn hàng.
- **PaymentMethodModel:** Danh sách các phương thức hỗ trợ (COD, BANK_TRANSFER, ...).
- **TransactionRecordModel:** Nhật ký chi tiết các phản hồi từ cổng thanh toán.

2.8.2 Data Model

Dưới đây là sơ đồ Data Model (Physical Schema) của Payment Service:



Hình 2.22: Sơ đồ Data Model Payment Service

2.8.3 Django Model

Mô hình dữ liệu của Payment Service tập trung vào tính bảo mật và khả năng đối soát.

```
ecommerce-service > payment_service > app > models.py > ...
1 from django.db import models
2
3
4 class PaymentMethod(models.Model):
5     name = models.CharField(max_length=50, unique=True)
6     description = models.TextField(blank=True)
7     is_active = models.BooleanField(default=True)
8
9     def __str__(self):
10         return self.name
11
12
13 class Payment(models.Model):
14     STATUS_CHOICES = [
15         ('PENDING', 'Chờ thanh toán'),
16         ('SUCCESS', 'Thanh toán thành công'),
17         ('FAILED', 'Thanh toán thất bại'),
18     ]
19     METHOD_CHOICES = [
20         ('COD', 'Thanh toán khi nhận hàng'),
21         ('BANK_TRANSFER', 'Chuyển khoản ngân hàng'),
22         ('CREDIT_CARD', 'Thẻ tín dụng / ghi nợ'),
23         ('EWALLET', 'Ví điện tử'),
24     ]
25     order_id = models.IntegerField()
26     amount = models.DecimalField(max_digits=10, decimal_places=2)
27     method = models.CharField(max_length=30, choices=METHOD_CHOICES, default='COD')
28     payment_method = models.ForeignKey(
29         PaymentMethod, on_delete=models.SET_NULL, null=True, blank=True,
30         related_name='payments'
31     )
32     status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='PENDING')
33     created_at = models.DateTimeField(auto_now_add=True, null=True)
34
35     def __str__(self):
36         return f"Payment#{self.pk} order={self.order_id} status={self.status}"
37
38
39 class TransactionRecord(models.Model):
40     payment = models.ForeignKey(Payment, on_delete=models.CASCADE, related_name='transactions')
41     gateway_response = models.JSONField(default=dict)
42     processed_at = models.DateTimeField(auto_now_add=True)
43
44     def __str__(self):
45         return f"Transaction#{self.pk} payment={self.payment_id}"
46
```

Hình 2.23: Cấu trúc mã nguồn Model của Payment Service

2.8.4 Thiết kế API

Dưới đây là bảng liệt kê các API chính được cung cấp bởi Payment Service:

Phương thức	Endpoint	Mô tả chức năng
GET	/payment-methods/	Lấy danh sách các phương thức thanh toán khả dụng
POST	/payments/	Khởi tạo một giao dịch thanh toán mới
GET	/payments/{id}/	Kiểm tra trạng thái của một giao dịch
POST	/payments/{id}/callback/	Tiếp nhận phản hồi từ cổng thanh toán bên ngoài

Bảng 2.4: Bảng thiết kế API của Payment Service

2.8.5 Luồng xử lý

Các chức năng chính và luồng xử lý trong Payment Service bao gồm:

- **Khởi tạo giao dịch:** Tạo yêu cầu thanh toán và chuyển hướng đến cổng thanh toán tương ứng.

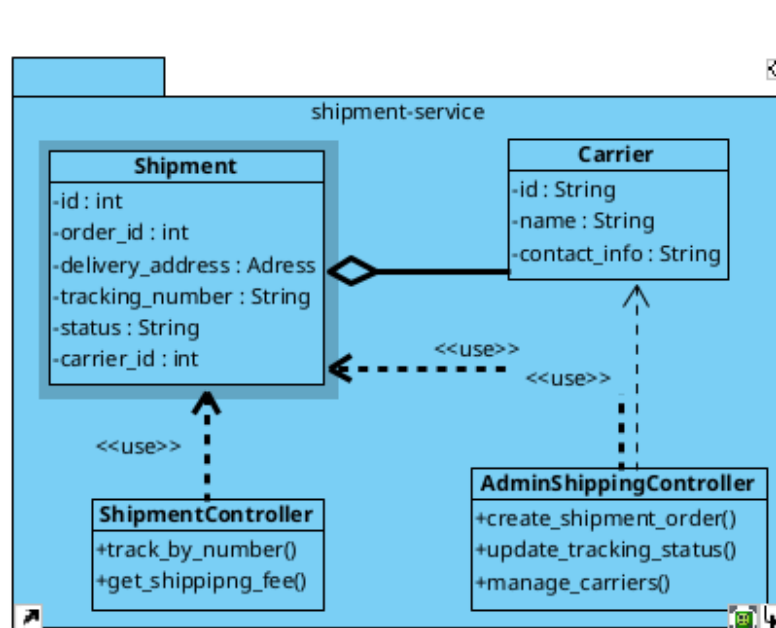
- **Xử lý Callback:** Xác thực chữ ký số và cập nhật kết quả giao dịch vào hệ thống.
- **Đổi soát giao dịch:** Ghi lại toàn bộ lịch sử tương tác với cổng thanh toán để phục vụ kiểm toán.

2.9 Shipping Service

Shipping Service quản lý việc giao hàng, theo dõi lộ trình và tích hợp với các đơn vị vận chuyển đối tác.

2.9.1 Sơ đồ lớp thiết kế

Dưới đây là sơ đồ lớp chi tiết của Shipping Service:



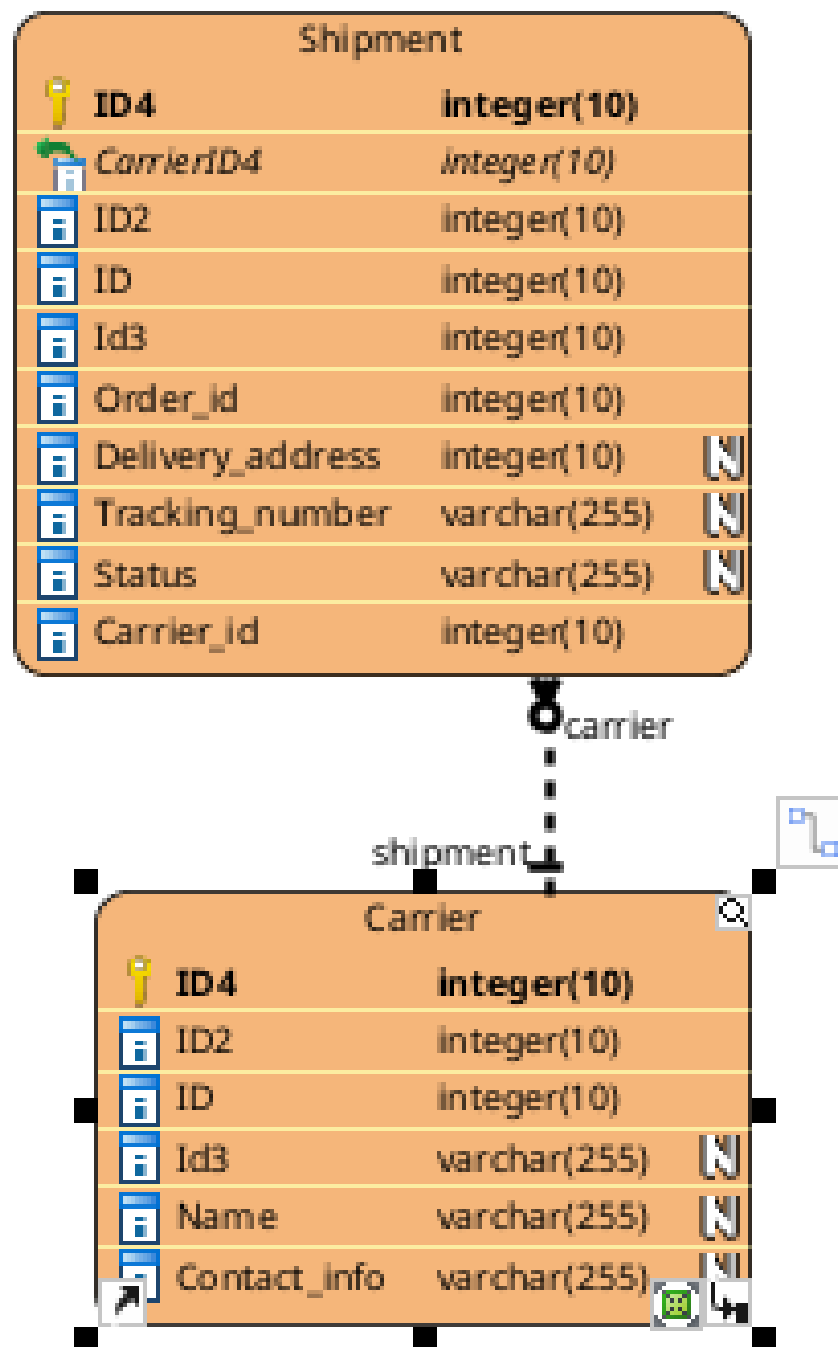
Hình 2.24: Sơ đồ lớp thiết kế Shipping Service

Sơ đồ lớp bao gồm:

- **ShipmentModel:** Thông tin vận chuyển cho từng đơn hàng (địa chỉ, mã vận đơn, trạng thái).
- **CarrierModel:** Thông tin về các đối tác vận chuyển.

2.9.2 Data Model

Dưới đây là sơ đồ Data Model (Physical Schema) của Shipping Service:



Hình 2.25: Sơ đồ Data Model Shipping Service

2.9.3 Django Model

Mô hình dữ liệu của Shipping Service được thiết kế để theo dõi sát sao hành trình của kiện hàng.

```
ecommerce-service > shipping_service > app > models.py > ...
1  from django.db import models
2
3
4  class Carrier(models.Model):
5      name = models.CharField(max_length=100)
6      contact_info = models.TextField(blank=True)
7      is_active = models.BooleanField(default=True)
8
9      def __str__(self):
10         return self.name
11
12
13  class Shipment(models.Model):
14      METHOD_CHOICES = [
15          ('STANDARD', 'Giao hàng tiêu chuẩn'),
16          ('EXPRESS', 'Giao hàng nhanh'),
17          ('SAME_DAY', 'Giao trong ngày'),
18      ]
19      STATUS_CHOICES = [
20          ('PENDING', 'Chờ xử lý'),
21          ('PROCESSING', 'Đang chuẩn bị'),
22          ('SHIPPING', 'Đang vận chuyển'),
23          ('DELIVERED', 'Đã giao thành công'),
24          ('FAILED', 'Giao thất bại'),
25      ]
26      order_id = models.IntegerField()
27      address = models.TextField(default='')
28      method = models.CharField(max_length=20, choices=METHOD_CHOICES, default='STANDARD')
29      tracking_number = models.CharField(max_length=100, blank=True, default='')
30      status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='PENDING')
31      carrier = models.ForeignKey(
32          Carrier, on_delete=models.SET_NULL, null=True, blank=True,
33          related_name='shipments'
34      )
35      created_at = models.DateTimeField(auto_now_add=True, null=True)
36
37      def __str__(self):
38         return f"Shipment#{self.pk} order={self.order_id} tracking={self.tracking_number}"
39
```

Hình 2.26: Cấu trúc mã nguồn Model của Shipping Service

2.9.4 Thiết kế API

Dưới đây là bảng liệt kê các API chính được cung cấp bởi Shipping Service:

Phương thức	Endpoint	Mô tả chức năng
GET	/shipments/track/	Tra cứu lộ trình kiện hàng qua mã vận đơn
GET	/shipments/fee/	Tính toán phí vận chuyển dự kiến
POST	/shipments/	Tạo yêu cầu vận chuyển mới (Admin)
GET	/carriers/	Lấy danh sách các đơn vị vận chuyển

Bảng 2.5: Bảng thiết kế API của Shipping Service

2.9.5 Luồng xử lý

Các chức năng chính và luồng xử lý trong Shipping Service bao gồm:

- **Tính phí vận chuyển:** Dựa trên khoảng cách, khối lượng và phương thức vận chuyển.
- **Theo dõi vận đơn:** Cập nhật liên tục trạng thái từ đơn vị vận chuyển đến người dùng.

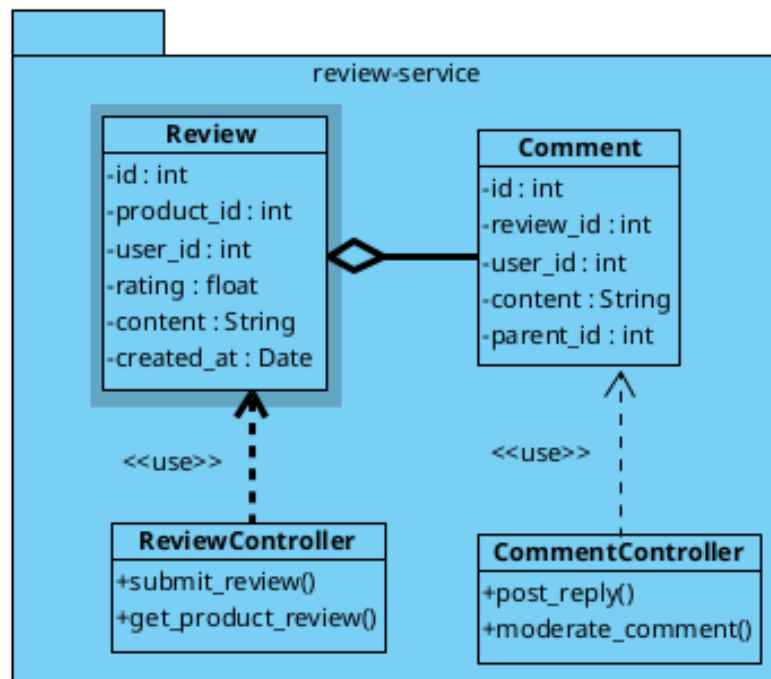
- **Điều phối vận chuyển:** Tự động lựa chọn carrier phù hợp nhất theo cấu hình hệ thống.

2.10 Comment Service

Comment Service quản lý các tương tác của người dùng bao gồm đánh giá sản phẩm, bình luận và các phản hồi thảo luận.

2.10.1 Sơ đồ lớp thiết kế

Dưới đây là sơ đồ lớp chi tiết của Comment Service:



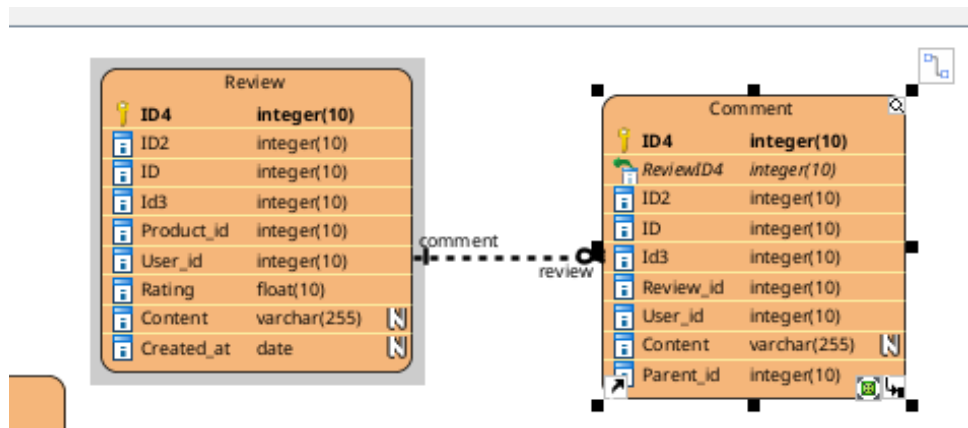
Hình 2.27: Sơ đồ lớp thiết kế Comment Service

Sơ đồ lớp bao gồm:

- **ReviewModel:** Lưu trữ đánh giá sản phẩm kèm số sao từ khách hàng.
- **CommentModel:** Lưu trữ các bình luận và phản hồi theo dạng phân cấp (threaded comments).

2.10.2 Data Model

Dưới đây là sơ đồ Data Model (Physical Schema) của Comment Service:



Hình 2.28: Sơ đồ Data Model Comment Service

2.10.3 Django Model

Mô hình dữ liệu của Comment Service hỗ trợ các cấu trúc tương tác đa cấp.

```

ecommerce-service > comment_service > app > models.py > ...
1  from django.db import models
2
3
4  class Review(models.Model):
5      """Đánh giá sản phẩm kèm số sao (ReviewController)."""
6      product_id = models.IntegerField()
7      user_id = models.IntegerField()
8      rating = models.PositiveSmallIntegerField() # 1-5
9      content = models.TextField()
10     is_visible = models.BooleanField(default=True)
11     created_at = models.DateTimeField(auto_now_add=True)
12
13     class Meta:
14         unique_together = ('product_id', 'user_id')
15
16     def __str__(self):
17         return f"Review#{self.pk} product={self.product_id} rating={self.rating}"
18
19
20     class Comment(models.Model):
21         """Bình luận / phản hồi đánh giá (CommentController)."""
22         product_id = models.IntegerField()
23         review = models.ForeignKey(
24             Review, on_delete=models.CASCADE, related_name='comments',
25             null=True, blank=True
26         )
27         user_id = models.IntegerField()
28         content = models.TextField()
29         parent = models.ForeignKey(
30             'self', on_delete=models.CASCADE, null=True, blank=True,
31             related_name='replies'
32         )
33         is_visible = models.BooleanField(default=True)
34         created_at = models.DateTimeField(auto_now_add=True)
35
36     def __str__(self):
37         return f"Comment#{self.pk} product={self.product_id}"
38
39

```

Hình 2.29: Cấu trúc mã nguồn Model của Comment Service

2.10.4 Thiết kế API

Dưới đây là bảng liệt kê các API chính được cung cấp bởi Comment Service:

Phương thức	Endpoint	Mô tả chức năng
POST	/reviews/	Gửi đánh giá và số sao cho sản phẩm
GET	/reviews/	Xem danh sách đánh giá của sản phẩm
POST	/reviews/{id}/moderate/	Kiểm duyệt nội dung đánh giá (Ẩn/Hiện)
POST	/comments/	Gửi bình luận hoặc phản hồi
POST	/comments/{id}/moderate/	Kiểm duyệt nội dung bình luận

Bảng 2.6: Bảng thiết kế API của Comment Service

2.10.5 Luồng xử lý

Các chức năng chính và luồng xử lý trong Comment Service bao gồm:

- **Quản lý tương tác:** Xử lý các quy tắc gửi bài và ngăn chặn spam cơ bản.
- **Cấu trúc phân cấp:** Tổ chức các phản hồi theo dạng cây để người dùng dễ theo dõi.
- **Kiểm duyệt nội dung:** Cho phép nhân viên quản trị ẩn các nội dung không phù hợp.

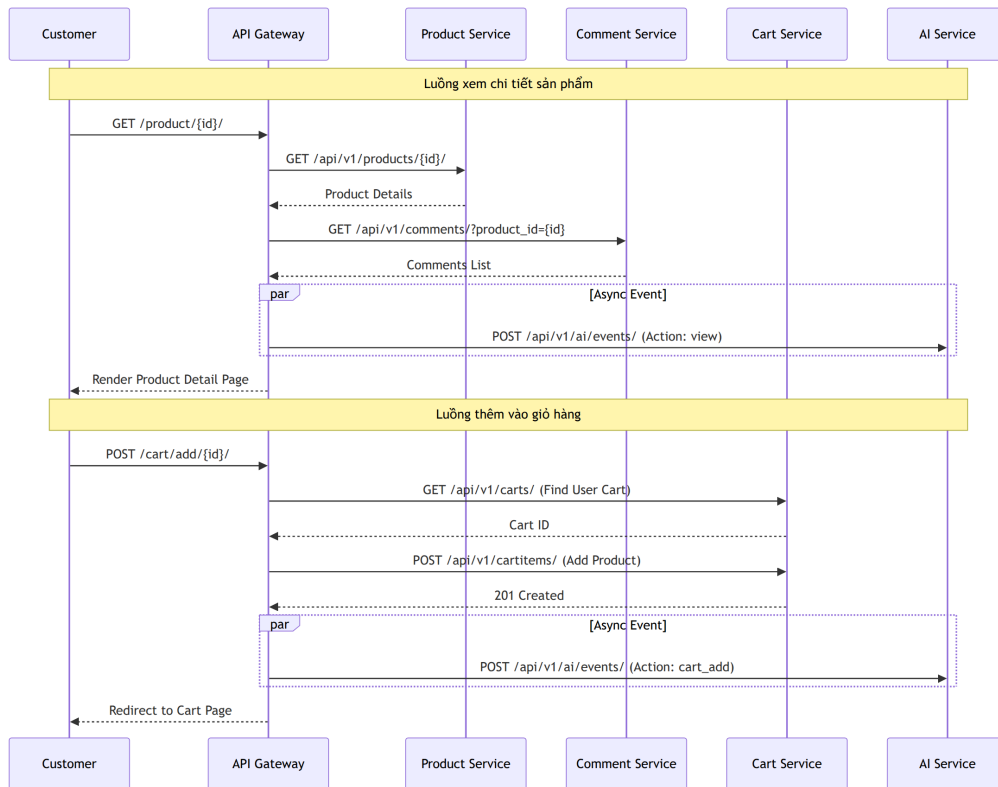
2.11 Luồng hệ thống tổng thể

Hệ thống E-Commerce Microservices vận hành dựa trên sự phối hợp chặt chẽ giữa các dịch vụ thông qua API Gateway. Dưới đây là các luồng nghiệp vụ cốt lõi thể hiện sự tương tác liên dịch vụ:

2.11.1 Luồng xem sản phẩm và Thêm vào giỏ hàng (Product View & Cart Flow)

Hành trình của khách hàng bắt đầu từ việc khám phá sản phẩm và đưa vào giỏ hàng:

- **Xem chi tiết:** Khi khách hàng truy cập trang sản phẩm, Gateway gọi Product Service để lấy thông tin kỹ thuật và Comment Service để lấy các đánh giá.
- **Ghi nhận hành vi:** Một sự kiện "view" được gửi bất đồng bộ sang AI Service để phục vụ thuật toán gợi ý.
- **Thêm vào giỏ:** Khi khách hàng nhấn "Thêm vào giỏ", Gateway sẽ tương tác với Cart Service để cập nhật trạng thái giỏ hàng của người dùng.

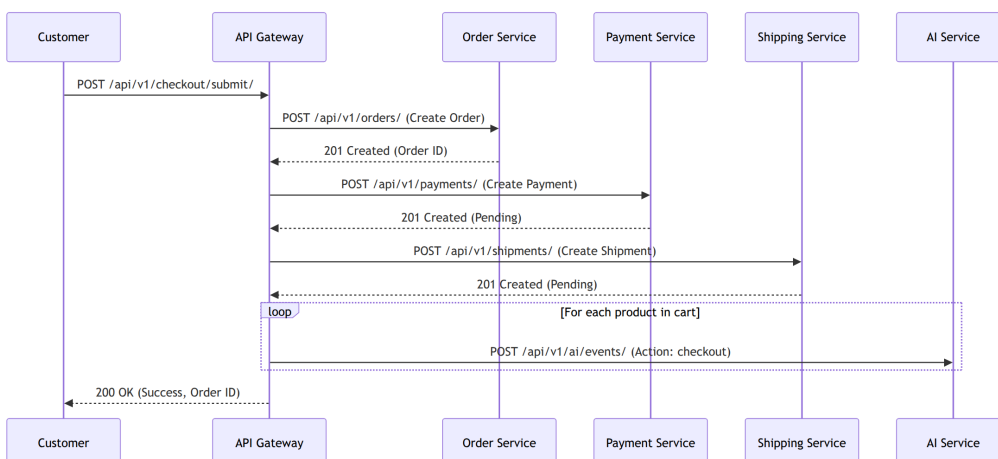


Hình 2.30: Sơ đồ tuần tự luồng xem sản phẩm và thêm vào giỏ hàng

2.11.2 Luồng mua sắm và Thanh toán (Purchase & Checkout Flow)

Đây là luồng quan trọng nhất, kết hợp dữ liệu từ nhiều service để hoàn tất đơn hàng.

- **Khởi tạo:** Khách hàng gửi yêu cầu checkout từ API Gateway.
- **Điều phối:** Gateway gọi Order Service để tạo đơn hàng, sau đó gọi Payment Service để khởi tạo giao dịch và Shipping Service để tạo vận đơn.
- **Hoàn tất:** Thông tin được trả về cho khách hàng và sự kiện checkout được gửi sang AI Service để phân tích.

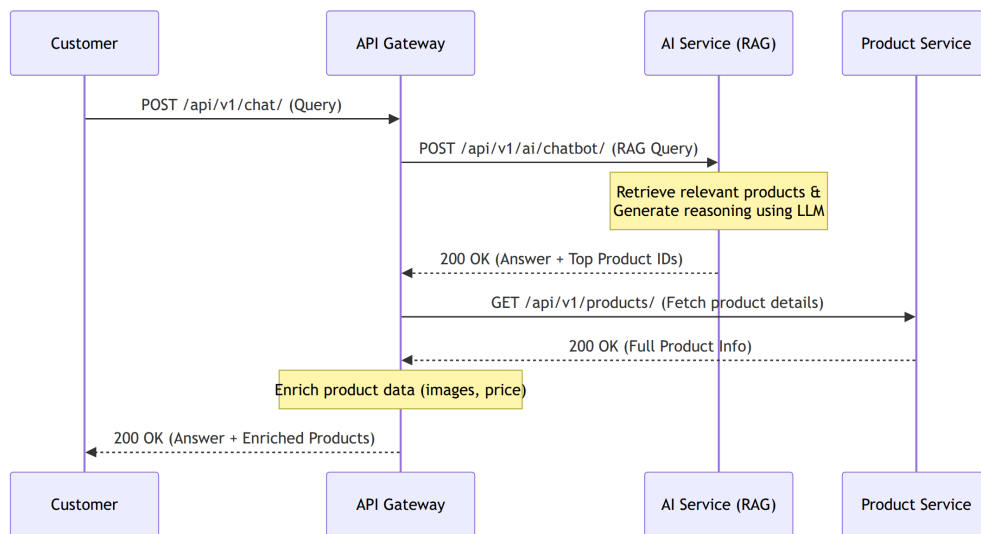


Hình 2.31: Sơ đồ tuần tự luồng mua sắm và thanh toán

2.11.3 Hệ thống tư vấn thông minh (AI Chatbot Flow)

Luồng này thể hiện khả năng tích hợp RAG (Retrieval-Augmented Generation) để hỗ trợ khách hàng.

- **Truy vấn:** Khách hàng đặt câu hỏi về sản phẩm qua giao diện chat.
- **Xử lý AI:** AI Service tìm kiếm sản phẩm liên quan trong cơ sở dữ liệu tri thức và tạo câu trả lời kèm lý giải (reasoning).
- **Làm giàu dữ liệu:** Gateway nhận danh sách ID sản phẩm từ AI, gọi Product Service để lấy thông tin chi tiết (ảnh, giá) trước khi hiển thị cho người dùng.

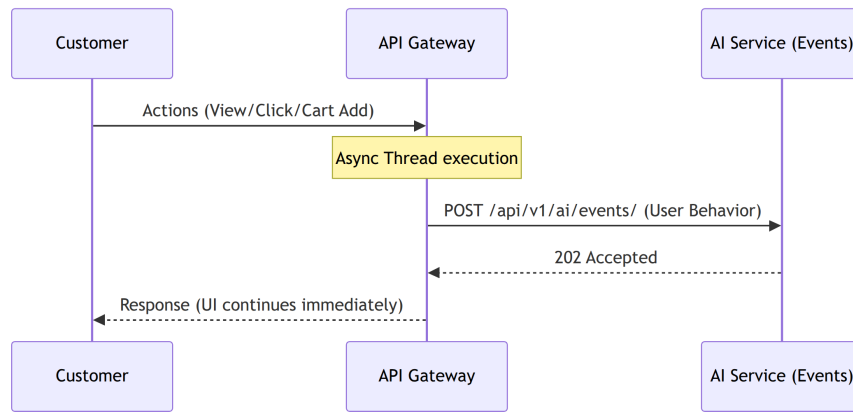


Hình 2.32: Sơ đồ tuần tự luồng tư vấn AI

2.11.4 Theo dõi hành vi và Gợi ý (Behavior Tracking Flow)

Để tối ưu hóa gợi ý, hệ thống theo dõi mọi tương tác của người dùng một cách bất đồng bộ.

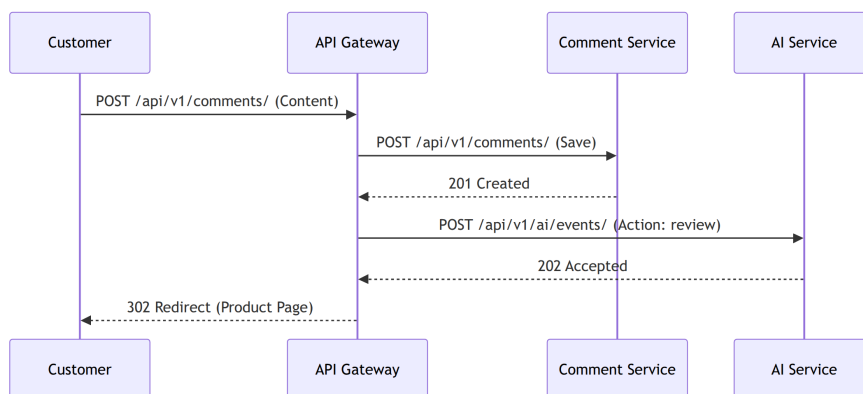
- **Bắt sự kiện:** Các hành động View, Click, Add-to-cart được Gateway ghi nhận.
- **Xử lý nền:** Gateway sử dụng Multi-threading để gửi dữ liệu hành vi sang AI Service mà không làm gián đoạn trải nghiệm người dùng (Fire-and-forget).



Hình 2.33: Sơ đồ tuần tự luồng theo dõi hành vi

2.11.5 Quản lý Đánh giá và Bình luận (Product Review Flow)

- **Lưu trữ:** Bình luận được gửi đến Comment Service để lưu trữ và phân cấp.
- **Phản hồi:** Sau khi lưu thành công, một sự kiện 'review' được gửi sang AI Service để cập nhật hồ sơ sở thích của người dùng.



Hình 2.34: Sơ đồ tuần tự luồng đánh giá sản phẩm

2.12 Các hệ quản trị cơ sở dữ liệu được sử dụng

Trong kiến trúc Microservices của hệ thống, chúng tôi áp dụng chiến lược *Polyglot Persistence* - sử dụng nhiều loại cơ sở dữ liệu khác nhau tùy theo đặc thù của từng service để tối ưu hóa hiệu năng và khả năng mở rộng.

2.12.1 MySQL

MySQL được lựa chọn làm cơ sở dữ liệu cho các service như **User Service**, **Payment Service** và **Shipping Service**.

- **Lý do sử dụng:** Đây là những service có cấu trúc dữ liệu quan hệ chặt chẽ, yêu cầu tính ổn định cao và các giao dịch tài chính (ACID) tiêu chuẩn. MySQL cung cấp hiệu năng đọc/ghi cực tốt cho các thao tác OLTP (Online Transactional Processing)

thông thường và có cộng đồng hỗ trợ lớn, dễ dàng triển khai trên các môi trường cloud.

- **Đặc điểm:** Sử dụng Storage Engine InnoDB để đảm bảo an toàn dữ liệu qua cơ chế khóa dòng (row-level locking) và hỗ trợ khóa ngoại (foreign keys) mạnh mẽ.

2.12.2 PostgreSQL

PostgreSQL được sử dụng cho các service trọng yếu như **Product Service, Order Service, Cart Service** và **AI Service**.

- **Lý do sử dụng:**
 - Đối với **Product Service:** PostgreSQL có khả năng xử lý kiểu dữ liệu JSONB vượt trội. Điều này cho phép lưu trữ các thuộc tính sản phẩm động (như cấu hình máy tính, thông số quần áo) trong cùng một cột mà vẫn có thể đánh chỉ mục (GIN Index) để tìm kiếm nhanh chóng.
 - Đối với **Order Service:** Yêu cầu tính toàn vẹn dữ liệu cực kỳ khắt khe và các câu lệnh truy vấn phức tạp để thống kê báo cáo.
 - Đối với **AI Service:** PostgreSQL hỗ trợ các extension mở rộng (như pgvector) giúp lưu trữ và truy vấn vector phục vụ cho RAG trong tương lai.

2.12.3 So sánh MySQL và PostgreSQL

Dưới đây là bảng so sánh các đặc tính kỹ thuật chính dẫn đến quyết định lựa chọn trong hệ thống:

Đặc tính	MySQL	PostgreSQL
Kiểu dữ liệu	Hỗ trợ các kiểu cơ bản, hỗ trợ JSON hạn chế hơn.	Hỗ trợ cực mạnh JSONB, mảng (Array), và kiểu tùy chỉnh.
Hiệu năng	Tối ưu cho các truy vấn đọc đơn giản, tốc độ cao.	Tối ưu cho các truy vấn phức tạp, tính toán nhiều và xử lý đồng thời.
Khả năng mở rộng	Chủ yếu mở rộng theo chiều dọc hoặc Master-Slave.	Hỗ trợ tốt hơn cho các tính năng nâng cao, phân vùng dữ liệu (Partitioning).
Tính năng ACID	Tuân thủ tốt qua InnoDB.	Tuân thủ cực kỳ khắt khe, mặc định an toàn dữ liệu cao hơn.
Ứng dụng trong dự án	User, Payment, Shipping, Staff Service.	Product, Order, Cart, Comment, AI Service.

Bảng 2.7: So sánh MySQL và PostgreSQL trong hệ thống

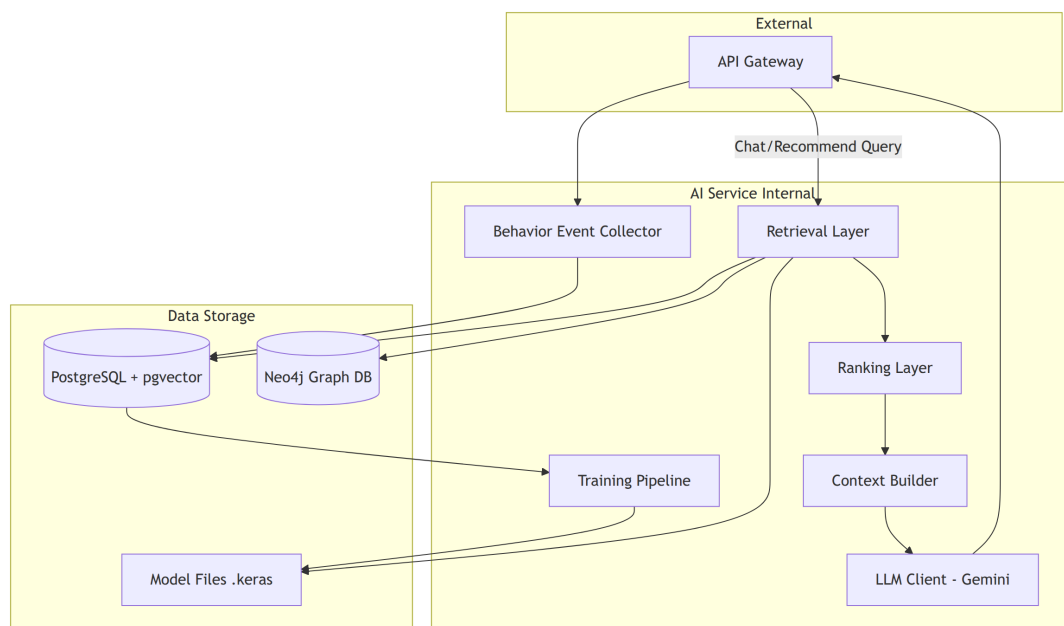
2.13 Kết luận

CHƯƠNG 3. AI Service cho tư vấn sản phẩm

3.1 Mô tả yêu cầu hệ thống AI

Hệ thống AI Service được thiết kế để giải quyết bài toán tư vấn và gợi ý sản phẩm cá nhân hóa trong môi trường thương mại điện tử (E-commerce). Các yêu cầu chính bao gồm:

- **Xây dựng Pipeline dữ liệu:** Thu thập và xử lý các sự kiện hành vi người dùng (view, click, cart, checkout) theo thời gian thực để tạo ra các chuỗi phiên (sessions).
- **Áp dụng Deep Learning:** Sử dụng các mô hình học sâu để nắm bắt ý định mua sắm và dự đoán sản phẩm tiếp theo dựa trên chuỗi hành vi lịch sử.
- **Tích hợp RAG Chatbot:** Kết hợp khả năng truy xuất thông tin (Retrieval) từ cơ sở dữ liệu tri thức (Knowledge Base) với mô hình ngôn ngữ lớn (LLM) để hỗ trợ hội thoại tự nhiên và tư vấn chuyên sâu.
- **Đảm bảo hiệu năng:** Phân tách luồng gợi ý nhanh (không LLM) và luồng tư vấn chi tiết (có LLM) để tối ưu hóa tài nguyên hệ thống.



Hình 3.1: Kiến trúc tổng thể của AI Service và các thành phần chính

3.2 Deep Learning trong tư vấn sản phẩm

3.2.1 Các mô hình sử dụng

Hệ thống sử dụng kết hợp hai kiến trúc mô hình chính:

- **LSTM (Long Short-Term Memory):** Xử lý dữ liệu chuỗi thời gian, giúp nắm bắt

các phụ thuộc dài hạn trong hành vi người dùng.

- **Embedding Models:** Sử dụng mô hình đa ngôn ngữ để chuyển đổi văn bản thành các vector nhúng (384 chiều) phục vụ cho tìm kiếm ngữ nghĩa.

3.2.2 Cấu trúc mã nguồn và Quy trình huấn luyện

Mã nguồn phần huấn luyện được tổ chức tại thư mục `app/training/`. Quy trình huấn luyện mô hình LSTM được thực hiện qua các bước:

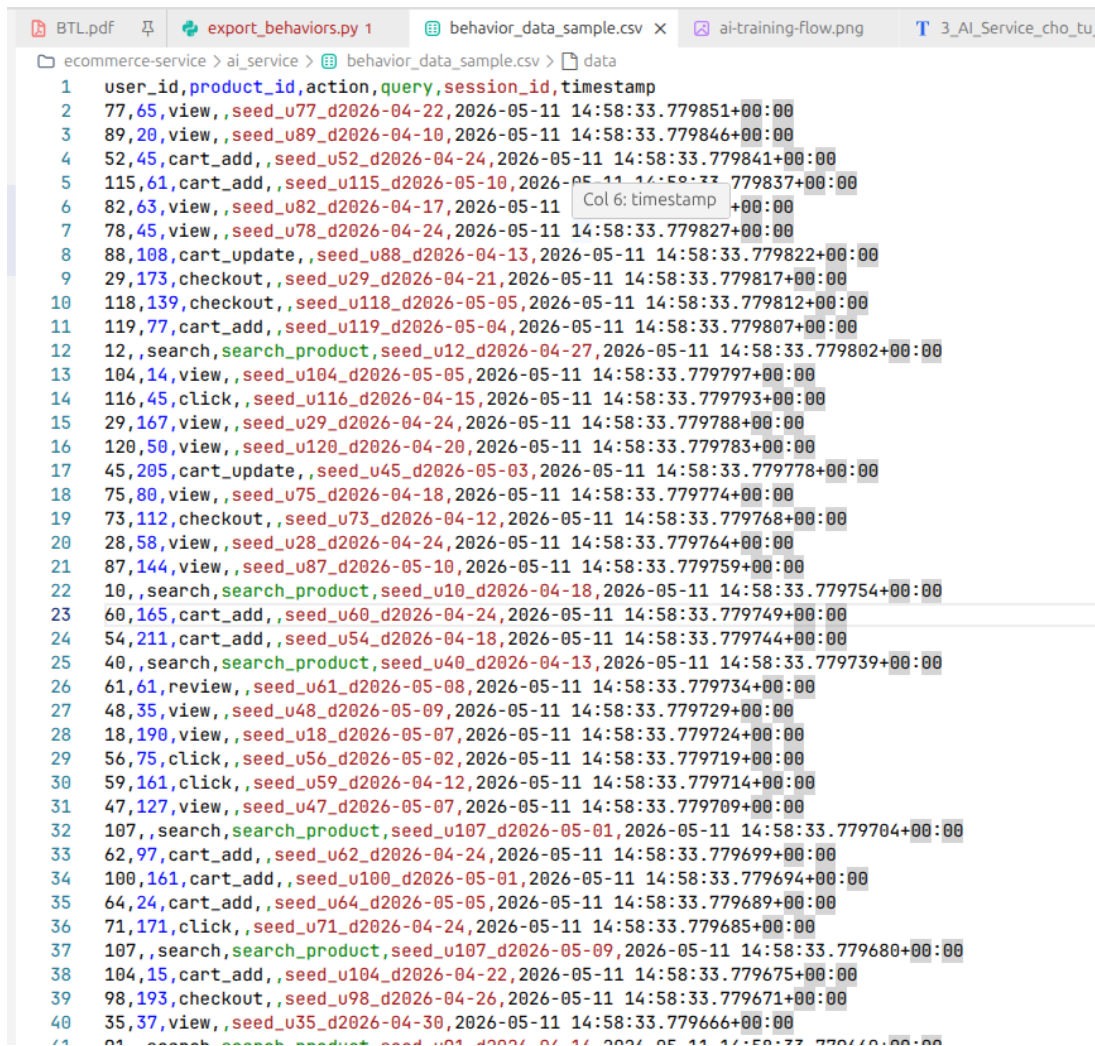
1. **Session Building:** Chuyển đổi dữ liệu thô từ `BehaviorEvent` thành các chuỗi hành vi (Keyword: `SessionBuilder`).
2. **Encoding:** Mã hóa các ID sản phẩm và hành động thành dạng số (Keyword: `SequenceEncoder`).
3. **Model Building:** Khởi tạo kiến trúc LSTM đa đầu ra (Keyword: `build_behavior_lstm`).

```
ecommerce-service > ai_service > app > training > lstm_model.py > ...
5 logger = logging.getLogger(__name__)
6
7
8 def build_behavior_lstm(vocab_size, embed_dim=None, hidden_units=None, seq_len=None):
9     """
10     S4: Build LSTM model with TensorFlow Keras
11
12     Multi-task outputs:
13     - next_item: predict next product ID
14     - intent: predict session intent (buy/browse)
15     - session_embedding: extract user session embedding
16     """
17
18     embed_dim = embed_dim or settings.EMBEDDING_DIM
19     hidden_units = hidden_units or settings.HIDDEN_UNITS
20     seq_len = seq_len or settings.MAX_SEQ_LEN
21
22     try:
23         # Input layer
24         inputs = tf.keras.Input(shape=(seq_len,), name='sequence_input')
25
26         # Embedding layer
27         x = tf.keras.layers.Embedding(vocab_size, embed_dim)(inputs)
28
29         # Stacked LSTM layers (2 layers)
30         x = tf.keras.layers.LSTM(hidden_units, hidden_units)(x)
31
32         lstm_out = tf.keras.layers.LSTM(hidden_units, hidden_units)(x)
33
34         # S6: Next-Item Prediction head
35         next_item = tf.keras.layers.Dense(vocab_size)(lstm_out)
36
37         # S5: Intent Prediction head (buy=1, browse=0)
38         intent = tf.keras.layers.Dense(2)(lstm_out)
39
40         # Build model with multiple outputs
41         model = tf.keras.Model(inputs, [next_item, intent])
42
43         logger.info(f"Built LSTM model: vocab_size={vocab_size}, embed_dim={embed_dim}, hidden_units={hidden_units}")
44
45         return model
46
47     except Exception as e:
48         logger.error(f"LSTM model building error: {e}")
49         raise
```

Hình 3.2: Mô phỏng quy trình và mã nguồn huấn luyện mô hình

3.2.3 Dữ liệu thử nghiệm hiện tại

Dữ liệu hành vi người dùng được sử dụng để huấn luyện bao gồm các trường thông tin: `user_id`, `product_id`, `action`, `session_id` (đã được xuất ra file `behavior_data_sample.csv`).



```

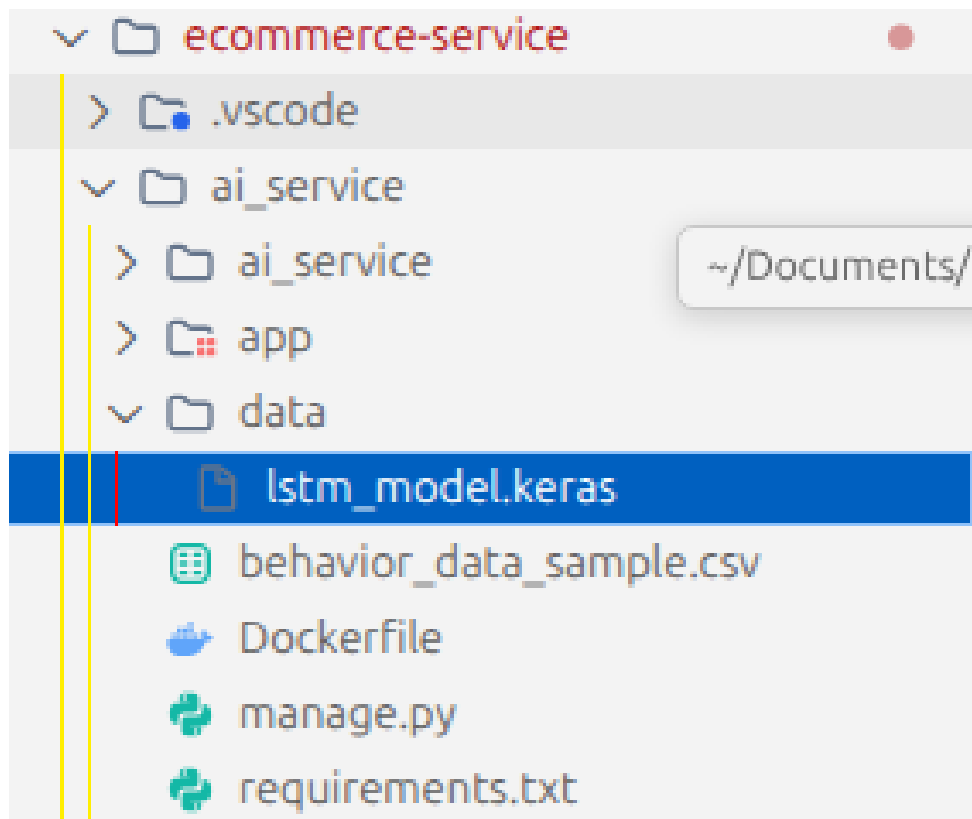
ecommerce-service > ai_service > behavior_data_sample.csv > data
1 user_id,product_id,action,query,session_id,timestamp
2 77,65,view,,seed_u77_d2026-04-22,2026-05-11 14:58:33.779851+00:00
3 89,20,view,,seed_u89_d2026-04-10,2026-05-11 14:58:33.779846+00:00
4 52,45,card_add,,seed_u52_d2026-04-24,2026-05-11 14:58:33.779841+00:00
5 115,61,card_add,,seed_u115_d2026-05-10,2026-05-11 14:58:33.779837+00:00
6 82,63,view,,seed_u82_d2026-04-17,2026-05-11 14:58:33.779827+00:00
7 78,45,view,,seed_u78_d2026-04-24,2026-05-11 14:58:33.779827+00:00
8 88,108,card_update,,seed_u88_d2026-04-13,2026-05-11 14:58:33.779822+00:00
9 29,173,checkout,,seed_u29_d2026-04-21,2026-05-11 14:58:33.779817+00:00
10 118,139,checkout,,seed_u118_d2026-05-05,2026-05-11 14:58:33.779812+00:00
11 119,77,card_add,,seed_u119_d2026-05-04,2026-05-11 14:58:33.779807+00:00
12 12,,search,search_product,seed_u12_d2026-04-27,2026-05-11 14:58:33.779802+00:00
13 104,14,view,,seed_u104_d2026-05-05,2026-05-11 14:58:33.779797+00:00
14 116,45,click,,seed_u116_d2026-04-15,2026-05-11 14:58:33.779793+00:00
15 29,167,view,,seed_u29_d2026-04-24,2026-05-11 14:58:33.779788+00:00
16 120,50,view,,seed_u120_d2026-04-20,2026-05-11 14:58:33.779783+00:00
17 45,205,card_update,,seed_u45_d2026-05-03,2026-05-11 14:58:33.779778+00:00
18 75,80,view,,seed_u75_d2026-04-18,2026-05-11 14:58:33.779774+00:00
19 73,112,checkout,,seed_u73_d2026-04-12,2026-05-11 14:58:33.779768+00:00
20 28,58,view,,seed_u28_d2026-04-24,2026-05-11 14:58:33.779764+00:00
21 87,144,view,,seed_u87_d2026-05-10,2026-05-11 14:58:33.779759+00:00
22 10,,search,search_product,seed_u10_d2026-04-18,2026-05-11 14:58:33.779754+00:00
23 60,165,card_add,,seed_u60_d2026-04-24,2026-05-11 14:58:33.779749+00:00
24 54,211,card_add,,seed_u54_d2026-04-18,2026-05-11 14:58:33.779744+00:00
25 40,,search,search_product,seed_u40_d2026-04-13,2026-05-11 14:58:33.779739+00:00
26 61,61,review,,seed_u61_d2026-05-08,2026-05-11 14:58:33.779734+00:00
27 48,35,view,,seed_u48_d2026-05-09,2026-05-11 14:58:33.779729+00:00
28 18,190,view,,seed_u18_d2026-05-07,2026-05-11 14:58:33.779724+00:00
29 56,75,click,,seed_u56_d2026-05-02,2026-05-11 14:58:33.779719+00:00
30 59,161,click,,seed_u59_d2026-04-12,2026-05-11 14:58:33.779714+00:00
31 47,127,view,,seed_u47_d2026-05-07,2026-05-11 14:58:33.779709+00:00
32 107,,search,search_product,seed_u107_d2026-05-01,2026-05-11 14:58:33.779704+00:00
33 62,97,card_add,,seed_u62_d2026-04-24,2026-05-11 14:58:33.779699+00:00
34 100,161,card_add,,seed_u100_d2026-05-01,2026-05-11 14:58:33.779694+00:00
35 64,24,card_add,,seed_u64_d2026-05-05,2026-05-11 14:58:33.779689+00:00
36 71,171,click,,seed_u71_d2026-04-24,2026-05-11 14:58:33.779685+00:00
37 107,,search,search_product,seed_u107_d2026-05-09,2026-05-11 14:58:33.779680+00:00
38 104,15,card_add,,seed_u104_d2026-04-22,2026-05-11 14:58:33.779675+00:00
39 98,193,checkout,,seed_u98_d2026-04-26,2026-05-11 14:58:33.779671+00:00
40 35,37,view,,seed_u35_d2026-04-30,2026-05-11 14:58:33.779666+00:00
41 01,,search,search_product,seed_u01_d2026-04-14,2026-05-11 14:58:33.779661+00:00

```

Hình 3.3: Dữ liệu hành vi mẫu được sử dụng để huấn luyện mô hình

3.3 Triển khai (Deploy)

Mô hình sau khi huấn luyện được đóng gói và triển khai dưới dạng một microservice độc lập. Các mô hình và embedding được nạp vào bộ nhớ khi service khởi động (Singleton pattern) để đảm bảo tốc độ phản hồi nhanh.



Hình 3.4: Cấu trúc thư mục triển khai AI Service

3.4 Kiến trúc RAG (Retrieval-Augmented Generation)

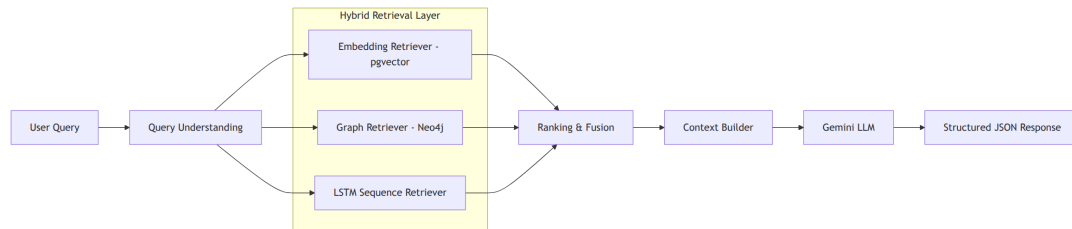
Hệ thống RAG là trái tim của chatbot tư vấn, kết hợp giữa khả năng truy xuất mạnh mẽ của cơ sở dữ liệu và sức mạnh ngôn ngữ của LLM.

3.4.1 Pipeline RAG chi tiết

Quy trình xử lý một câu hỏi của người dùng diễn ra qua các giai đoạn sau:

1. **Query Understanding:** Phân tích ý định (intent), thực thể (entities như category, brand) và các ràng buộc (Keyword: RecommenderQueryUnderstanding).
2. **Hybrid Retrieval (3 nhánh song song):**
 - **Embedding Branch:** Tìm kiếm ngữ nghĩa trên pgvector cho các sản phẩm tương đồng.
 - **Graph Branch:** Sử dụng Neo4j để tìm sản phẩm liên quan dựa trên quan hệ đồ thị.
 - **Sequence Branch:** Phân tích 40 hành động gần nhất của người dùng để suy luận ý định hiện tại.
3. **Ranking & Fusion:** Hợp nhất kết quả từ 3 nguồn, áp dụng trọng số và bộ lọc (Keyword: RecommenderRanker).

4. **Context Construction:** Xây dựng prompt ngữ cảnh gửi cho LLM (Keyword: `ContextBuilder`).
5. **LLM Generation:** Gửi ngữ cảnh sang Gemini API để sinh câu trả lời (Keyword: `GeminiClient`).



Hình 3.5: Quy trình Hybrid Retrieval và tích hợp LLM trong hệ thống RAG

3.4.2 Cơ sở dữ liệu tri thức (Knowledge Base)

Hệ thống sử dụng mô hình lưu trữ Hybrid để tối ưu hóa việc truy xuất thông tin:

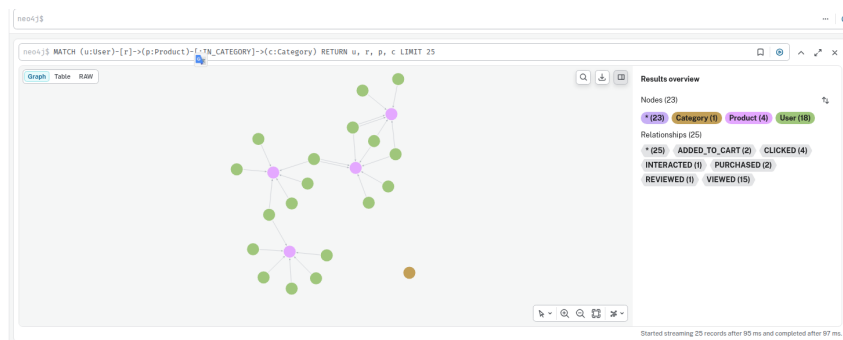
- **Vector Database (PostgreSQL + pgvector):**
 - **ProductKnowledge:** Lưu trữ vector nhúng 384 chiều của thông tin sản phẩm.
 - **FAQKnowledge:** Lưu trữ vector nhúng của các cặp câu hỏi - câu trả lời thường gặp.
 - **Kỹ thuật:** Sử dụng Index HNSW (`vector_cosine_ops`) để thực hiện tìm kiếm láng giềng gần nhất (ANN) với tốc độ cao.
- **Knowledge Graph (Neo4j):** Đồ thị tri thức đóng vai trò quan trọng trong việc mô hình hóa các mối quan hệ phức tạp giữa người dùng và sản phẩm, cho phép thực hiện các truy vấn gợi ý dựa trên sự kết nối (Link Analysis).

a, Mô hình đồ thị

Hệ thống xây dựng đồ thị với các thành phần chính:

- **Node (Thực thể):**
 - * **User:** Đại diện cho người dùng với các thuộc tính như `id`, `username`.
 - * **Product:** Đại diện cho sản phẩm với các thuộc tính như `id`, `name`.
- **Edge (Quan hệ):**
 - * **VIEW:** Người dùng xem sản phẩm.
 - * **BUY:** Người dùng đã mua sản phẩm (Action: `checkout`).
 - * **SIMILAR:** Quan hệ giữa các sản phẩm có đặc điểm tương đồng (được

tính toán từ embedding hoặc cùng danh mục).



Hình 3.6: Mô hình đồ thị tri thức người dùng - sản phẩm

b, Ví dụ câu lệnh Cypher

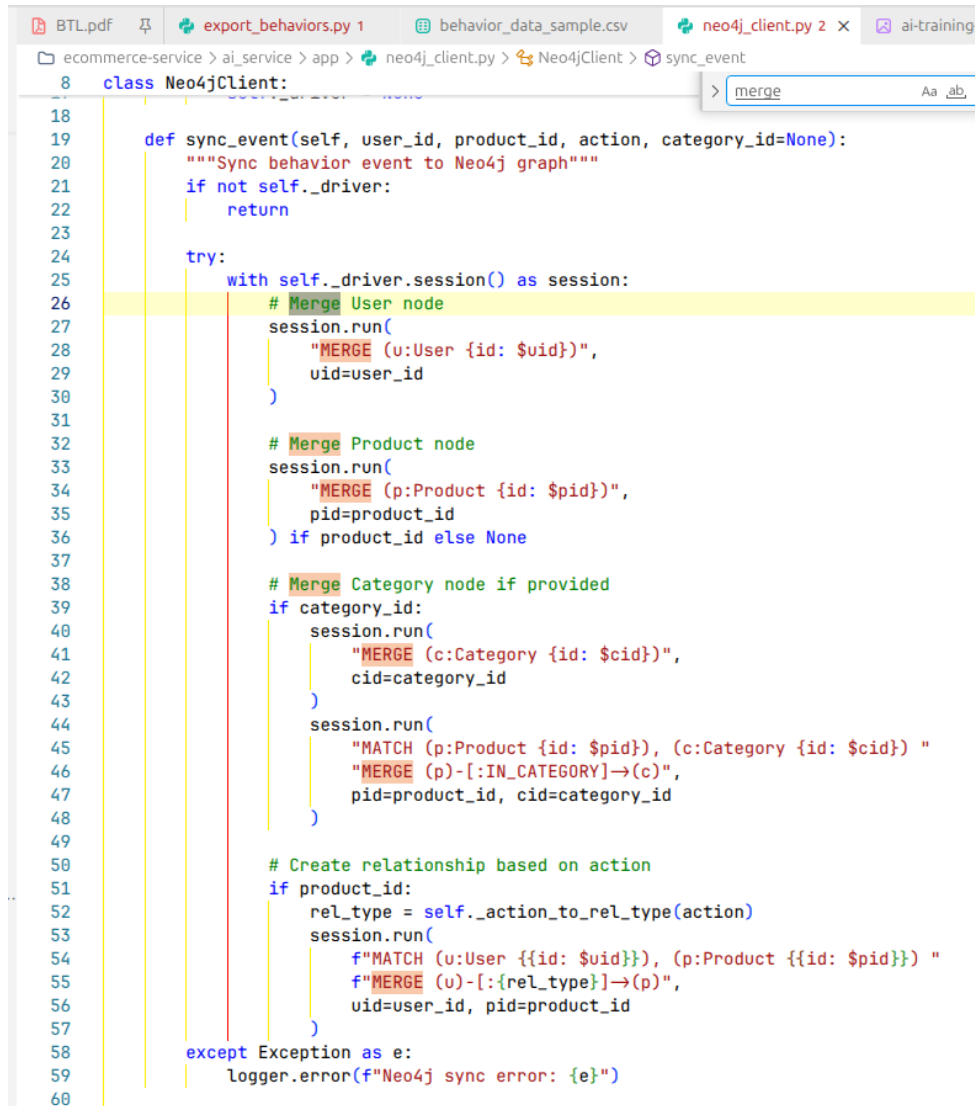
Dữ liệu được đồng bộ và khởi tạo bằng các câu lệnh Cypher (Keyword: Neo4jClient):

- Khởi tạo quan hệ mua hàng:

```

1 // Merge User node
2 MERGE (u:User {id: $user_id})
3
4 // Merge Product node
5 MERGE (p:Product {id: $product_id})
6
7 // Create relationship based on action
8 MATCH (u:User {id: $user_id}), (p:Product {id: $product_id})
9 MERGE (u)-[:PURCHASED]->(p)
10

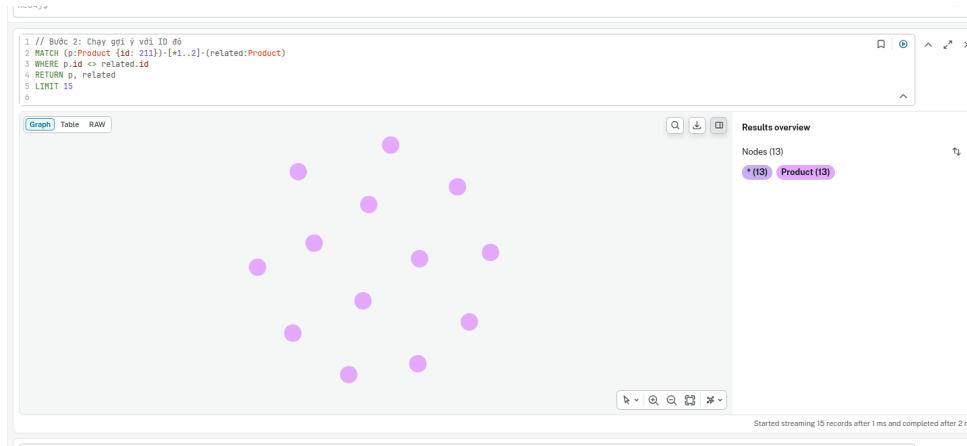
```



Hình 3.7: Câu lệnh Cypher tạo thực thể và quan hệ

- **Truy vấn gợi ý (Recommendation Query):** Dựa trên hành vi mua hàng và sự tương đồng của cộng đồng người dùng:

```
1 MATCH (p:Product {id: $pid})-[*1..2]-(related:Product)
2 WHERE p.id <> $pid
3 RETURN DISTINCT related.id as product_id
4 LIMIT 20
5
```

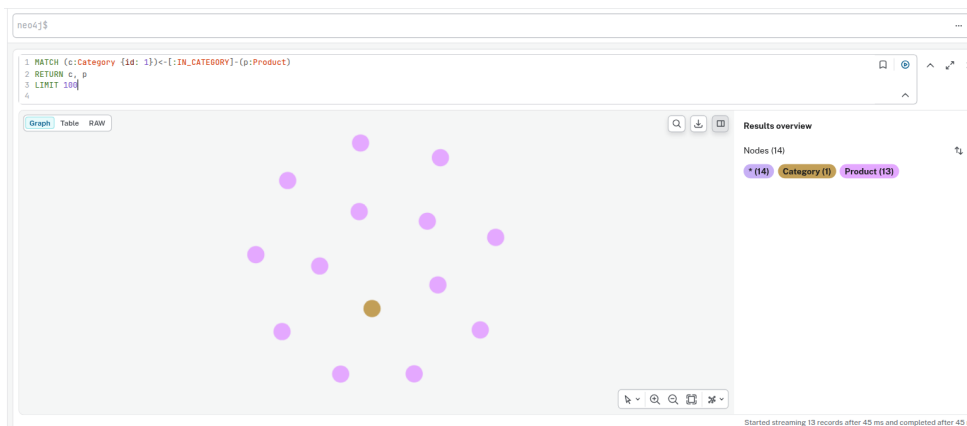


Hình 3.8: Truy vấn gợi ý sản phẩm dựa trên quan hệ đồ thị

c, Truy vấn sản phẩm theo danh mục và thuộc tính

Bên cạnh gợi ý cá nhân hóa, Neo4j còn hỗ trợ truy vấn nhanh các sản phẩm cùng danh mục hoặc cùng thương hiệu dựa trên cấu trúc cây phân cấp:

```
1 MATCH (p:Product)-[:IN_CATEGORY]->(c:Category {id: $cat_id})
2 RETURN p.id, p.name, c.name
3 LIMIT 10
4
```



Hình 3.9: Truy vấn lọc sản phẩm qua quan hệ IN_CATEGORY

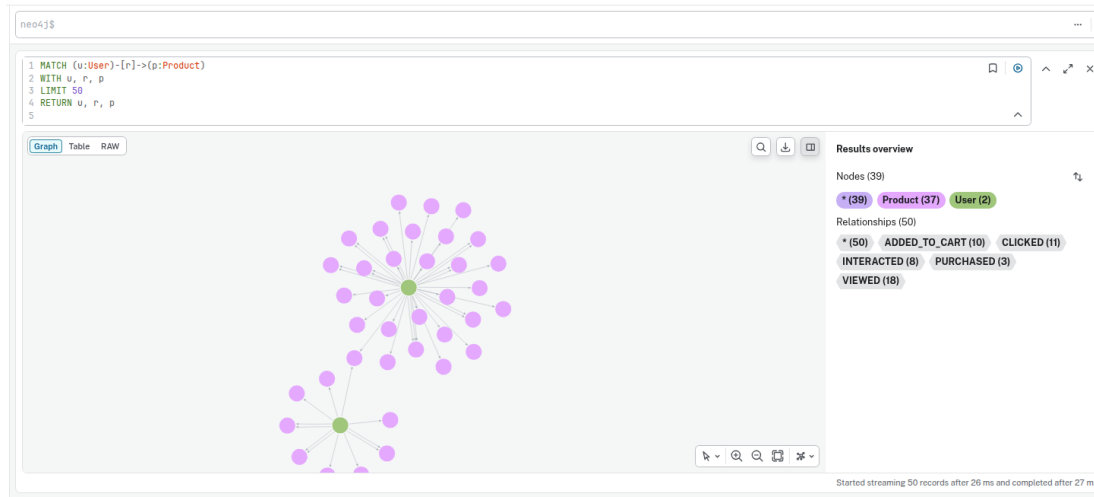
d, Trực quan hóa tương tác người dùng đa chiều

Đồ thị thực tế với hàng ngàn quan hệ giúp AI Service nhận diện được các cụm người dùng có sở thích tương đồng (Community Detection) thông qua các sản phẩm họ cùng tương tác:

```

1 MATCH (u:User)-[r]->(p:Product)
2 WHERE u.id IN [1, 75, 78, 88]
3 RETURN u, r, p
4 LIMIT 50
5

```



Hình 3.10: Minh họa các cụm tương tác đa người dùng trên đồ thị

3.4.3 Chiến lược Hybrid Retrieval Fusion

Hệ thống áp dụng chiến lược kết hợp điểm số có trọng số từ ba nguồn tín hiệu bổ sung cho nhau:

Nhánh	Điểm mạnh	Vai trò
Embedding	Hiểu ngữ nghĩa sâu của câu hỏi.	Tìm sản phẩm khớp yêu cầu kỹ thuật.
Graph	Phản ánh mối quan hệ giữa các thực thể.	Tìm sản phẩm gợi ý liên quan (Upsell/Cross-sell).
LSTM	Dự đoán ý định theo chuỗi thời gian.	Cá nhân hóa theo hành vi gần đây nhất.

Bảng 3.1: Vai trò của các nhánh trong Hybrid Retrieval

3.4.4 Cơ chế Dual-Pipeline (Fast vs Full)

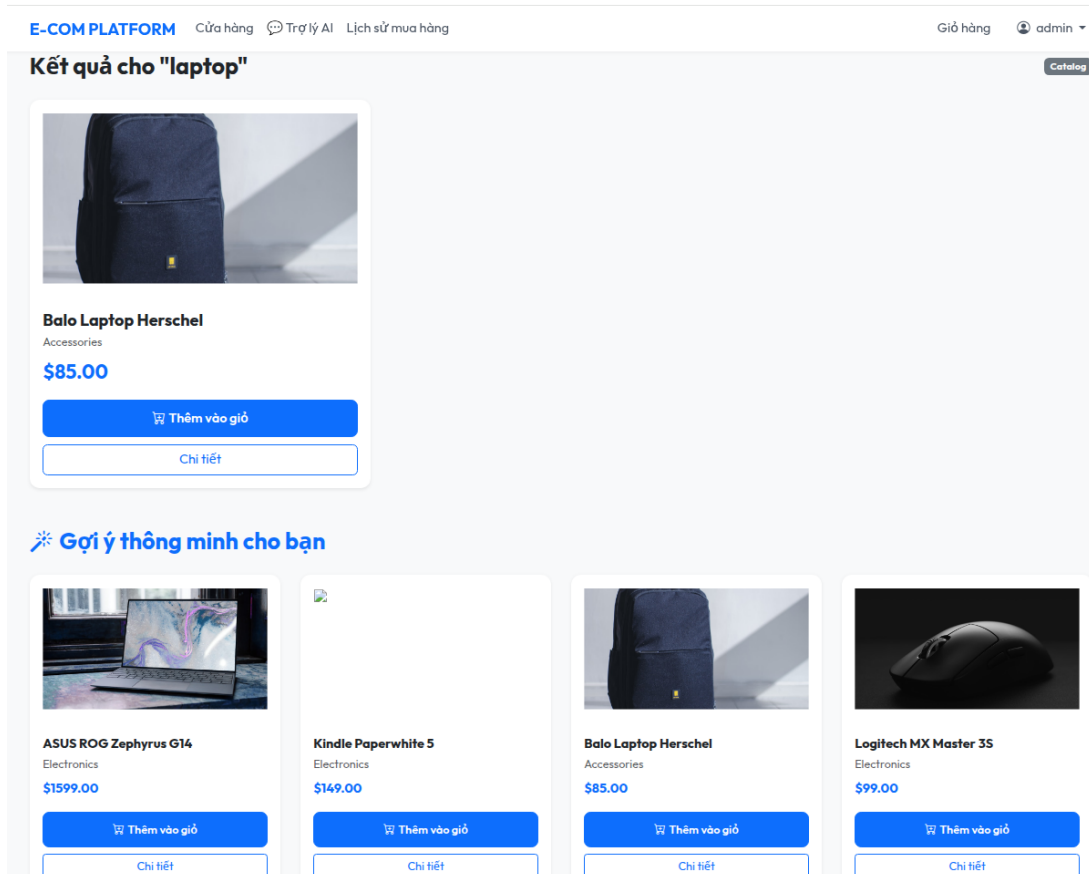
Để tối ưu hóa chi phí và tốc độ, hệ thống phân tách hai chế độ vận hành:

- **Chế độ Fast (Quick Recommendation):** Chỉ thực hiện Query Understanding rule-based và Retrieval, không gọi LLM. Dùng cho các widget gợi ý sản phẩm tại trang chủ.
- **Chế độ Full (Chatbot Consultation):** Thực hiện đầy đủ pipeline, sử dụng LLM để sinh ngôn ngữ tự nhiên. Dùng cho giao diện chat tư vấn.

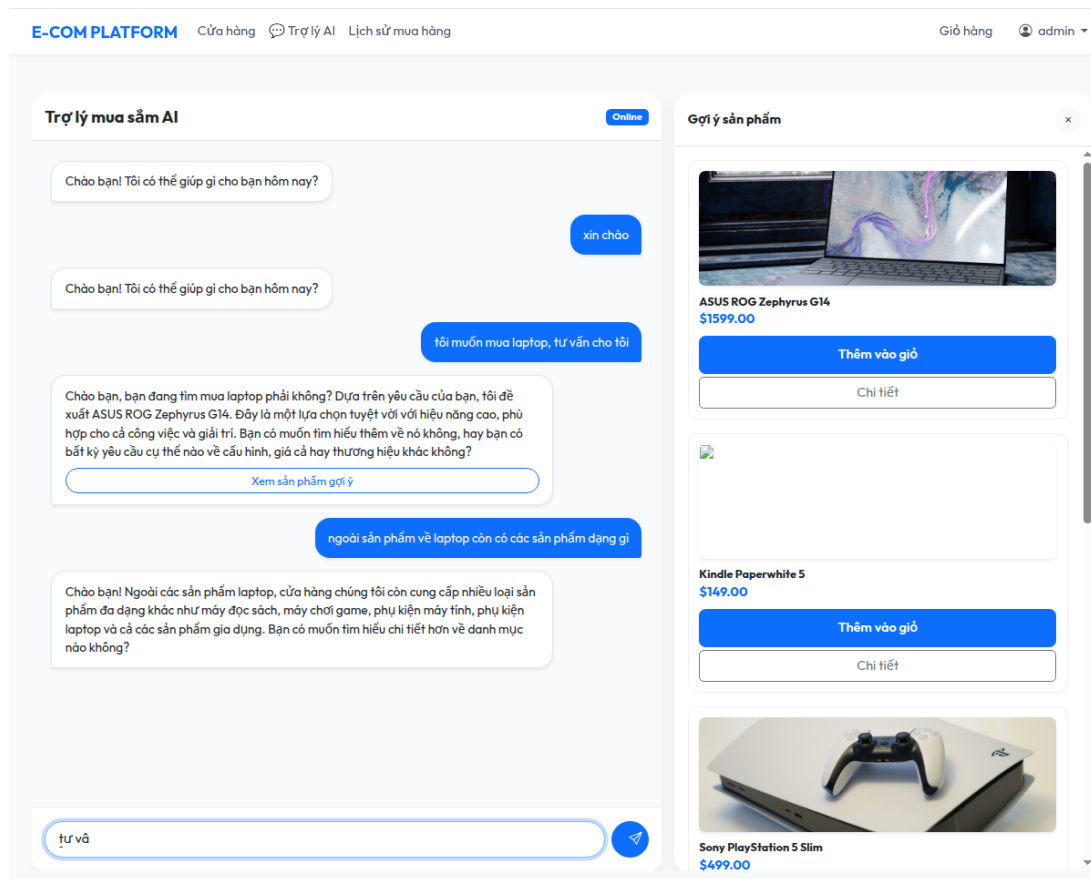
3.4.5 Tích hợp Deep Learning vào RAG

Kết quả từ mô hình LSTM giúp LLM biết được người dùng đang ở giai đoạn nào của hành trình mua sắm (đang xem chơi hay sắp chốt đơn) để đưa ra câu trả lời phù hợp. Điều này giúp hệ thống tránh được việc tư vấn máy móc và tăng tỷ lệ chuyển đổi.

3.5 Tích hợp E-commerce



Hình 3.11: Giao diện hiển thị sản phẩm gợi ý tại trang chủ



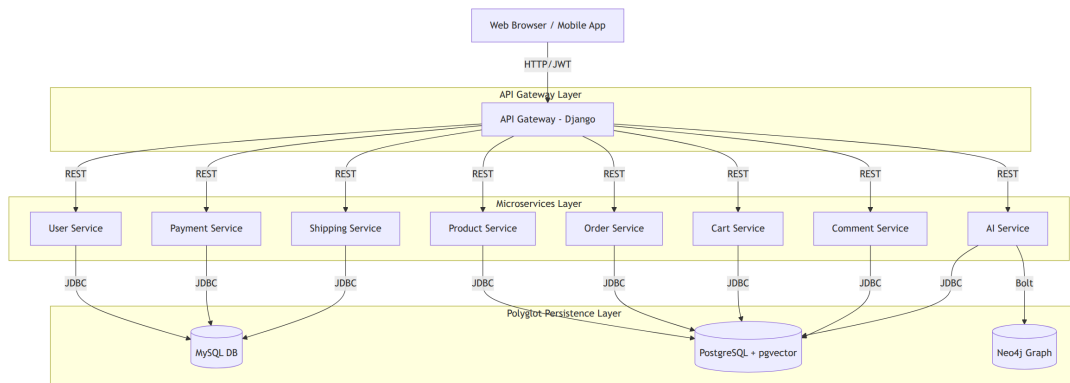
Hình 3.12: Giao diện Chatbot tương tác và tư vấn sản phẩm

CHƯƠNG 4. Xây dựng hệ thống hoàn chỉnh

4.1 Mô tả kiến trúc hệ thống

4.1.1 Mô hình hệ thống (Microservices Architecture)

Hệ thống được xây dựng theo kiến trúc Microservices hiện đại, trong đó mỗi nghiệp vụ cốt lõi được tách thành một dịch vụ độc lập. Điều này giúp hệ thống có khả năng mở rộng linh hoạt, dễ bảo trì và cô lập lỗi hiệu quả.



Hình 4.1: Mô hình kiến trúc Microservices của hệ thống ecom-final

Các thành phần chính bao gồm:

- **API Gateway:** Điểm vào duy nhất cho mọi yêu cầu từ phía client, chịu trách nhiệm điều phối (Routing), xác thực (Authentication) và tổng hợp dữ liệu.
- **Business Services:** Bao gồm `user-service`, `product-service`, `order-service`, `cart-service`, `payment-service`, `shipping-service`, `comment-service` và `staff-service`. Mỗi service là một dự án Django độc lập.
- **AI Service:** Dịch vụ chuyên biệt xử lý các bài toán về gợi ý và chatbot tư vấn thông minh.

4.1.2 Nguyên tắc thiết kế cốt lõi

Hệ thống tuân thủ nghiêm ngặt các nguyên lý thiết kế Microservices:

- **Database per Service:** Mỗi dịch vụ quản lý cơ sở dữ liệu riêng, không truy cập trực tiếp vào DB của dịch vụ khác để đảm bảo tính đóng gói (Keyword: `docker-compose.yml services`).
- **Giao tiếp qua REST API:** Các dịch vụ tương tác với nhau thông qua giao thức HTTP/JSON (Keyword: `api_gateway/app/views.py service_call`).

- **Loose Coupling:** Các dịch vụ có tính liên kết lỏng lẻo, có thể triển khai và nâng cấp độc lập.

4.2 Các công nghệ sử dụng

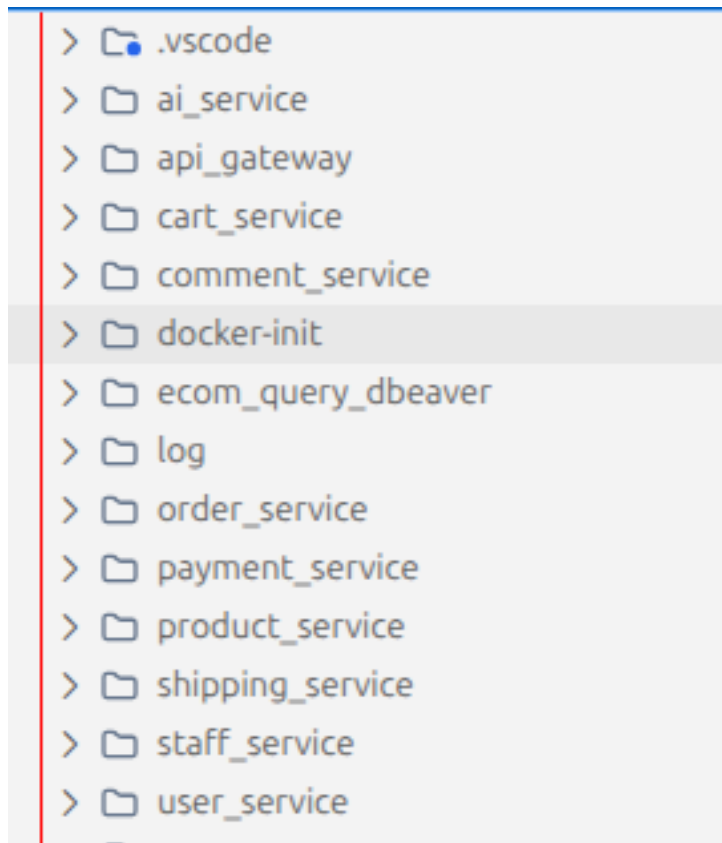
Hệ thống tích hợp nhiều công nghệ hiện đại nhằm tối ưu hóa hiệu năng và trải nghiệm người dùng:

- **Backend Framework:** Django & Django REST Framework (DRF) cho tất cả các microservices.
- **Database Systems:**
 - **PostgreSQL + pgvector:** Lưu trữ dữ liệu sản phẩm, đơn hàng và vector nhúng cho AI.
 - **MySQL:** Lưu trữ dữ liệu người dùng, thanh toán và vận chuyển.
 - **Neo4j:** Xây dựng đồ thị tri thức (Knowledge Graph) phục vụ gợi ý sản phẩm.
- **Authentication:** JSON Web Token (JWT) được triển khai tùy chỉnh để xác thực người dùng xuyên suốt các service (Keyword: `jwt_utils.py`).
- **Infrastructure:** Docker & Docker Compose dùng để container hóa và quản lý vòng đời triển khai hệ thống.

4.3 Triển khai hệ thống

4.3.1 Cấu trúc dự án (System Structure)

Dự án được tổ chức thành các thư mục riêng biệt cho từng microservice, giúp dễ dàng quản lý mã nguồn và cấu hình Docker.



Hình 4.2: Cấu trúc tổ chức mã nguồn hệ thống

4.3.2 Triển khai với Docker và Docker Compose

Toàn bộ hệ thống được container hóa để đảm bảo tính nhất quán giữa các môi trường phát triển và vận hành.



Hình 4.3: Cấu hình điều phối các dịch vụ bằng Docker Compose

4.3.3 Điều phối tại API Gateway

API Gateway đóng vai trò là "người dẫn đường", phân phối các request đến đúng service đích dựa trên URL.

```
ecommerce-service > api_gateway > app > views.py > SERVICES
8 from django.views.decorators.csrf import csrf_exempt
9 from jwt_utils import create_tokens, verify_token, extract_token_from_auth_header, get_token
10
11 logger = logging.getLogger(__name__)
12
13 def debug_log(msg):
14     sys.stderr.write(f"{msg}\n")
15     sys.stderr.flush()
16
17
18 > class CustomRequests:
19
20
21
22
23
24
25
26
27
28
29 requests = CustomRequests()
30 from django.shortcuts import render, redirect
31 from django.http import JsonResponse, HttpResponse
32 from rest_framework.decorators import api_view, permission_classes
33 from rest_framework.permissions import AllowAny
34
35
36 SERVICES = {
37     'product': 'http://product_service:8000',
38     'user': 'http://user_service:8000',
39     'cart': 'http://cart_service:8000',
40     'order': 'http://order_service:8000',
41     'comment': 'http://comment_service:8000',
42     'payment': 'http://payment_service:8000',
43     'shipping': 'http://shipping_service:8000',
44     'staff': 'http://staff_service:8000',
45     'ai': 'http://aiservice:8000',
46 }
47
48
49 def _fire_behavior_event(user_id, action, product_id=None, **meta):
50     """Fire-and-forget pattern: emit behavior event to AI service"""
51     def _send():
52         try:
53             requests.post(
54                 f"{SERVICES['ai']}/api/v1/ai/events/",
55                 json={
56                     'user_id': user_id,
57                     'action': action,
58                     'product_id': product_id,
59                     'metadata': meta,
60                 },
61                 timeout=2,
62             )
63         except Exception:
64             pass # Silently ignore errors - fire-and-forget
65
66     _send()
67
68
69
70
71
72
73
74
```

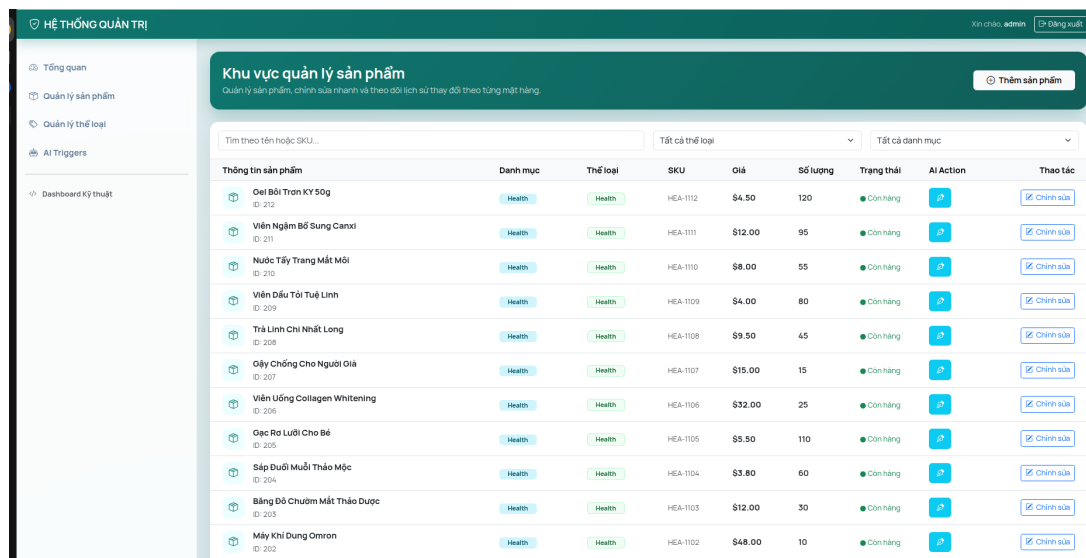
Hình 4.4: Cấu hình Routing và Proxy tại API Gateway

4.4 Kết quả triển khai

Sau khi triển khai thành công với Docker Compose, hệ thống hoạt động ổn định và các dịch vụ có thể giao tiếp thông suốt.

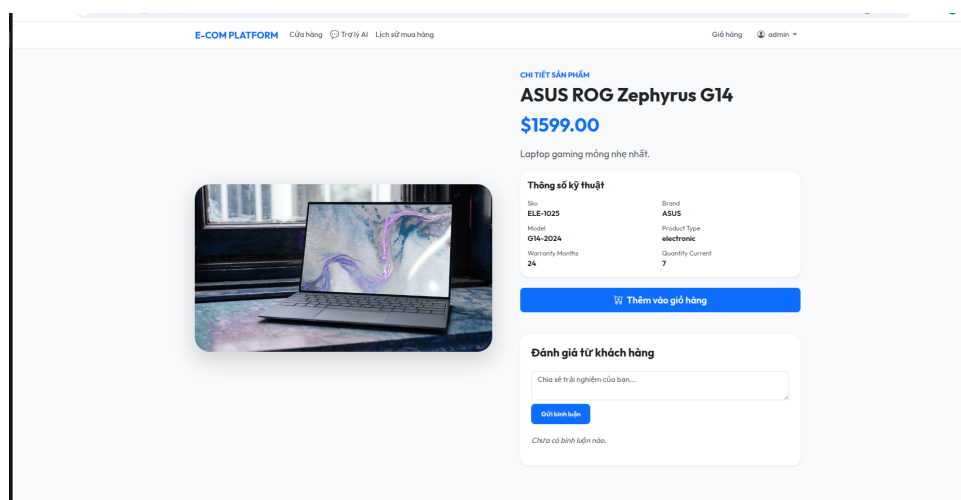
4.4.1 Giao diện Quản trị và Dashboard

Hệ thống cung cấp các giao diện quản trị trực quan giúp nhân viên vận hành hệ thống hiệu quả. Dưới đây là giao diện quản lý danh sách sản phẩm tập trung:



Hình 4.5: Giao diện quản lý sản phẩm tại trung tâm vận hành

Giao diện chỉnh sửa thông tin chi tiết sản phẩm cho phép cập nhật giá, mô tả và các thuộc tính động (Dynamic Attributes) một cách linh hoạt:



Hình 4.6: Tính năng chỉnh sửa thông tin sản phẩm và thuộc tính động

4.4.2 Luồng mua hàng hoàn chỉnh (End-to-End)

Kết quả kiểm thử cho thấy luồng mua hàng hoạt động chính xác từ khâu đăng nhập, chọn sản phẩm đến khi thanh toán thành công.

E-COM PLATFORM

Cửa hàng Trợ lý AI Lịch sử mua hàng

Giỏ hàng admin

1

Địa chỉ giao hàng

Địa chỉ giao hàng *

Lạng Sơn

Phương thức vận chuyển

Tiêu chuẩn
3-5 ngày

Nhanh
1-2 ngày

Hỏa tốc
Trong ngày

2

Thanh toán

Khí nhận hàng

Chuyển khoản

Hoàn tất đặt hàng

Tóm tắt đơn hàng

Viên Ngâm Bổ Sung Canxi

x5

\$60.00

Nước Tây Trang Mắt Mỏi

x100

\$800.00

Nước Tây Trang Mắt Mỏi

x3

\$24.00

Tạm tính (3 sản phẩm)

\$884.00

Phí vận chuyển

Miễn phí

Tổng tiền

\$884.00

Hình 4.7: Minh họa luồng thanh toán trên giao diện khách hàng

TÀI LIỆU THAM KHẢO

PHỤ LỤC